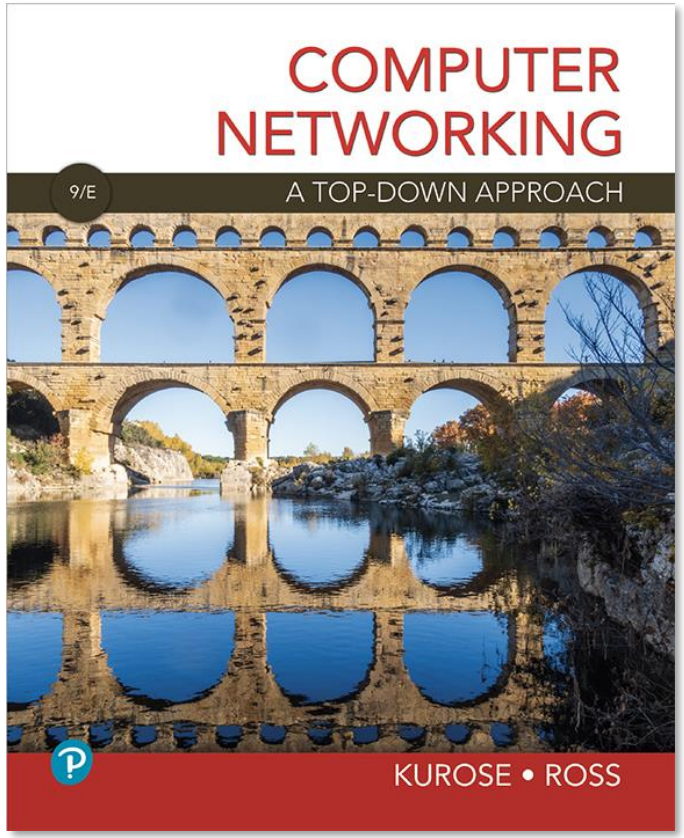
Chapter 2
Application Layer



第00讲-课程简介_哔哩哔哩_bilibili

https://www.bilibili.com/video/BV1Ut411d7RE/?p=1&vd_source=5fbc4920b6c246ce2d9126818c78b17b





Computer Networking: A Top-Down Approach

9th edition n

Jim Kurose, Keith Ross

Pearson, 2025

Application layer: overview

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS
- video streaming and content distribution networks
- socket programming with UDP and TCP



Application layer: overview

Our goals:

- conceptual *and* implementation aspects of application-layer protocols
  - transport-layer service models
  - client-server paradigm
  - peer-to-peer paradigm
- learn about protocols by examining popular application-layer protocols and infrastructure
  - HTTP
  - SMTP, IMAP
  - DNS
  - video streaming systems, CDNs
- programming network applications
  - socket API

Some network apps

- social media
- Web
- text messaging
- e-mail
- multi-user network games
- streaming stored video (YouTube, Hulu, Netflix)

- P2P file sharing
- real-time video conferencing (e.g., Zoom)
- Internet search
- remote login
- …










Q: your favorites?













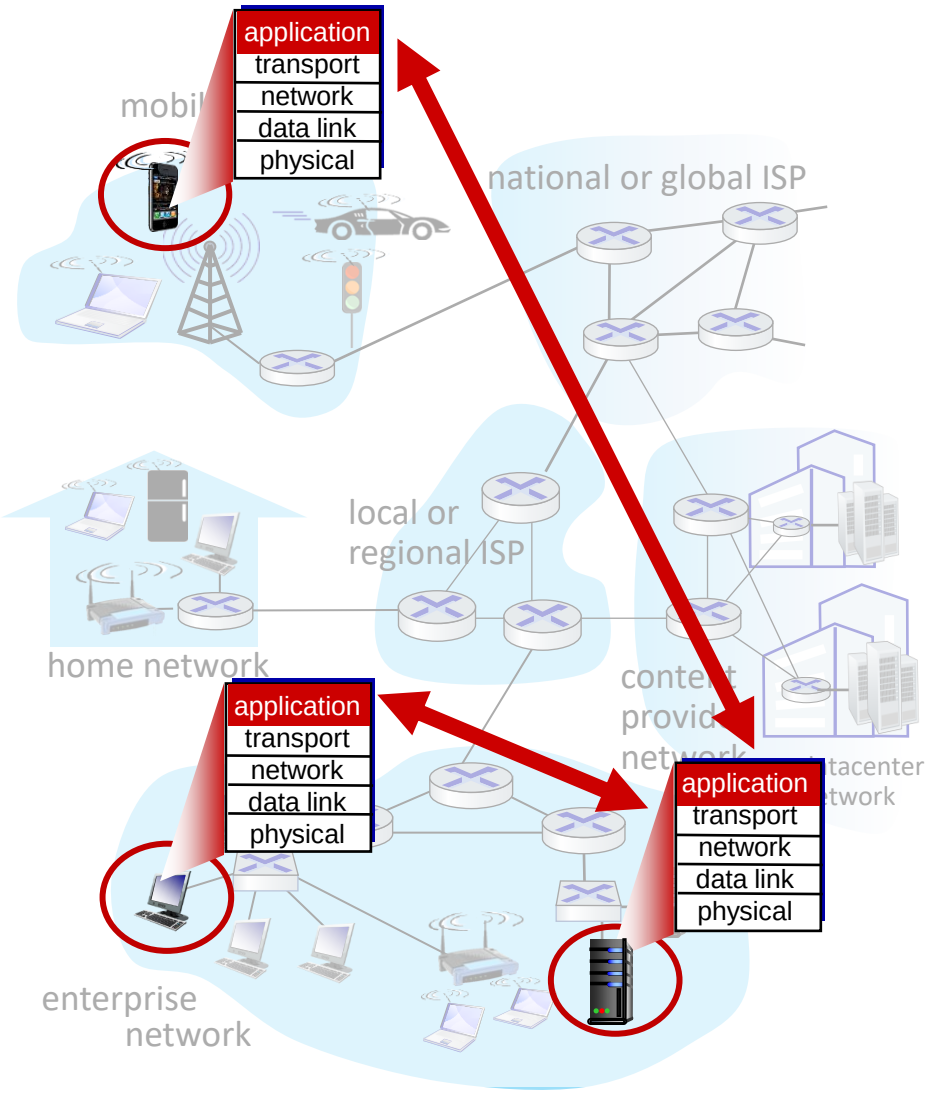
Creating a network app

write programs that:

- run on (different) end systems
- communicate over network
- e.g., web server software communicates with browser software

no need to write software for network-core devices

- network-core devices do not run user applications
- applications on end systems allows for rapid app development, propagation



Client-server paradigm

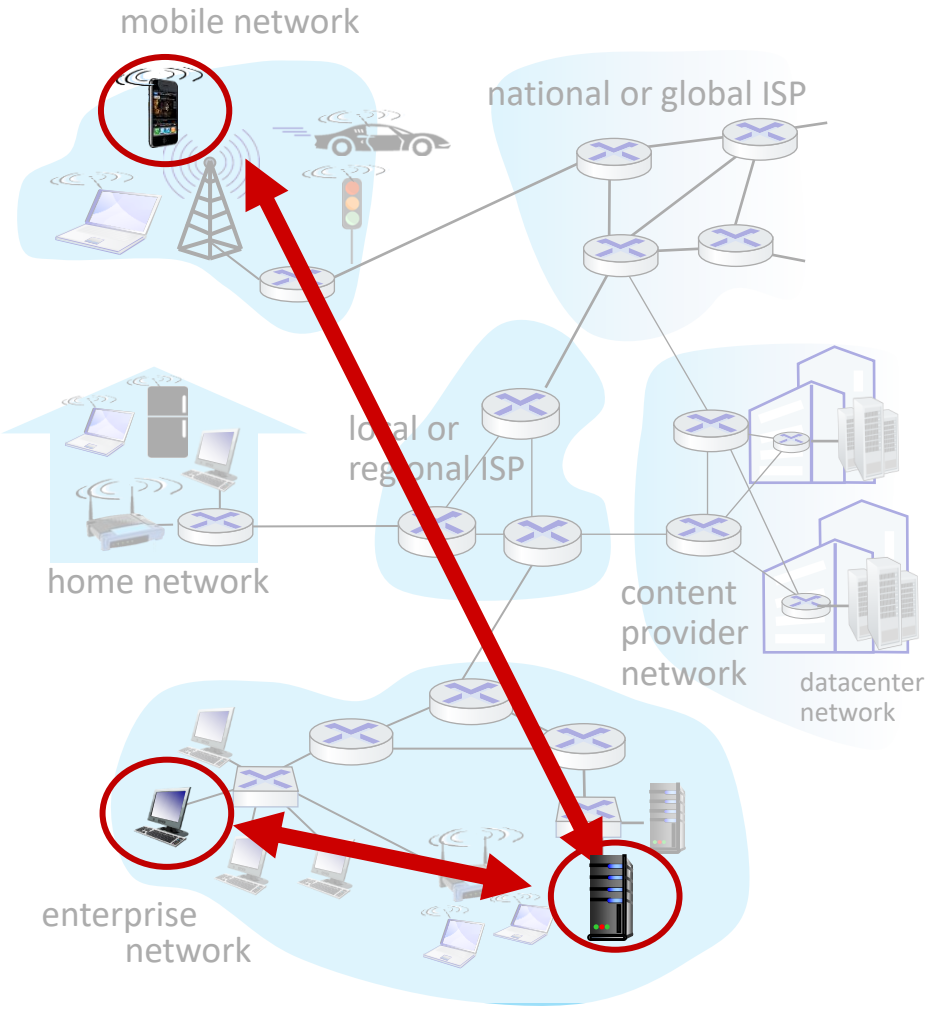
server:

- always-on host
- permanent IP address
- often in data centers, for scaling

clients:

- contact, communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do *NOT* communicate directly with each other

- examples: HTTP, IMAP, FTP

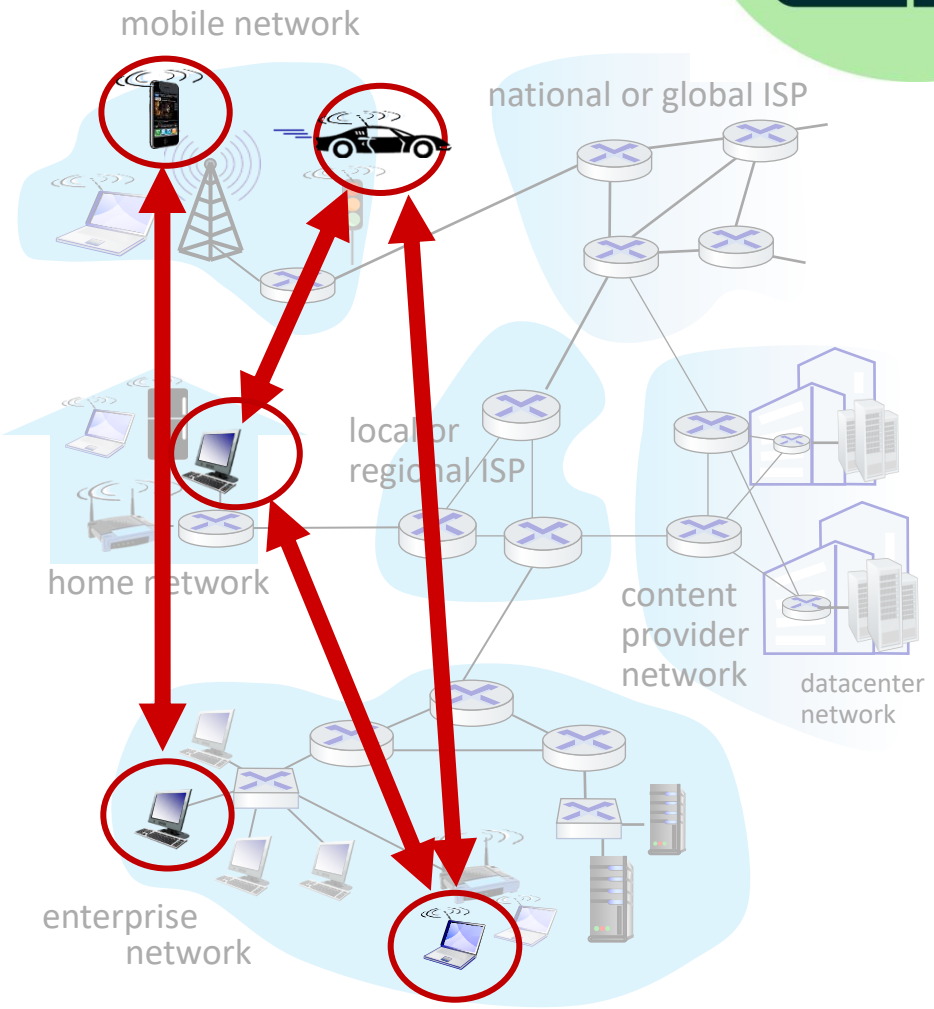


Peer-peer architecture

- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
  - *self scalability* – new peers bring new service capacity, as well as new service demands
- peers are intermittently connected and change IP addresses
  - complex management
- example: P2P file sharing [BitTorrent]

P2P compelling features





Processes communicating

process: program running within a host

- within same host, two processes communicate using inter-process communication, IPC (defined by OS)
- processes in different hosts communicate by exchanging messages

clients, servers

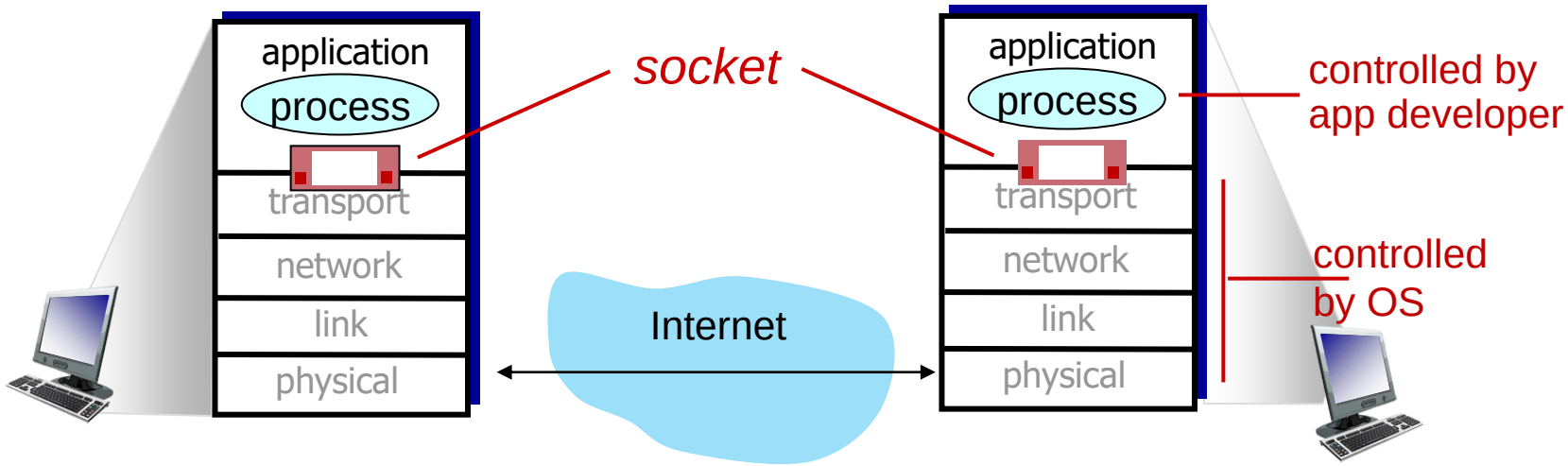
client process: process that initiates communication

server process: process that waits to be contacted

- **Note:** applications with P2P architectures have client processes & server processes

Sockets

- process sends/receives messages to/from its socket
- socket analogous to door
  - sending process shoves message out door
  - sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process
  - two sockets involved: one on each side



Addressing processes

- to receive messages, process must have *identifier*
- host device has unique 32-bit IP address
- *Q:* does IP address of host on which process runs suffice for identifying the process?
  - *A:* no, *many* processes can be running on same host

- *identifier* includes both IP address and port numbers associated with process on host.
- example port numbers:
  - HTTP server: 80
  - mail server: 25
- to send HTTP message to gaia.cs.umass.edu web server:
  - IP address: 128.119.245.12
  - port number: 80
- more shortly…

An application-layer protocol defines:

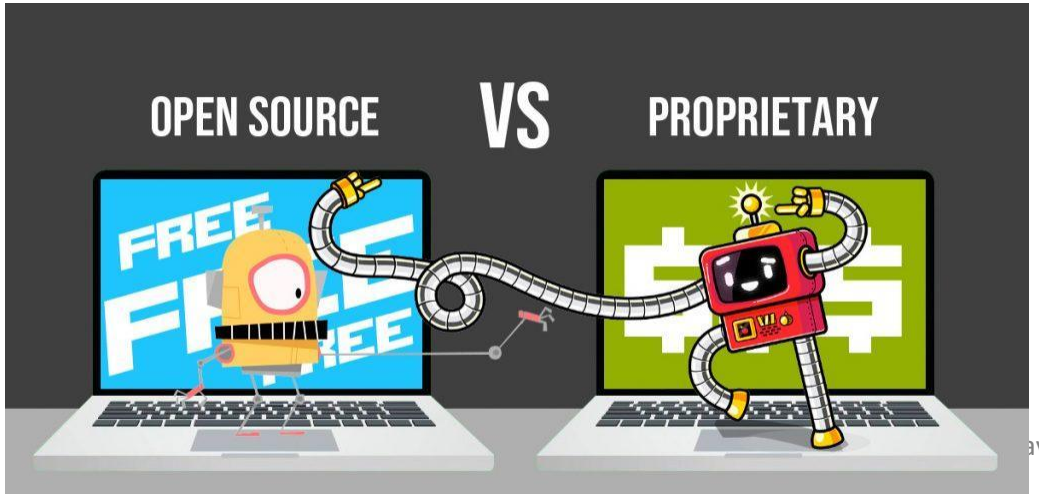
- types of messages exchanged,
  - e.g., request, response
- message syntax:
  - what fields in messages & how fields are delineated
- message semantics
  - meaning of information in fields
- rules for when and how processes send & respond to messages

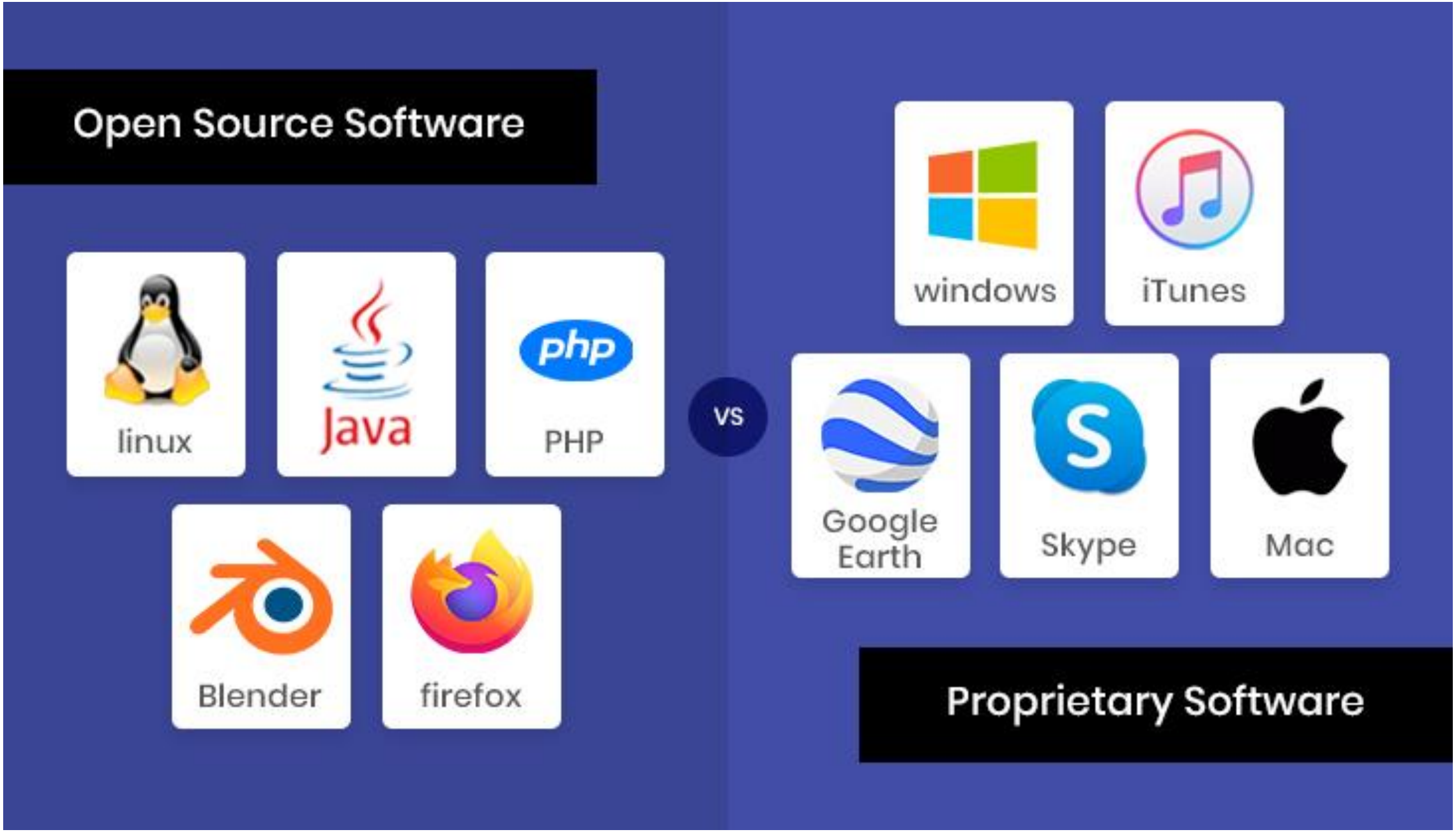
open protocols:

- defined in RFCs, everyone has access to protocol definition
- allows for interoperability
- e.g., HTTP, SMTP

proprietary protocols:

- e.g., Zoom




Open Source Software
linux
Java
PHP
Blender
firefox
vs
windows
iTunes
Google Earth
Skype
Mac
Proprietary Software

What transport service does an app need?



Reliable data transfer (rdt)

- data integrity: some apps (e.g., email, file transfer, web transactions) require 100% rdt
- other apps (e.g., audio, video) can tolerate some loss

timing

- some apps (e.g., Internet telephony, interactive games) require low delay to be "effective"

throughput

- Bandwidth-sensitive app: some apps (e.g., multimedia) require minimum amount of throughput to be "effective"
- other apps ("elastic apps") make use of whatever throughput they get

security

- encryption, data integrity, ...

Transport service requirements: common apps

application	data loss	throughput	time sensitive?
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video:10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

Internet transport protocols services

TCP service:

- *reliable transport* between sending and receiving process
- *flow control:* sender won't overwhelm receiver
- *congestion control:* throttle sender when network overloaded
- *connection-oriented:* setup required between client and server processes
- *does not provide:* timing, minimum throughput guarantee, security

UDP service:

- *unreliable data transfer* between sending and receiving process
- *does not provide:* reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup.

Q: why bother? *Why* is there a UDP?

Internet applications, and transport protocols

application	application layer protocol	transport protocol
file transfer/download	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP 1.1 [RFC 7320, 9110]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary	TCP or UDP
streaming audio/video	HTTP [RFC 7320], DASH	TCP
interactive games	WOW, FPS (proprietary)	UDP or TCP

Securing TCP

Vanilla TCP & UDP sockets:

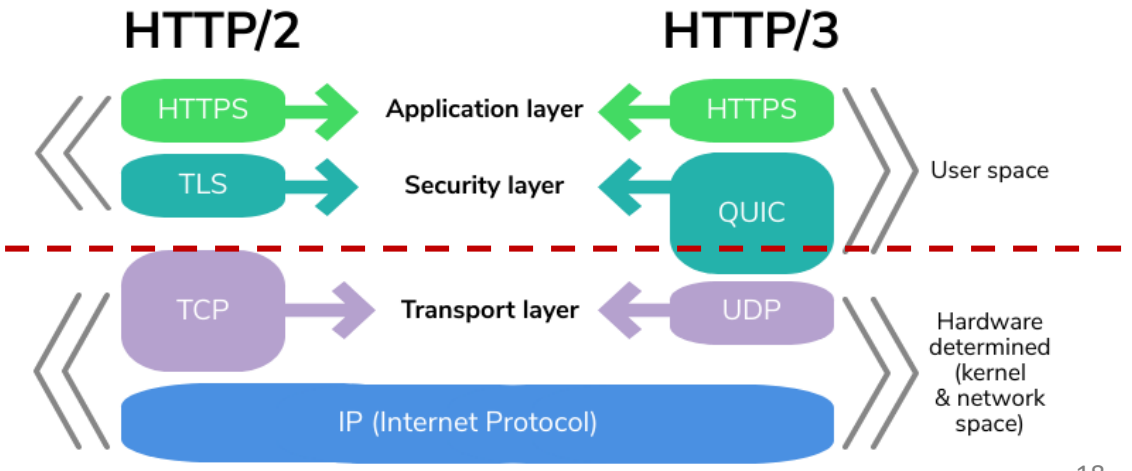
- no encryption
- cleartext passwords sent into socket traverse Internet in cleartext (!)

Transport Layer Security (TLS)

- provides encrypted TCP connections
- data integrity
- end-point authentication

TLS implemented in application layer

- apps use TLS libraries, that use TCP in turn
- cleartext sent into "socket" traverse Internet *encrypted*



Application layer: overview

- Principles of network applications
- **Web and HTTP**
- E-mail, SMTP, IMAP
- The Domain Name System DNS
- video streaming and content distribution networks
- socket programming with UDP and TCP



Web and HTTP

First, a quick review…

- web page consists of *objects,* each of which can be stored on different Web servers
- object can be HTML file, JPEG image, Java applet, audio file,…
- web page consists of *base HTML-file* which includes *several referenced objects, each* addressable by a *URL,* e.g.,



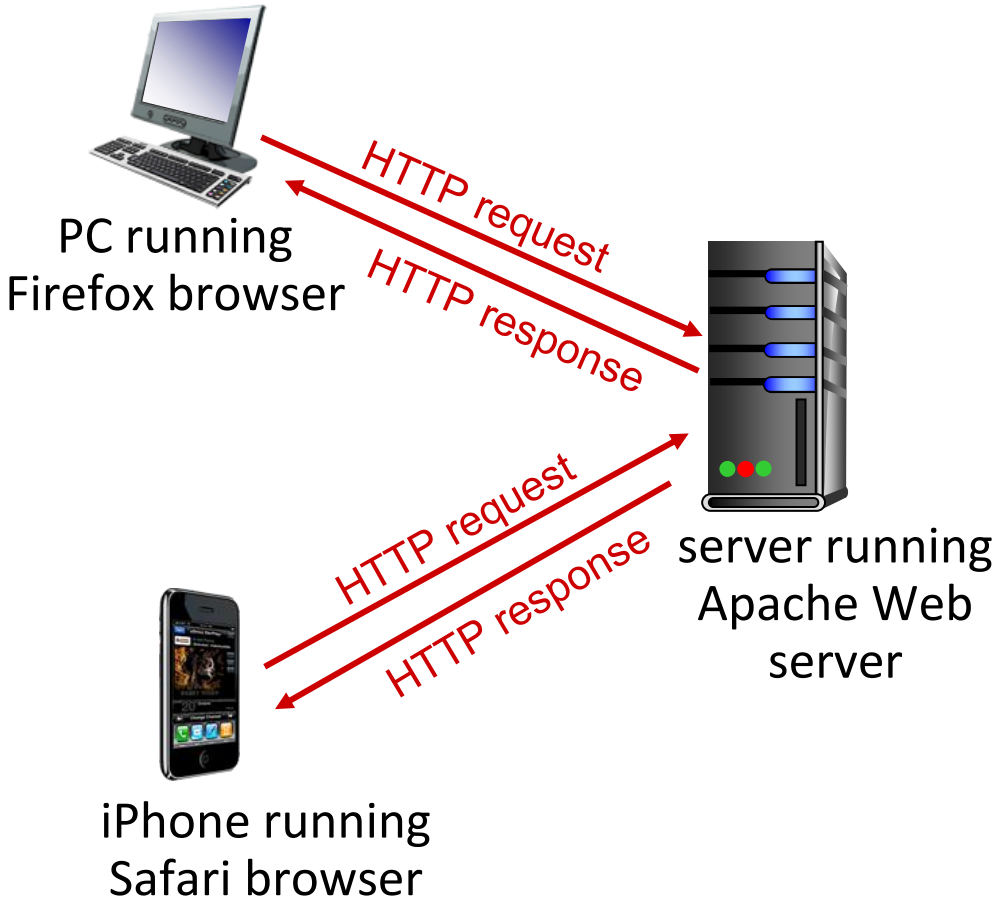
`www.someschool.edu/someDept/pic.gif`

host name | path name

HTTP overview

HTTP: hypertext transfer protocol

- Web's application-layer protocol
- client/server model:
  - *client:* browser that **requests**, **receives**, (*using HTTP protocol*) and "**displays**" Web objects
  - *server:* Web server sends (*using HTTP protocol*) objects in response to requests



HTTP overview (continued)

HTTP uses TCP:

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

HTTP is "stateless"

- server maintains *no* information about past client requests

aside

protocols that maintain "state" are complex!

- past history (state) must be maintained
- if server/client crashes, their views of "state" may be inconsistent, must be reconciled

HTTP connections: two types

Default mode of HTTP/1.0

Non-persistent HTTP

1. TCP connection opened
2. at most one object sent over TCP connection
3. TCP connection closed

downloading multiple objects required multiple connections

Default mode of HTTP/1.1+

Persistent HTTP

- TCP connection opened to a server
- multiple objects can be sent over *single* TCP connection between client, and that server
- TCP connection closed

Non-persistent HTTP: example

User enters URL: **`www.someSchool.edu/someDepartment/home.index`**
(containing text, references to 10 jpeg images)



1a. HTTP **client** initiates TCP connection to HTTP server (process) at www.someSchool.edu on port 80



1b. HTTP **server** at host www.someSchool.edu waiting for TCP connection at port 80 "accepts" connection, notifying client

2. HTTP client sends HTTP *request message* (containing URL) into TCP connection socket. Message indicates that client wants object someDepartment/home.index

3. HTTP server receives request message, forms *response message* containing requested object, and sends message into its socket

time

Non-persistent HTTP: example (cont.)

User enters URL: **`www.someSchool.edu/someDepartment/home.index`**
(containing text, references to 10 jpeg images)





4. HTTP server closes TCP connection.

5. HTTP client receives response message containing html file, displays html. Parsing html file, finds 10 referenced jpeg objects

6. Steps 1-5 repeated for each of 10 jpeg objects

time



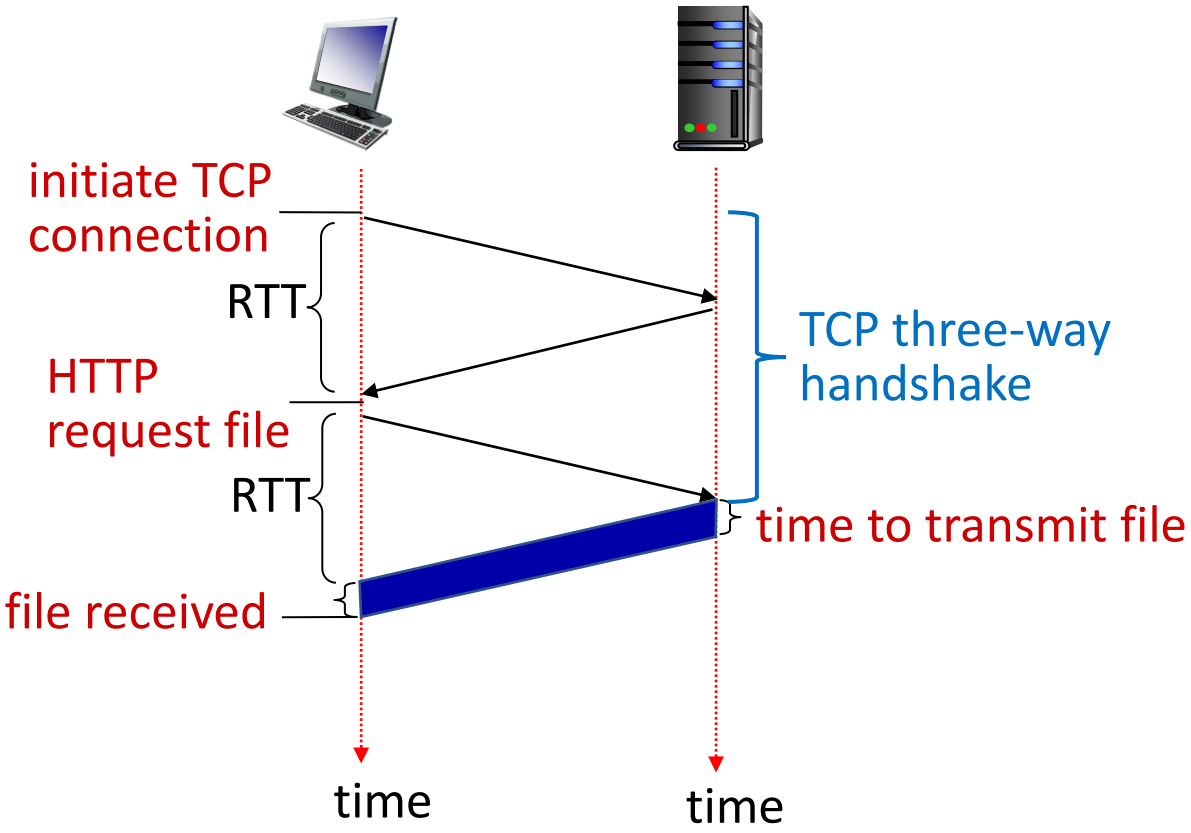


Non-persistent HTTP: response time

RTT (Round-Trip Time): time for a small packet to travel from client to server and back

HTTP response time (per object):

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- object/file transmission time



The first two parts of the three-way handshake take one RTT.

Non-persistent HTTP response time = 2RTT+ file transmission time

Persistent HTTP (HTTP 1.1)

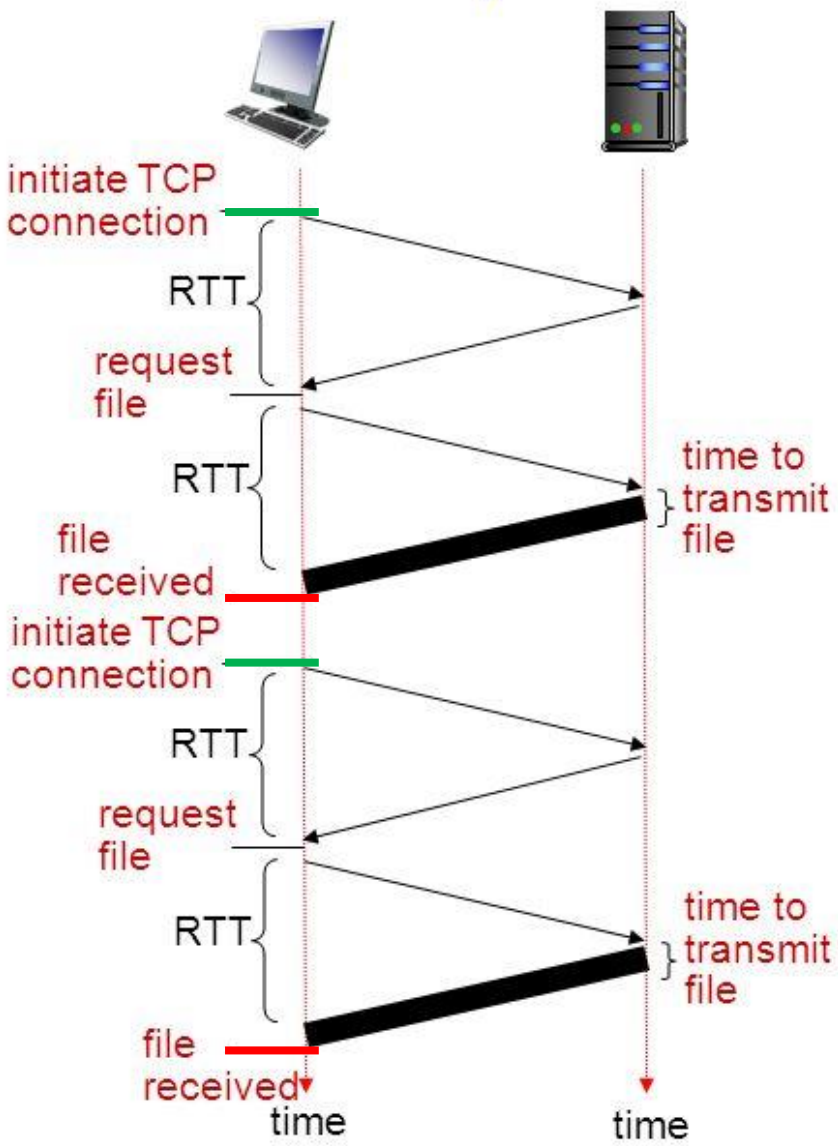
Non-persistent HTTP issues:

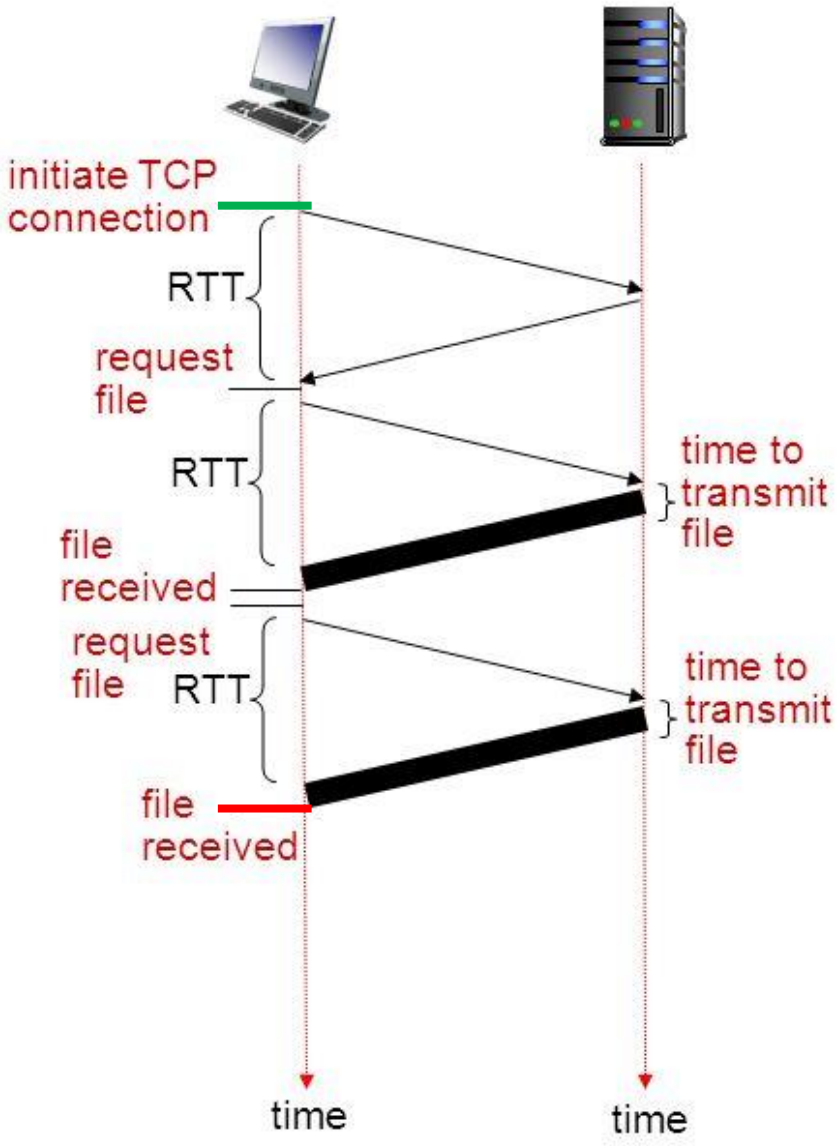
- requires 2 RTTs per object
- OS overhead for *each* TCP connection
- browsers often open multiple parallel TCP connections to fetch referenced objects in parallel

Persistent HTTP (HTTP1.1):

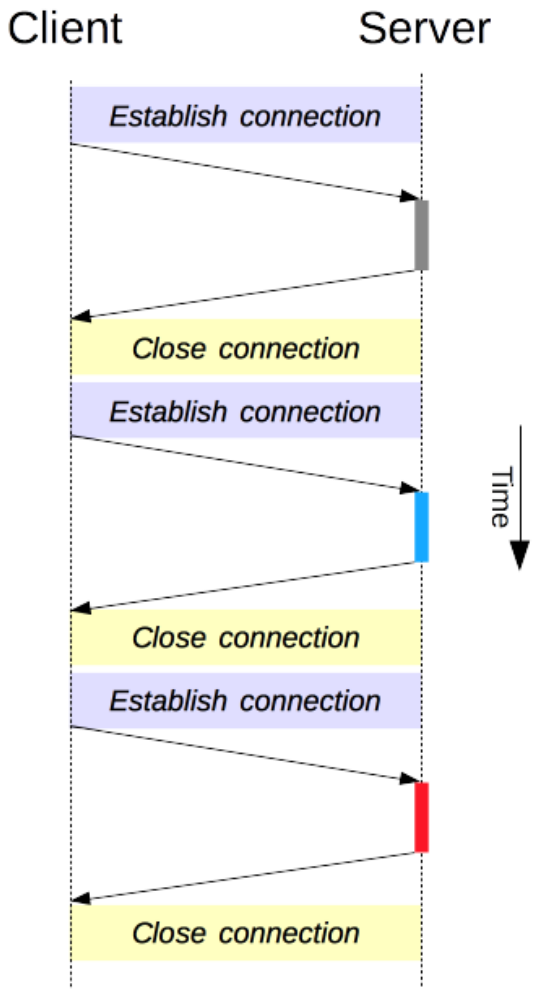
- server leaves connection open after sending response
- subsequent HTTP messages between same client/server sent over open connection
- client sends requests as soon as it encounters a referenced object
- as little as one RTT for all the referenced objects (cutting response time in half)

Non-persistent HTTP v.s. Persistent HTTP for Two Objects



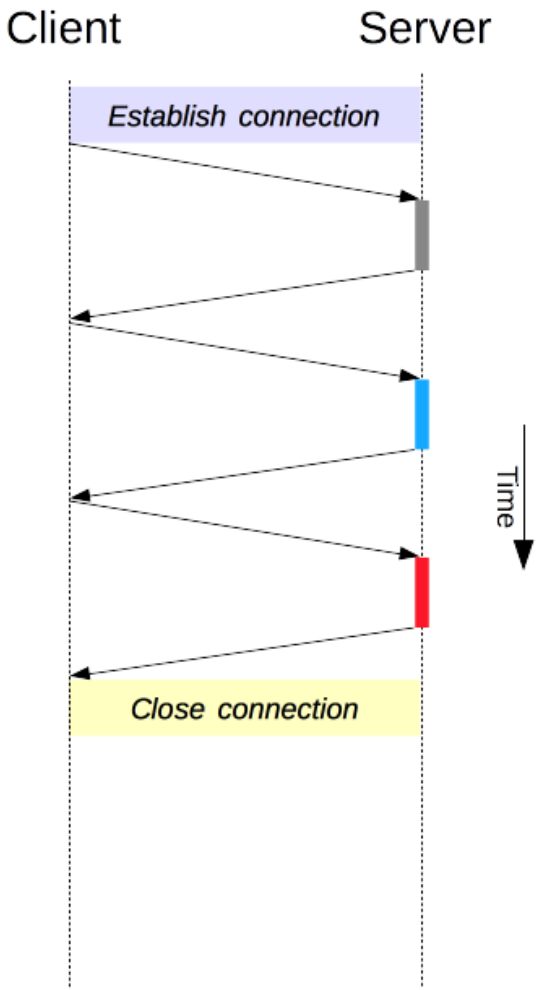


http/1.0



Short-lived connections

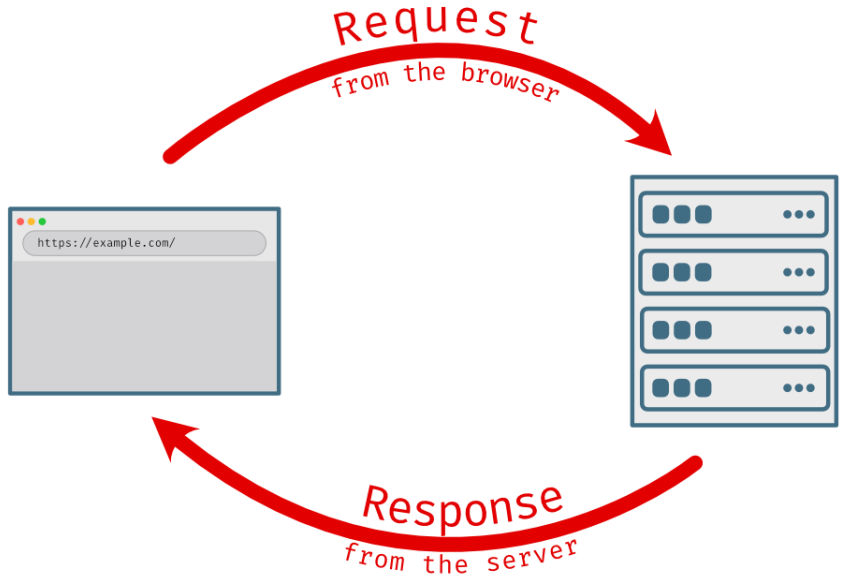
http/1.1



Persistent connection

HTTP request message

- two types of HTTP messages: ***request, response***
- HTTP request message:
  - ASCII (human-readable format)



carriage return (**cr**) character

line-feed (**lf**) character

request line (GET, POST, HEAD commands)

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X
   10.15; rv:80.0) Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n
```

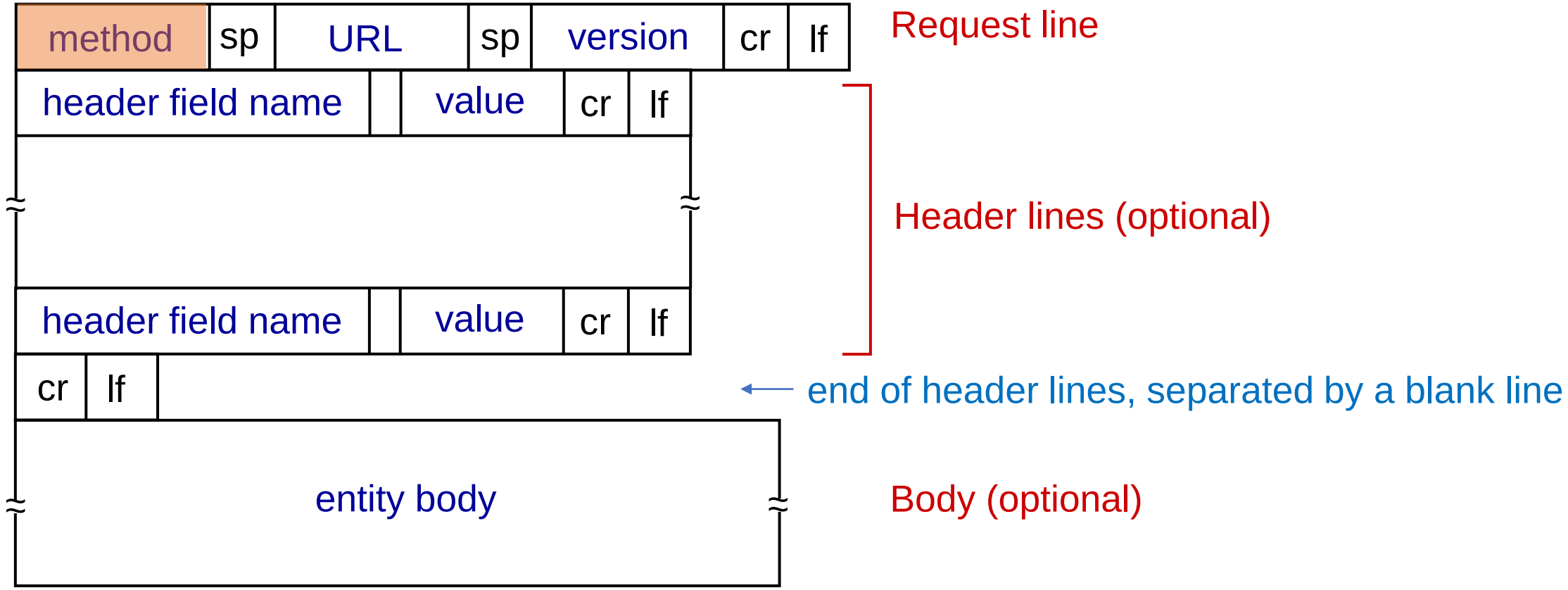
required by Web proxy caches

header lines

carriage return, line feed at start of line indicates end of header lines

HTTP request message: general format

Method field includes: **GET**, **POST**, **HEAD**, **PUT**, and **DELETE**.

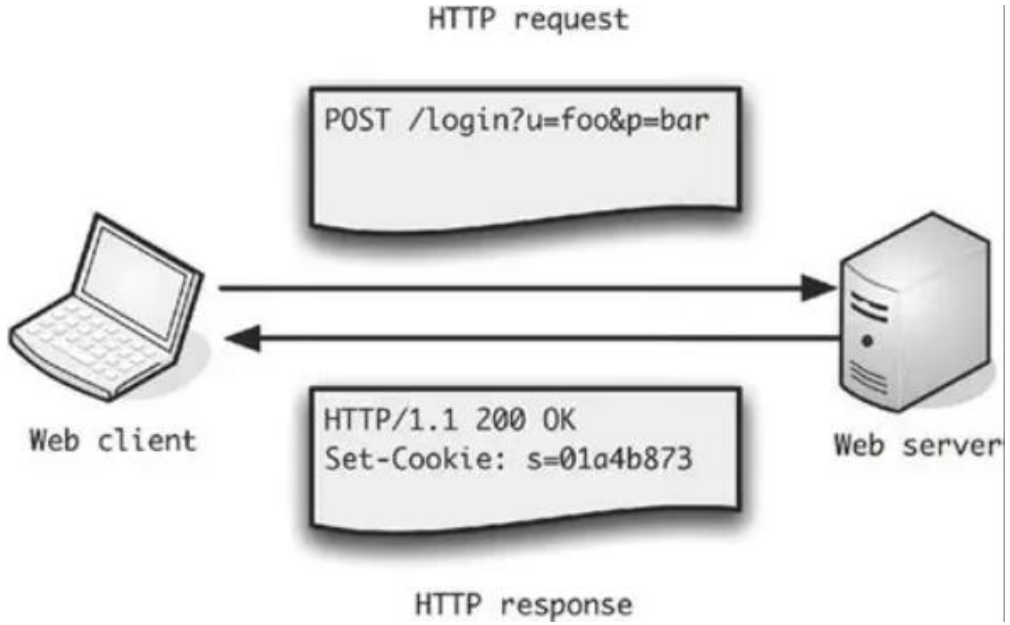


HTTP Request Methods

GET	POST	PUT	DELETE	PATCH	HEAD
retrieve data from server	add data to an existining file or resource	update(replace) an existing file or resource in server	delete data from server	update a resource partially (modify)	retrieve the resource's headers

- **CONNECT** is used to open a two-way socket connection to the remote server;
- **OPTIONS** is used to describe the communication options for specified resource;
- **TRACE** is designed for diagnostic purposes during the development.
- **HEAD** retrieves the resource's headers, without the resource itself.

HTTP response message



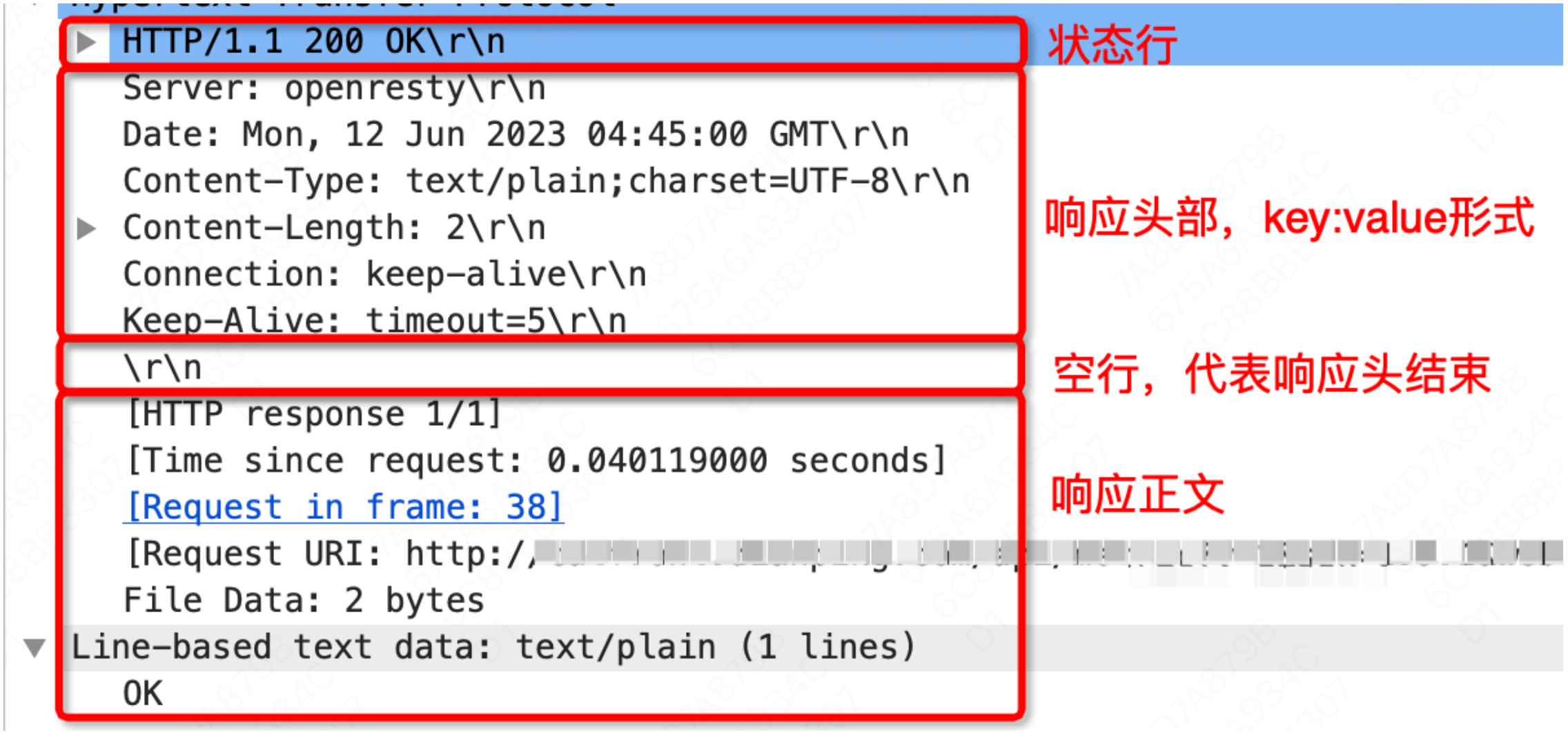
status line (protocol status code status phrase)

header lines

data, e.g., requested HTML file

```
HTTP/1.1 200 OK
Date: Tue, 08 Sep 2020 00:53:20 GMT
Server: Apache/2.4.6 (CentOS)
   OpenSSL/1.0.2k-fips PHP/7.4.9
   mod_perl/2.0.11 Perl/v5.16.3
Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT
ETag: "a5b-52d015789ee9e"
Accept-Ranges: bytes
Content-Length: 2651
Content-Type: text/html; charset=UTF-8
\r\n
data data data data data ...
```

HTTP Response example



HTTP response status codes

- status code appears in 1st line in server-to-client response message.



200 OK

- request succeeded, requested object later in this message

301 Moved Permanently

- requested object moved, new location specified later in this message (in Location: field)

400 Bad Request

- request msg not understood by server

404 Not Found

- requested document not found on this server

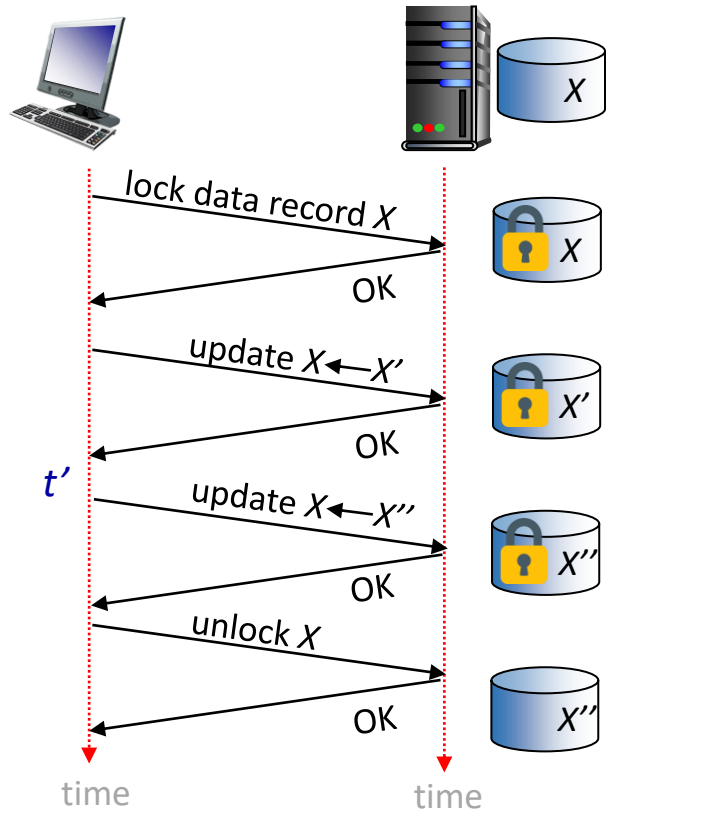
505 HTTP Version Not Supported

Maintaining user/server state: cookies

Recall: HTTP GET/response interaction is *stateless*

- This simplifies server design and has permitted engineers to develop high-performance Web servers
- no notion of multi-step exchanges of HTTP messages to complete a Web "transaction"
  - no need for client/server to track "state" of multi-step exchange
  - all HTTP requests are independent of each other
  - no need for client/server to "recover" from a partially-completed-but-never-completely-completed transaction

a stateful protocol: client makes two changes to X, or none at all



Q: what happens if network connection or client crashes at *t'* ?

Maintaining user/server state: cookies

Web sites and client browser use *cookies* to maintain some state between transactions

Cookie's four components:

1) cookie header line of HTTP *response* message
2) cookie header line in next HTTP *request* message
3) cookie file kept on user's host, managed by user's browser
4) back-end database at Web site

Example:

- Susan uses browser on laptop, visits specific e-commerce site for first time
- when initial HTTP requests arrives at site, site creates:
  - unique ID (aka "cookie")
  - entry in backend database for ID
- subsequent HTTP requests from Susan to this site will contain cookie ID value, allowing site to "identify" Susan

Maintaining user/server state: cookies



client

server

ebay 8734

cookie file

ebay 8734
amazon 1678

usual HTTP request msg

usual HTTP response
set-cookie: 1678

usual HTTP request msg
cookie: 1678

usual HTTP response msg

Amazon server creates ID 1678 for user

create entry

backend database

access

cookie-specific action

one week later:

ebay 8734
amazon 1678

usual HTTP request msg
cookie: 1678

usual HTTP response msg

access

cookie-specific action

time

time

HTTP cookies: comments

What cookies can be used for:

- authorization
- shopping carts
- recommendations
- user session state (Web e-mail)

aside

cookies and privacy:

- cookies permit sites to *learn* a lot about you on their site.
- third party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple web sites

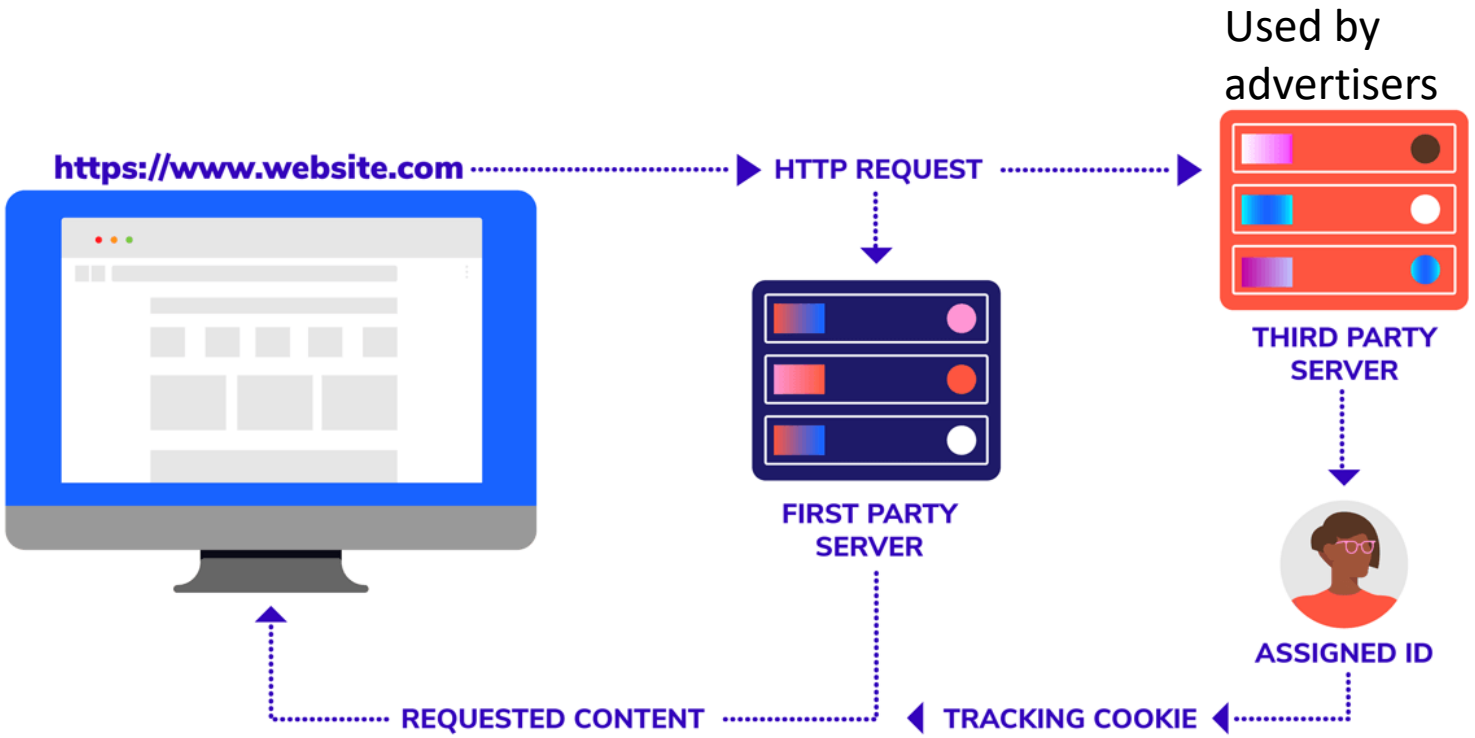
Challenge: How to keep state?

- *at protocol endpoints:* maintain state at sender/receiver over multiple transactions
- *in messages:* cookies in HTTP messages carry state





HTTP cookies





Using cookies and user-supplied account information, Amazon and other e-commerce sites can provide "one-click shopping" .

Since cookies can be used to identify a user, Cookies can thus be used to create a user session layer on top of stateless HTTP.

Although cookies often simplify the Internet shopping experience for the user, they are controversial because they can also be considered as an invasion of privacy.

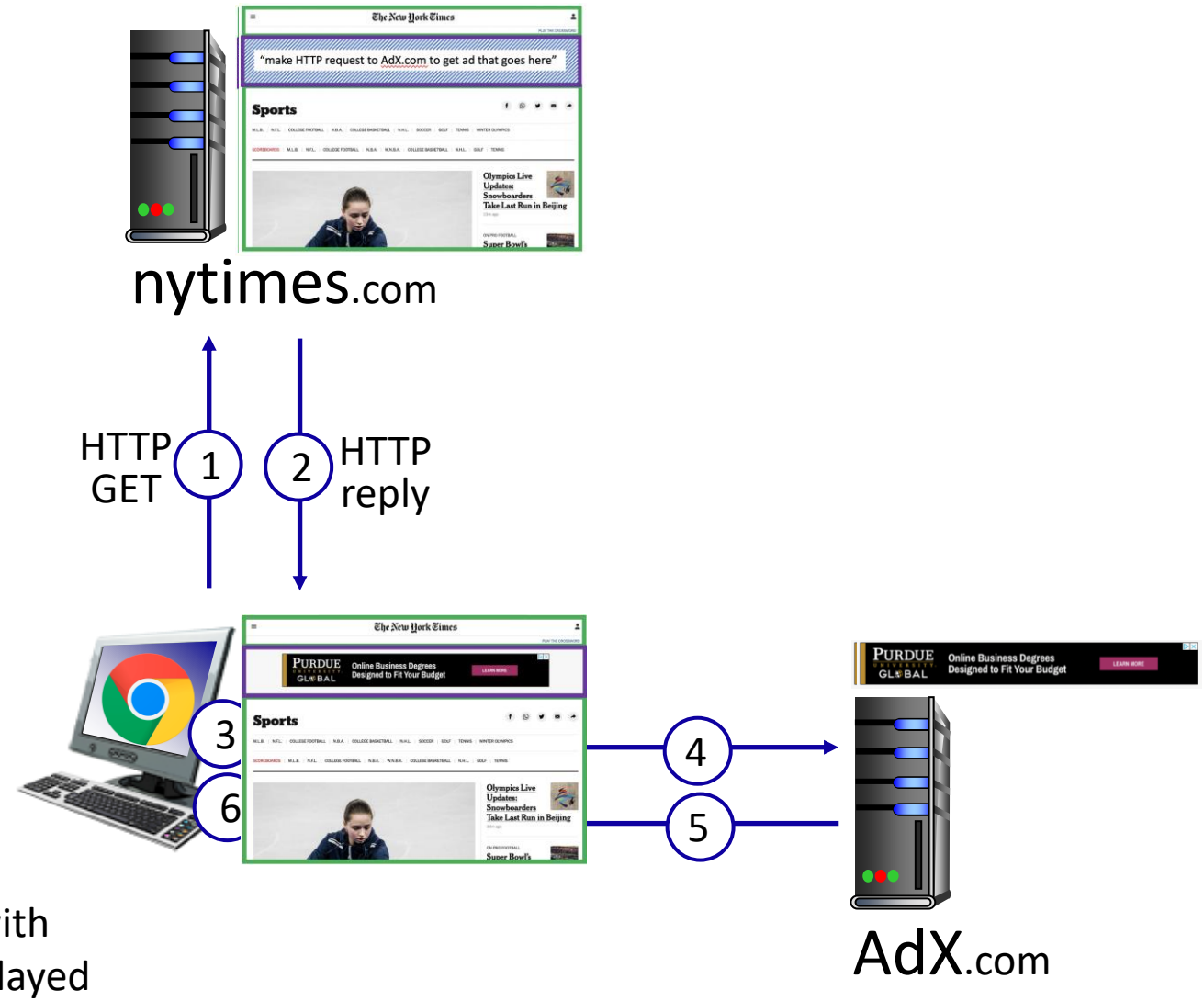


Example: displaying a NY Times web page

1 2 GET base html file from nytimes.com

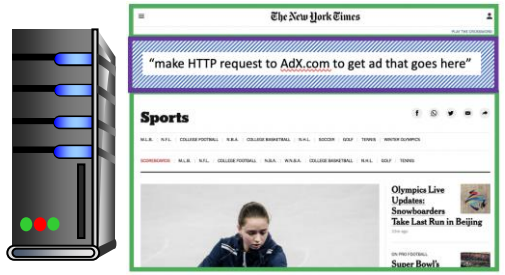
4 5 fetch ad from AdX.com

7 display composed page

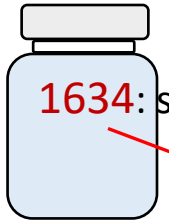


NY times page with embedded ad displayed

Cookies: tracking a user's browsing behavior



nytimes.com (sports)



1634: sports, 2/15/22

"first party" cookie – from website you chose to visit (provides base html file)

HTTP GET

HTTP reply
Set cookie: 1634

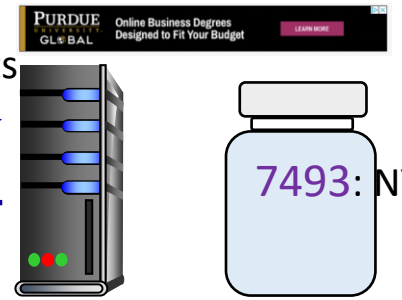


HTTP GET
Referrer: NY Times Sports

4

5

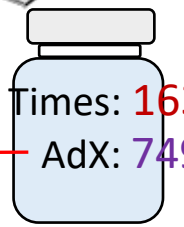
HTTP reply
Set cookie: 7493



7493: NY Times sports, 2/15/22

AdX.com

"third party" cookie – from website you did not choose to visit

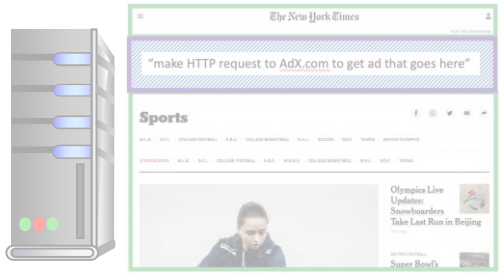


NY Times: 1634
AdX: 7493

Cookies: tracking a user's browsing behavior



socks.com



nytimes.com



HTTP reply 2

HTTP GET 1



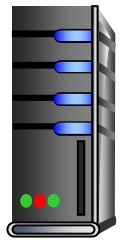
HTTP GET
Referrer: socks.com, cookie: 7493

4

5

HTTP reply
Set cookie: 7493





AdX.com



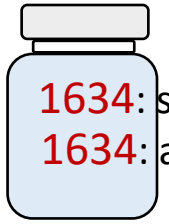
AdX:

- *tracks my web browsing* over sites with AdX ads
- can return targeted ads based on browsing history

Cookies: tracking a user's browsing behavior (one day later)







HTTP GET
cookie: 1634

HTTP reply
Set cookie: 1634

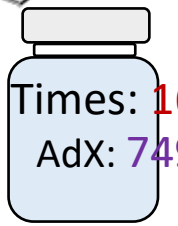


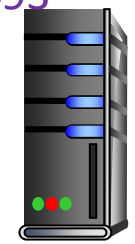
HTTP GET
Referrer: nytimes.com, cookie: 7493

4

5

HTTP reply
Set cookie: 7493
Returned ad for socks!





AdX.com



Cookies: tracking a user's browsing behavior

Cookies can be used to:

- track user behavior on a given website (first party cookies)
- track user behavior across multiple websites (third party cookies) without user ever choosing to visit tracker site (!)
- tracking may be *invisible* to user:
  - rather than displayed ad triggering HTTP GET to tracker, could be an invisible link

third party tracking via cookies:

- disabled by default in Firefox, Safari browsers
- to be disabled in Chrome browser in 2023

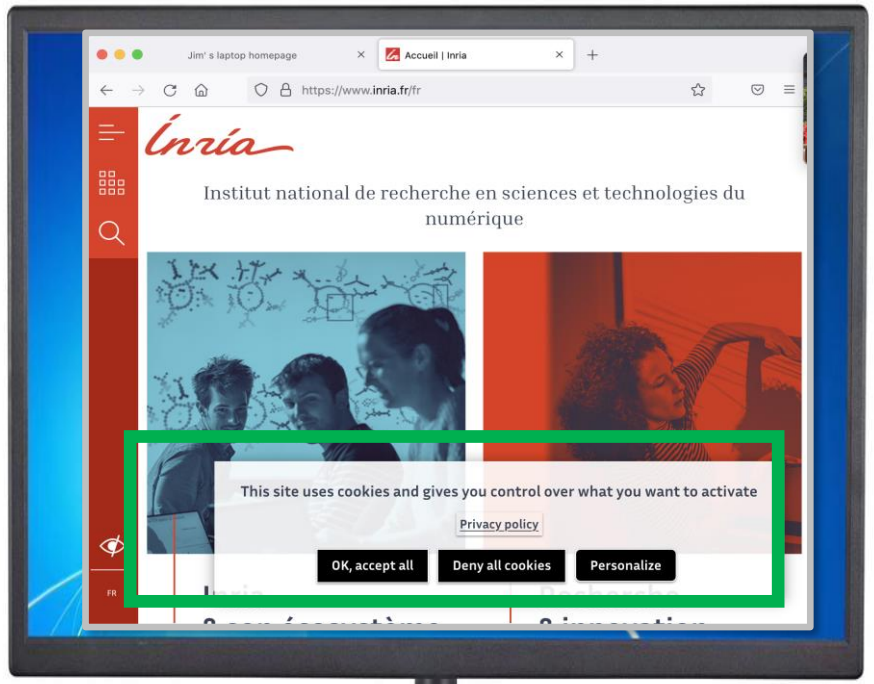
GDPR (EU General Data Protection Regulation) and cookies

"Natural persons may be associated with online identifiers [...] such as internet protocol addresses, cookie identifiers or other identifiers [...].

This may leave traces which, in particular when combined with unique identifiers and other information received by the servers, may be used to create profiles of the natural persons and identify them."

GDPR, recital 30 (May 2018)

when cookies can identify an individual, cookies are considered personal data, subject to GDPR personal data regulations



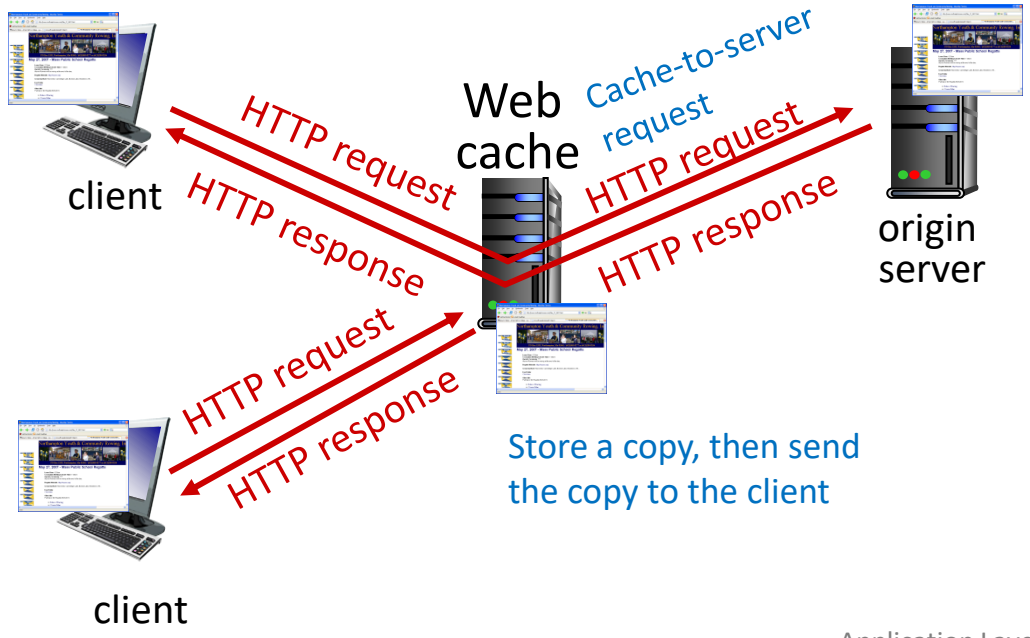
User has explicit control over whether or not cookies are allowed

Web caches (cache server ≈ proxy servers)

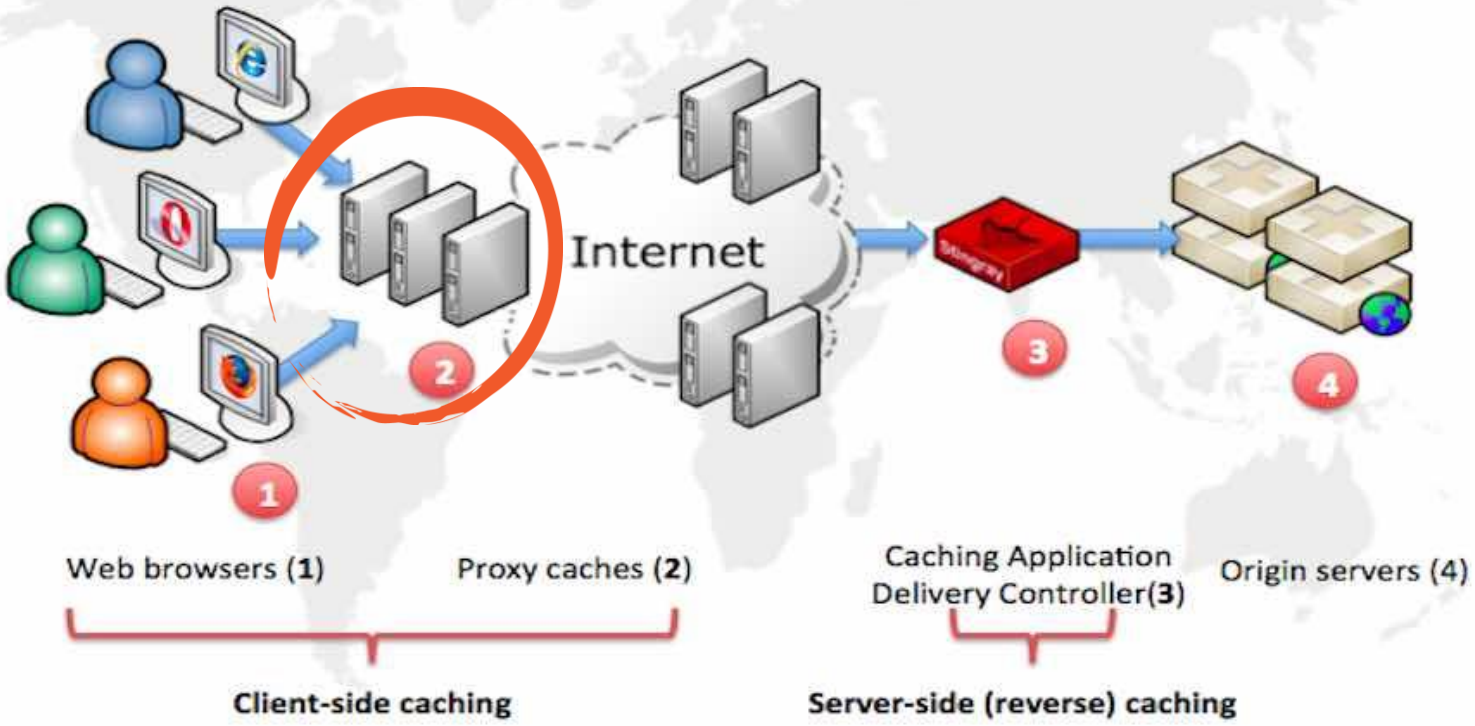
Goal: satisfy client requests without involving origin server

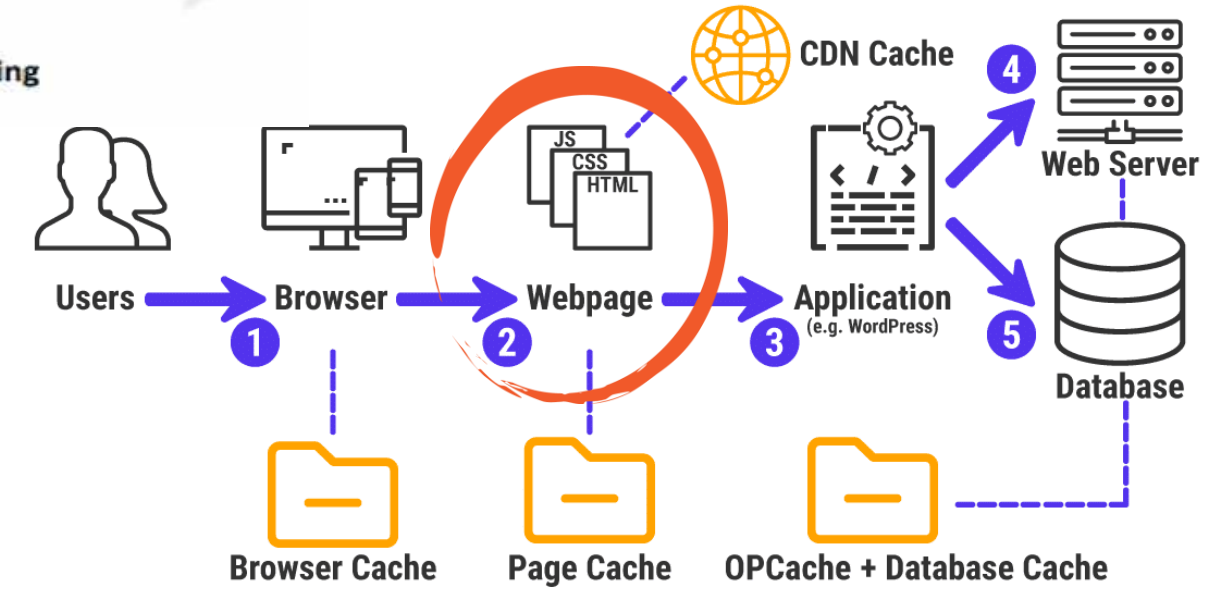
- user configures browser to point to a (local) *Web cache*
- browser sends all HTTP requests first to cache
  - *if* object in cache: cache returns object to client（cache hit）
  - *else* cache requests object from origin server, caches received object, then returns object to client (cache miss)

The Web cache has its **own disk storage** and **keeps copies of recently requested objects** in this storage.



Web caches (aka proxy servers)



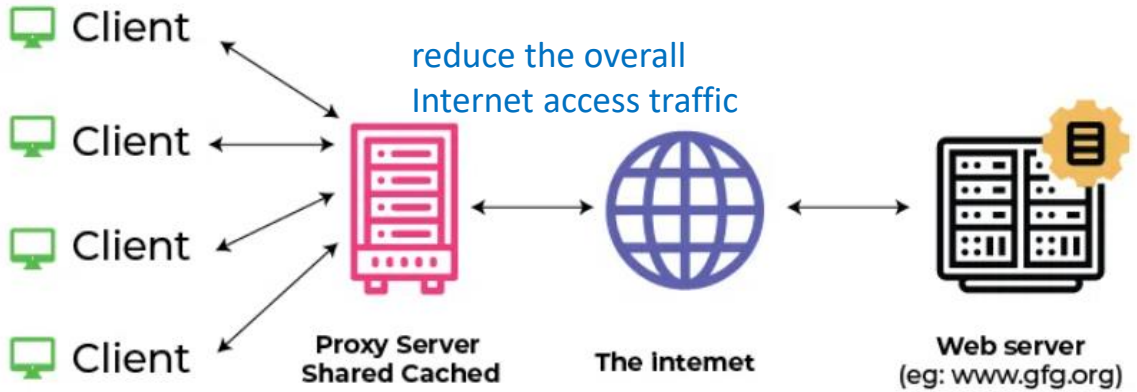


Web caches (aka proxy servers)

- Web cache acts as both client and server
  - server for original requesting client
  - client to origin server
- server tells cache about object's allowable caching in response header:

```
Cache-Control: max-age=<seconds>
```

```
Cache-Control: no-cache
```



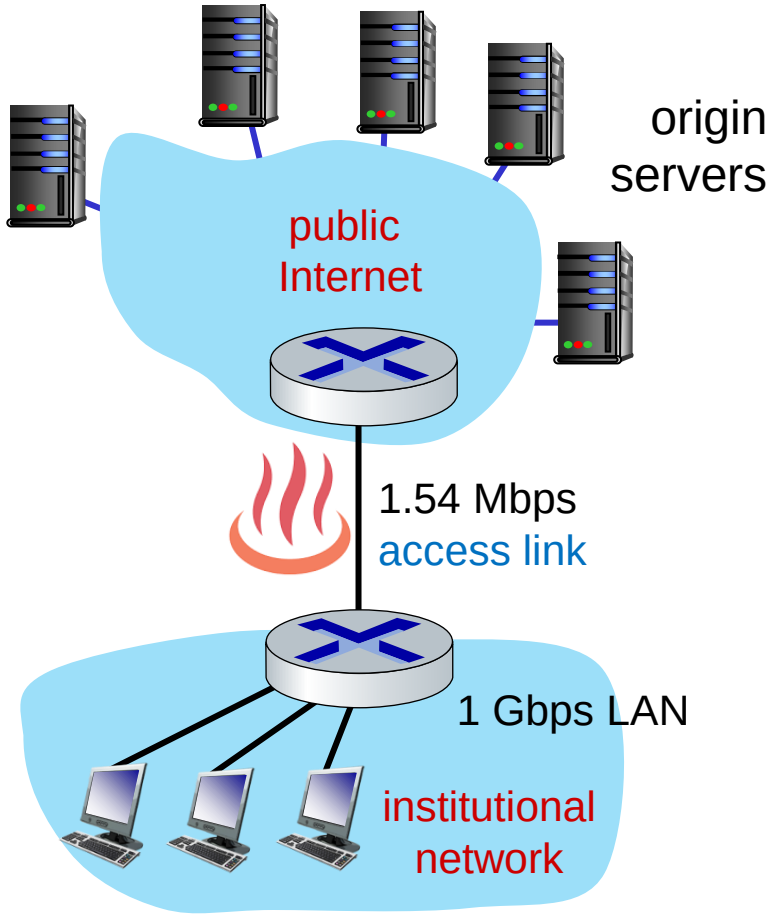
Why Web caching?

- reduce response time for client request
  - cache is closer to client
  - Typically a Web cache is purchased and installed by an ISP.
- reduce traffic on an institution's access link to the Internet.
- Internet is dense with caches
  - enables "poor" content providers to more effectively deliver content

Caching example

Scenario:

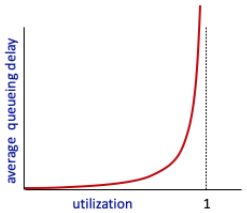
- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100Kbits/request
- average request rate from browsers to origin servers: 15 requests/sec
  - **avg data rate**: 1.50 Mbps = 15 x 100Kbps



Performance:

- access link utilization = 0.97 (1.5 /1.54 =)
- LAN utilization: .0015 (=1.5 / 1000)
- **end-end delay = Internet delay + access link delay + LAN delay**

  = 2 sec + minutes + µsecs



problem: large queueing delays at high utilization!

Unacceptable!!

can be neglected

Option 1: buy a faster access link

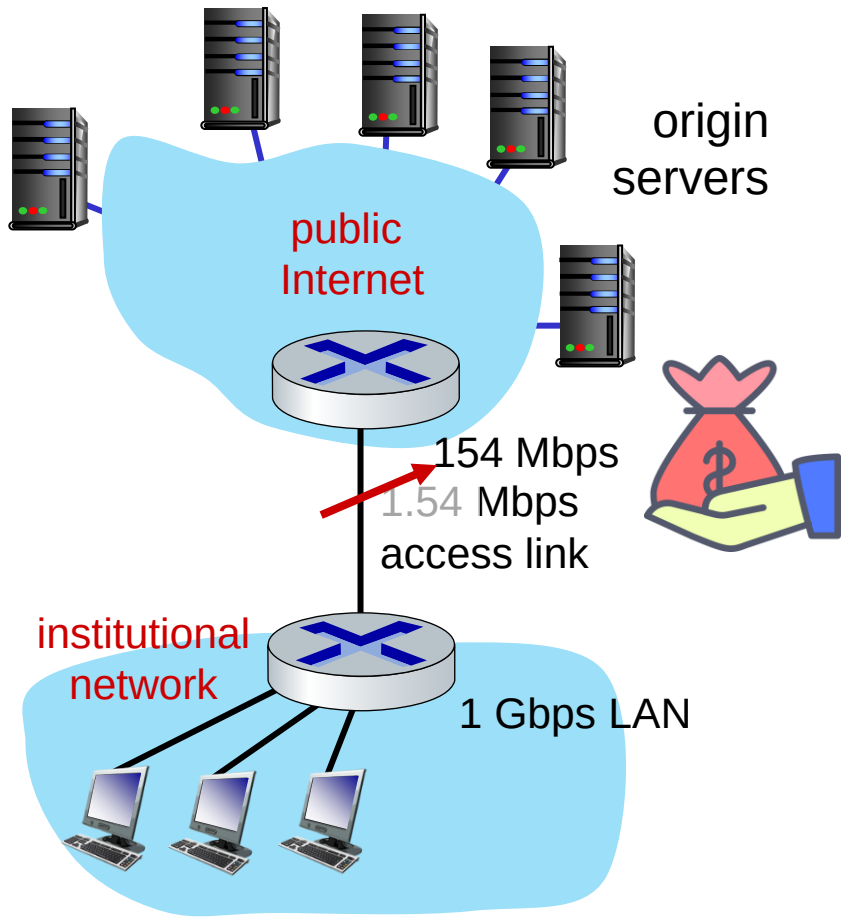
Scenario:

- access link rate: 1.54 Mbps → 154 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100Kbits/request
- average request rate from browsers to origin servers: 15 requests/sec
  - **avg data rate**: 1.50 Mbps

Performance:

- access link utilization = .97 → .0097 (1.5 /154 =)
- LAN utilization: .0015
- end-end delay = Internet delay + access link delay + LAN delay
  = 2 sec + minutes + µsecs ≈ 2 sec → msecs

Cost: faster access link (expensive!)



Option 2: install a web cache

Scenario:

- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100Kbits/request
- average request rate from browsers to origin servers: 15 requests/sec
  - **avg data rate**: 1.50 Mbps

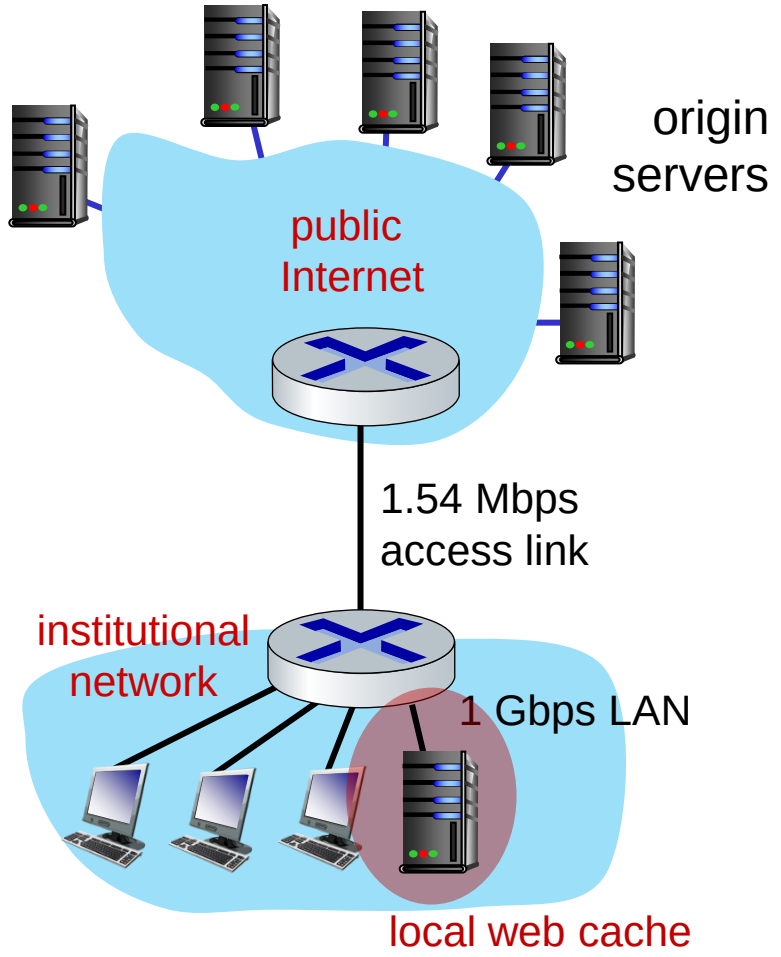
Cost: web cache (cheap!)



Performance:

- LAN utilization: .?
- access link utilization = ?
- average end-end delay = ?

How to compute link utilization, delay?

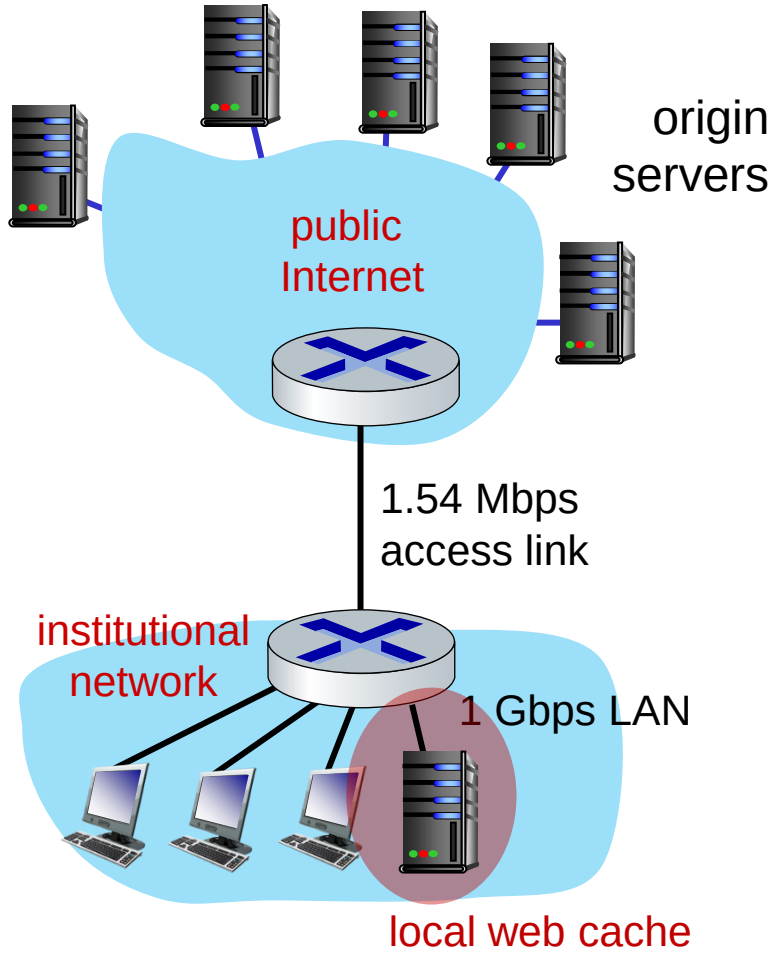


Calculating access link utilization, end-end delay with cache:

Cache hit rate typically range from 0.2 to 0.7 in practice.

suppose cache hit rate is 0.4:

- 40% requests served by cache, with low (msec) delay
- 60% requests satisfied at origin 1 – 0.4 = 0.6
  - data rate over access link
    = 0.6 * 1.50 Mbps = 0.9 Mbps
  - access link utilization = 0.9/1.54 = 0.58 means low (msec) queueing delay at access link
- average end-end delay:
  = 0.6 * (delay from origin servers)
    + 0.4 * (delay when satisfied at cache)
  = 0.6 *(2.01) + 0.4 (~msecs) = ~ 1.2 secs



lower average end-end delay than with 154 Mbps link (and cheaper too!)

Content Distribution Networks (CDNs)

- Through the use of **CDNs**, Web caches are increasingly playing an important role in the Internet.
- A CDN company installs many geographically distributed caches throughout the Internet, thereby localizing much of the traffic.
- There are shared CDNs (such as Akamai and Limelight) and dedicated CDNs (such as Google and Netflix).



Content Delivery Network

Browser caching: Conditional GET

Goal: don't send object if cache has up-to-date cached version

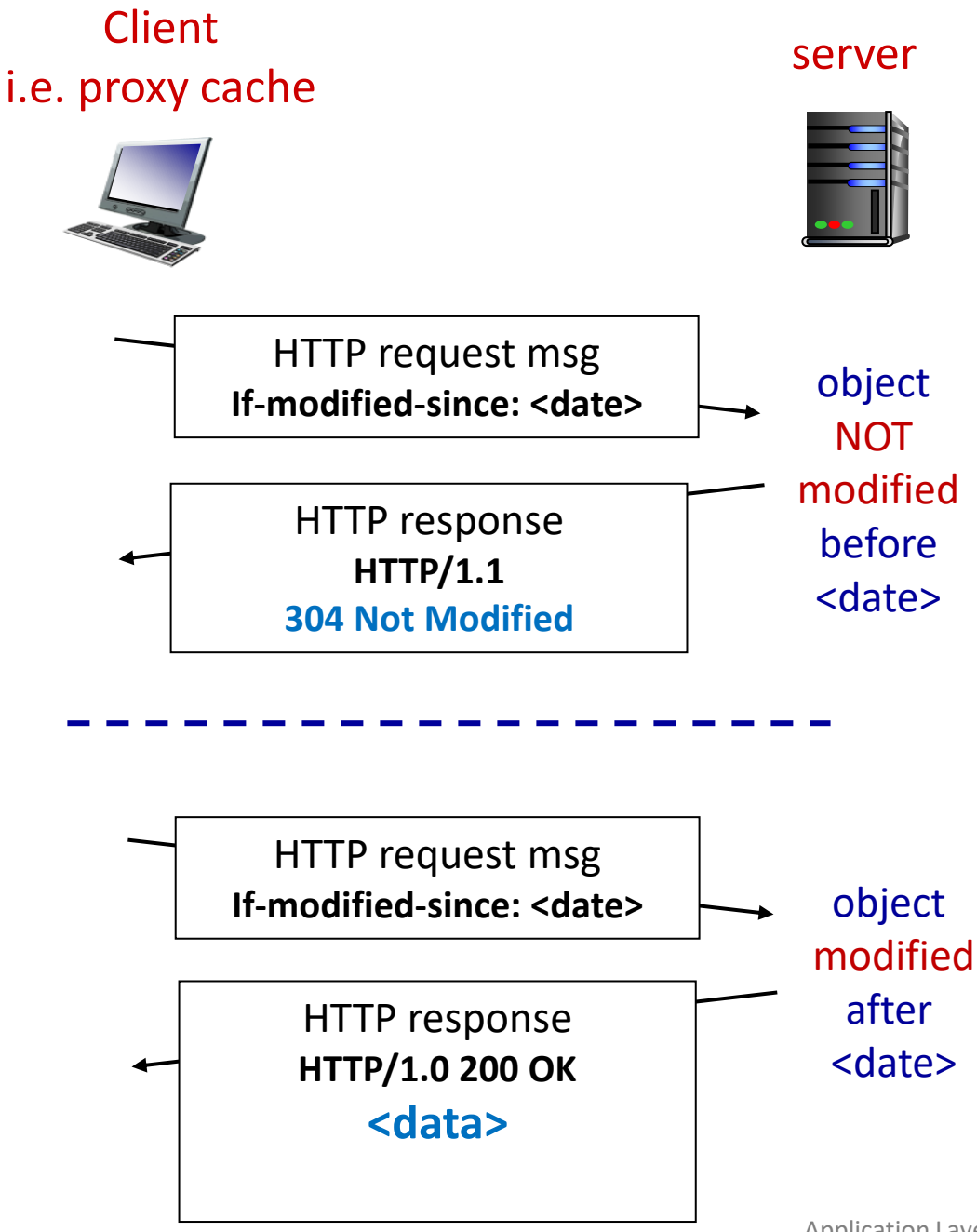
- no object transmission delay (or use of network resources)

- *client:* specify date of cached copy in HTTP request

  If-modified-since: <date>

- *server:* response contains no object if cached copy is up-to-date:

  HTTP/1.0 304 Not Modified



Conditional GET





GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com

HTTP/1.1 200 OK
Date: Fri, 3 Oct 2025 15:39:29
Server: Apache/1.3.0 (Unix)
Last-Modified: Tue, 9 Sep 2025 09:23:24
Content-Type: image/gif
(data data data data data ...)

Conditional GET: HTTP has a mechanism that allows a cache to verify that its objects are up to date.

One week later

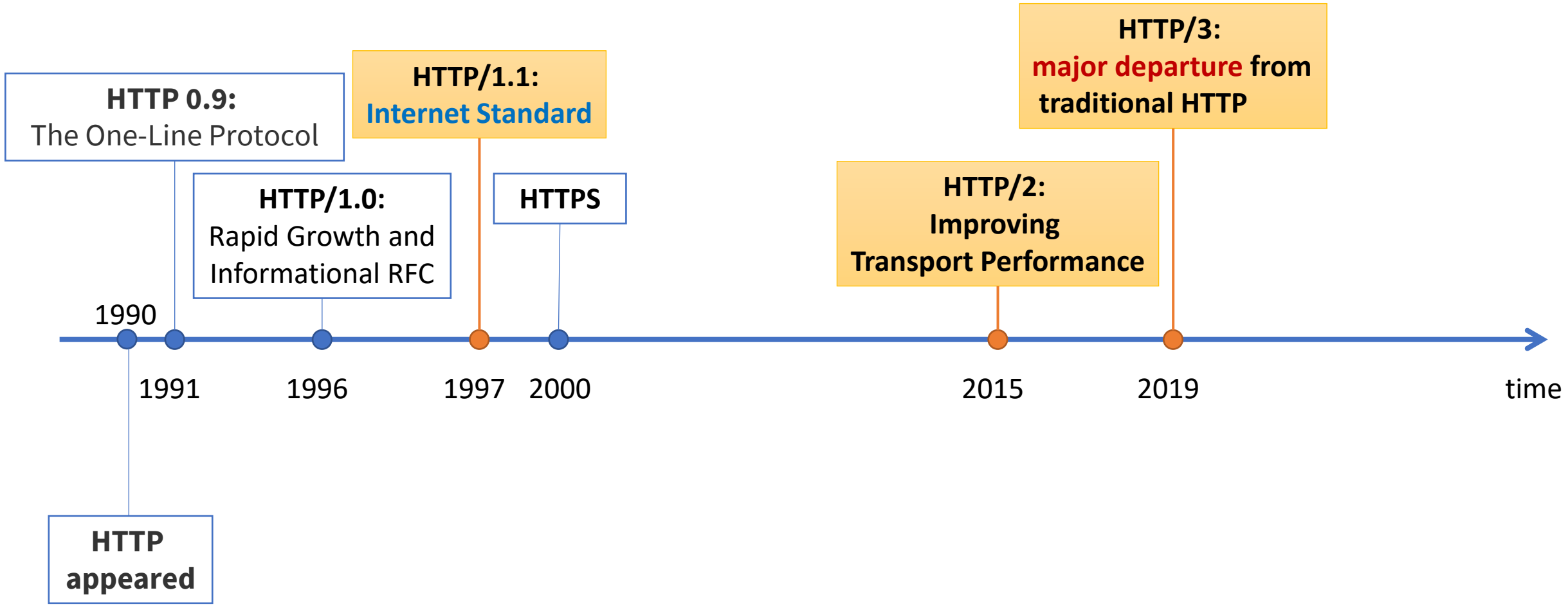
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
If-modified-since: Tue, 9 Sep 2025 09:23:24

the cache performs **an up-to-date check** by issuing a conditional GET

HTTP/1.1 304 Not Modified
Date: Fri, 10 Oct 2025 15:39:29
Server: Apache/1.3.0 (Unix)
(empty entity body)

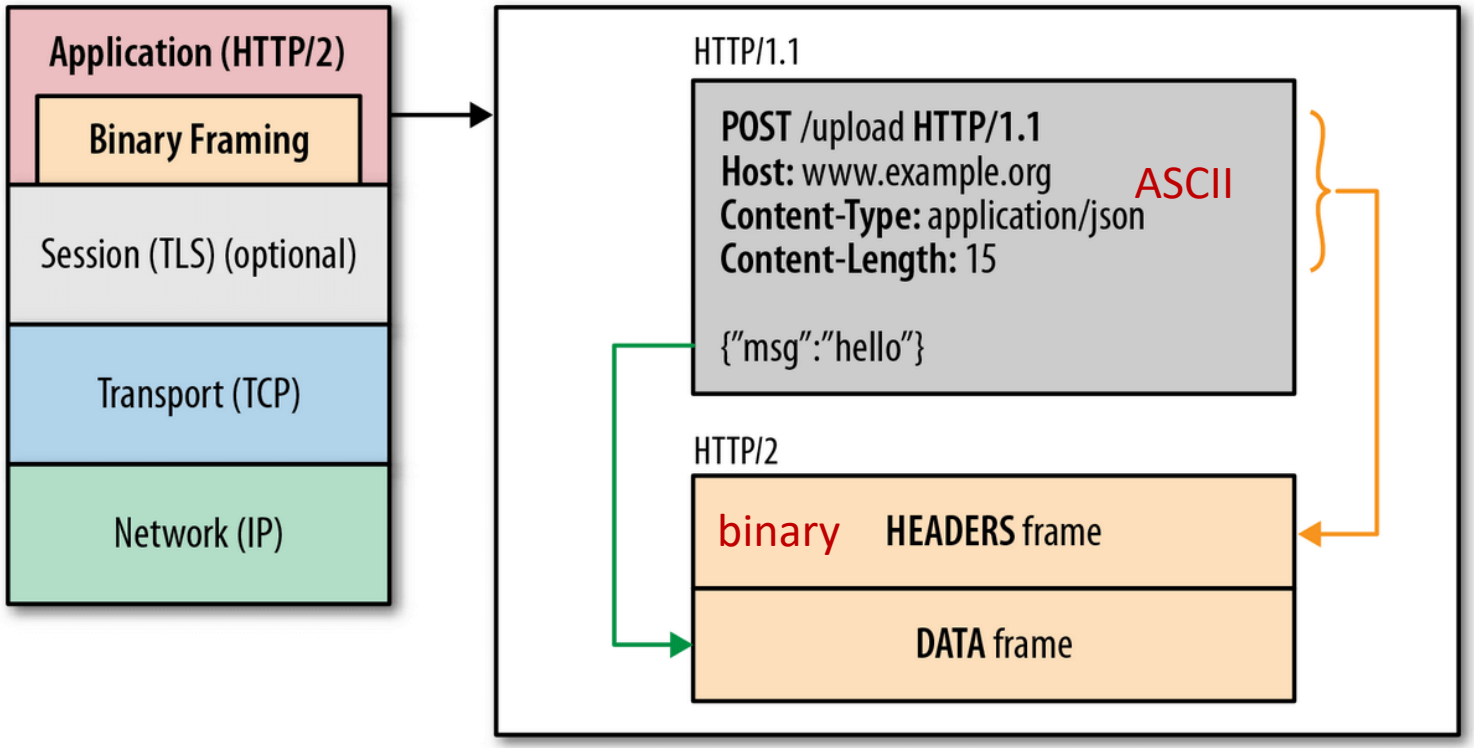
Suppose the object has **NOT** been modified since 9 Sep 2025 09:23:24.

A little history about HTTP



HTTP/2 in one slide…

1. **One TCP connection**
2. **Request → Stream**
   - Streams are multiplexed
   - Streams are prioritized
3. **Binary framing layer**
   - Prioritization
   - Flow control
   - Server push
4. **Header compression (HPACK)**



HTTP/2

Key goal: decreased delay in multi-object HTTP requests

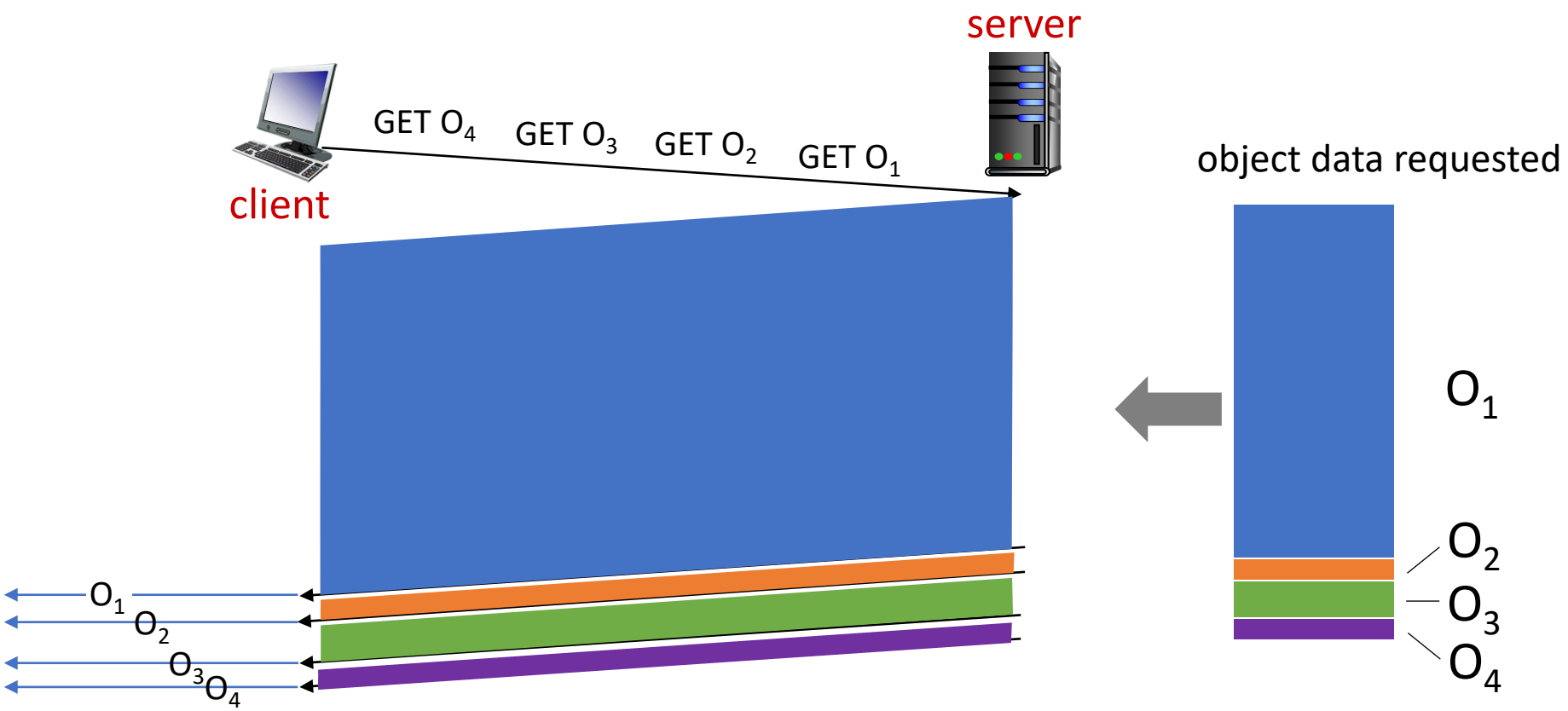
review

HTTP1.1: introduced multiple, pipelined GETs over single TCP connection

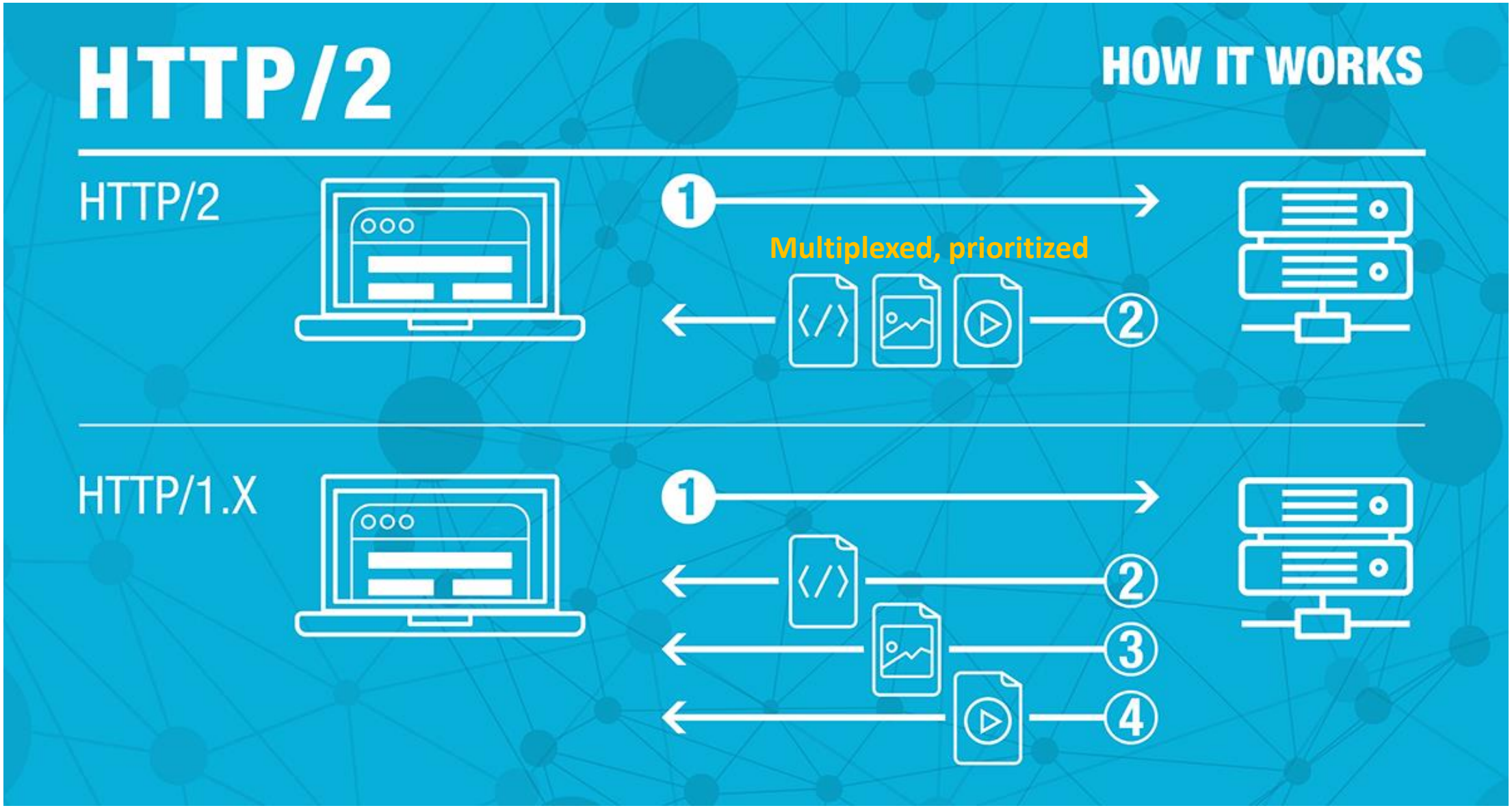
- server responds *in-order* (FCFS: first-come-first-served scheduling) to GET requests
- with FCFS, small object may have to wait for transmission (head-of-line (HOL) blocking) behind large object(s)
- loss recovery (retransmitting lost TCP segments) stalls object transmission

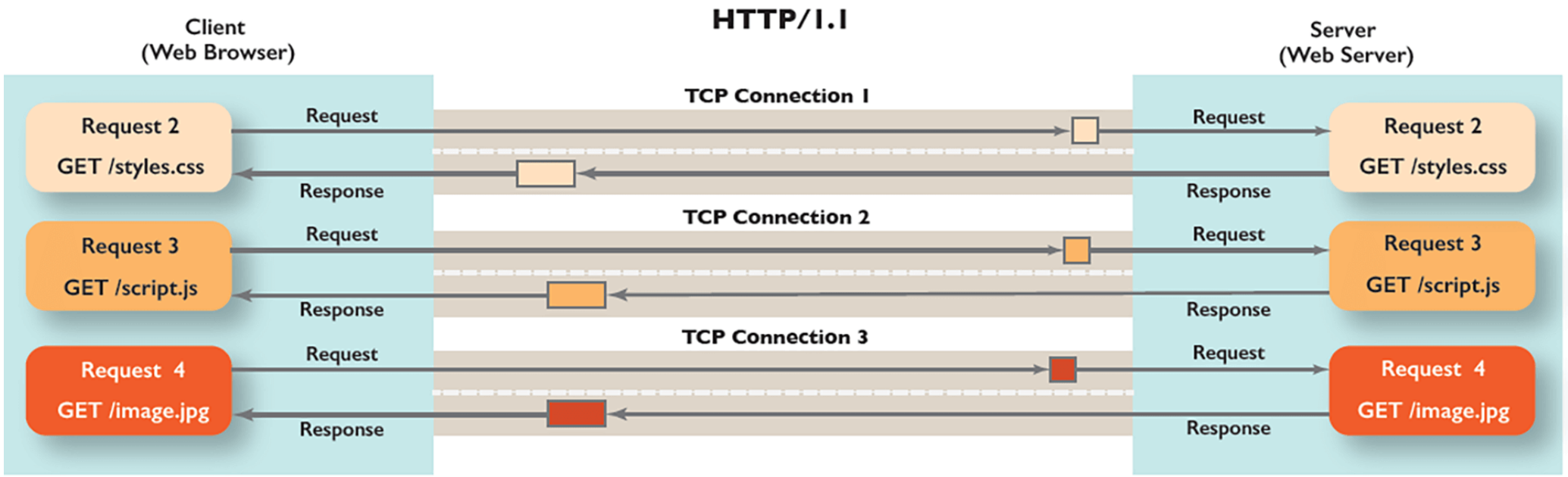
HTTP/1.1: HOL blocking

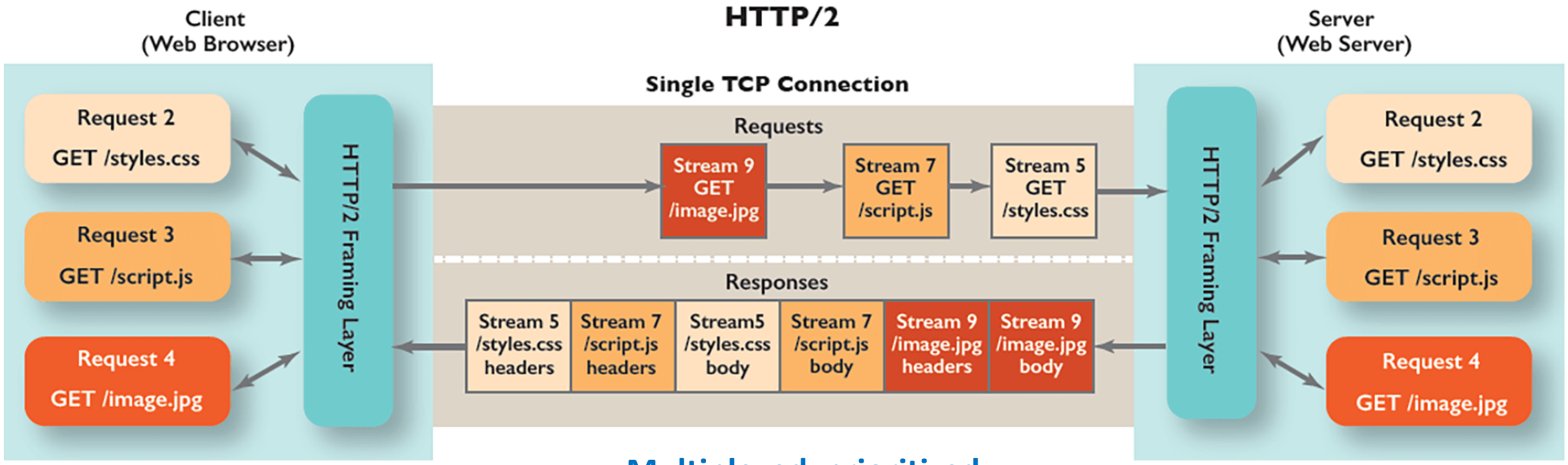
HTTP 1.1: client requests 1 large object (e.g., video file) and 3 smaller objects



objects delivered in order requested: O_2, O_3, O_4 wait behind O_1


HTTP/2
HOW IT WORKS
HTTP/2
Multiplexed, prioritized
1
2
HTTP/1.X
1
2
3
4





Multiplexed, prioritized

HTTP/2



Key goal: decreased delay in multi-object HTTP requests

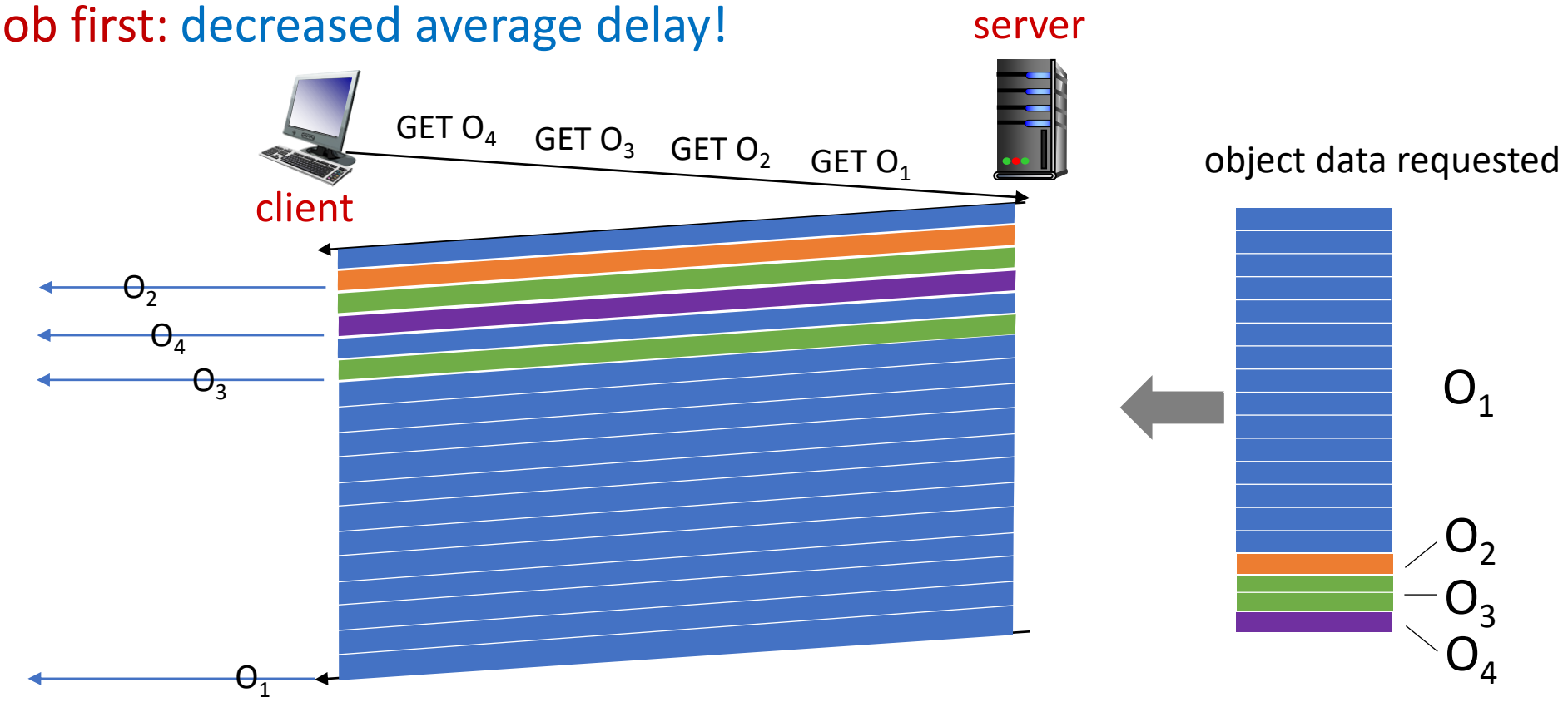
HTTP/2: [RFC 7540, 2015] increased flexibility at *server* in sending objects to client:

- methods, status codes, most header fields unchanged from HTTP 1.1
- transmission order of requested objects based on client-specified object priority (not necessarily FCFS)
- *push* unrequested objects to client
- divide objects into frames, schedule frames to mitigate HOL blocking
- **HTTP/2** introduces a **transport-level HOL blocking** problem.

HTTP/2: mitigating HOL blocking

HTTP/2: objects divided into frames, frame transmission interleaved

shortest job first: decreased average delay!



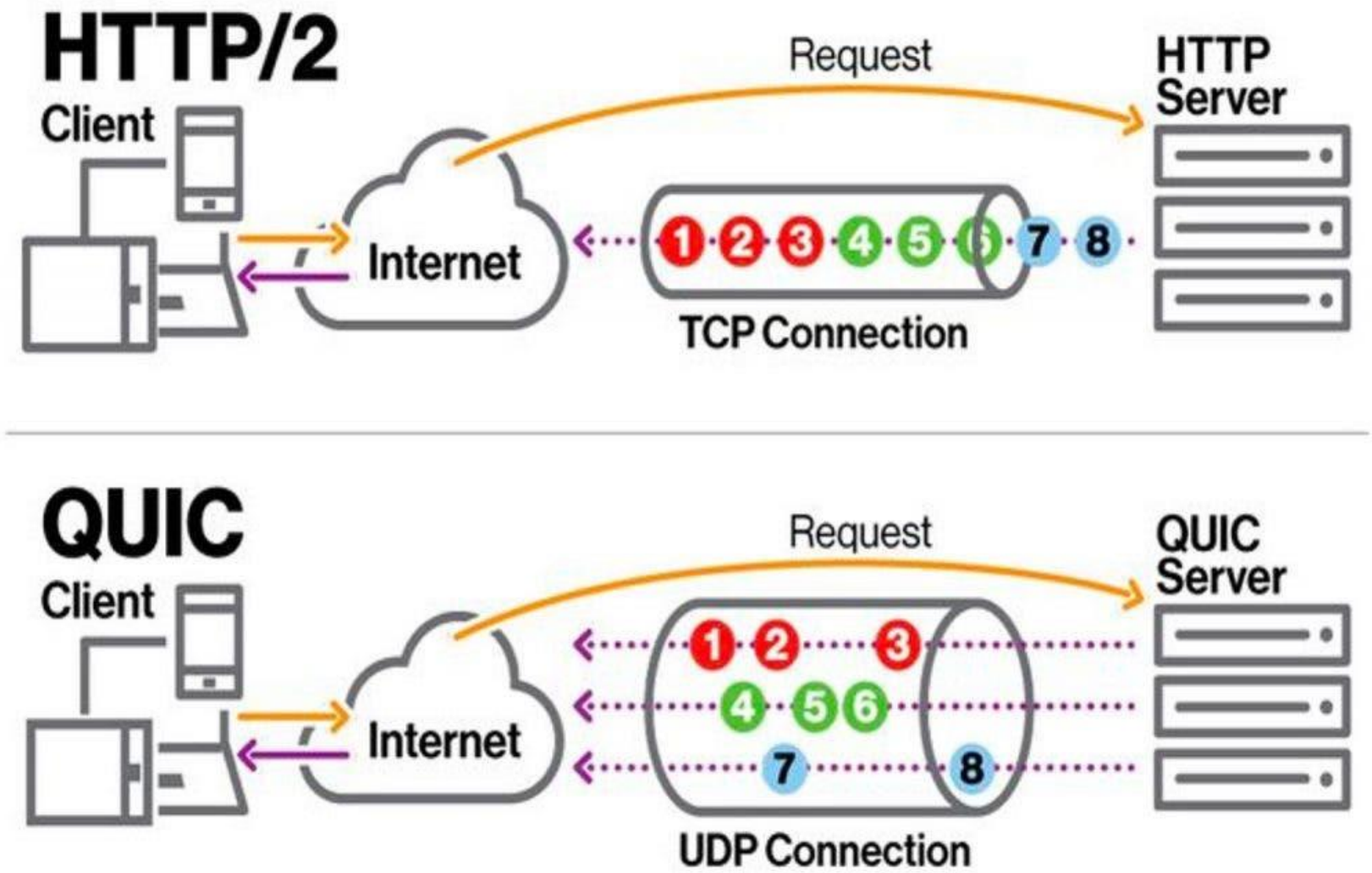
O_2, O_3, O_4 delivered quickly, O_1 slightly delayed

HTTP/2 to HTTP/3

HTTP/2 over single TCP connection means:

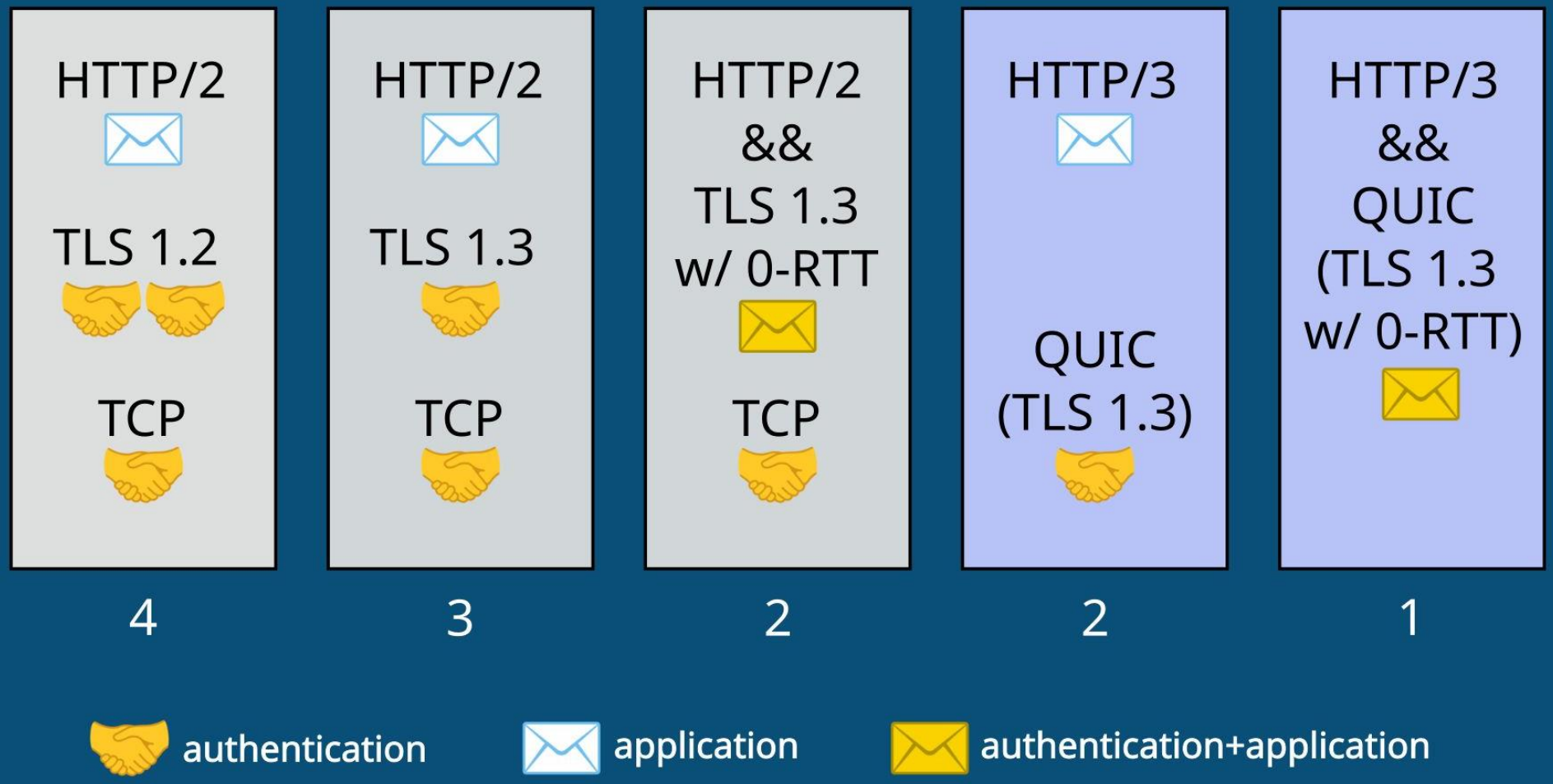
- recovery from packet loss still stalls all object transmissions
  - as in HTTP 1.1, browsers have incentive to open multiple parallel TCP connections to reduce stalling, increase overall throughput
- no security over vanilla TCP connection
- HTTP/3: adds security, per object error- and congestion-control (more pipelining) over UDP
  - more on HTTP/3 in chapter 3 transport layer
  - specifically designed to eliminate HOL blocking

HTTP/2 vs. HTTP/3

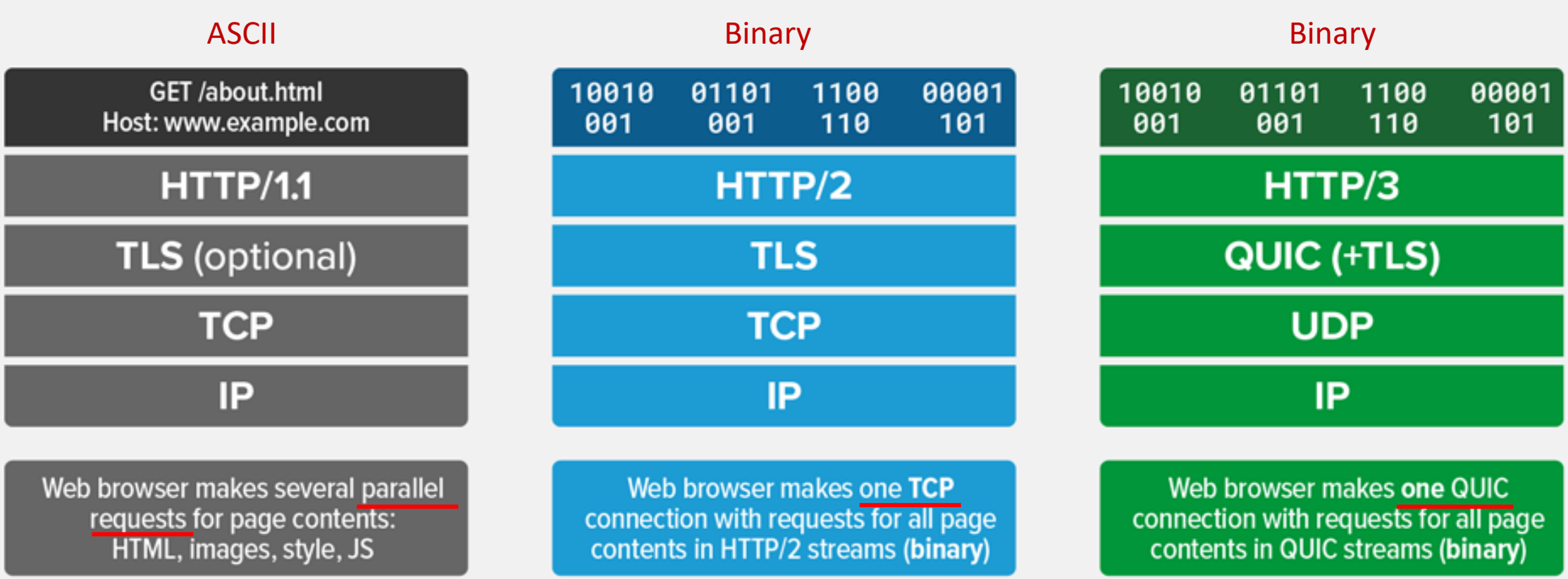


HTTP/2 vs. HTTP/3 connection setup

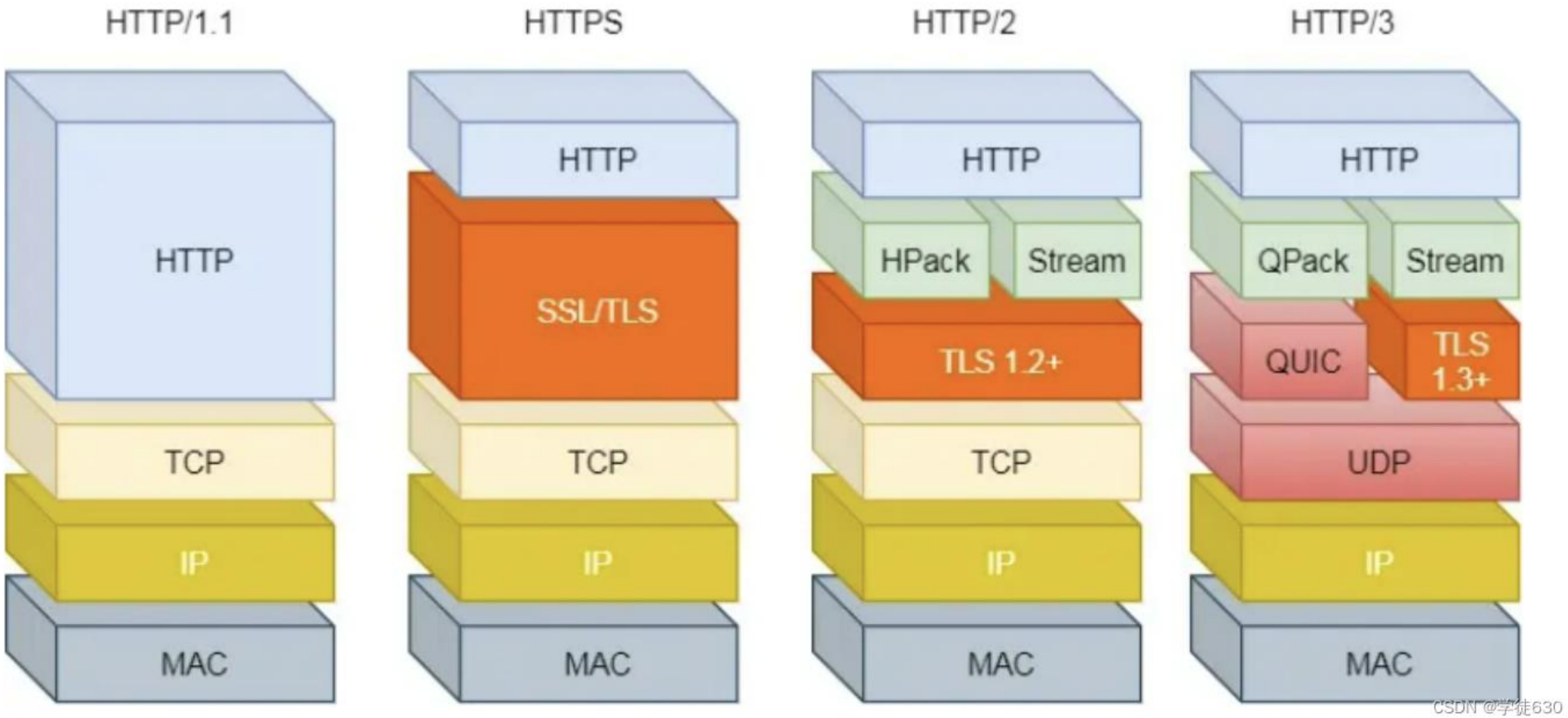
Number of round-trips needed for sending application (HTTP) data



Comparison among HTTP/1.1, /2, /3

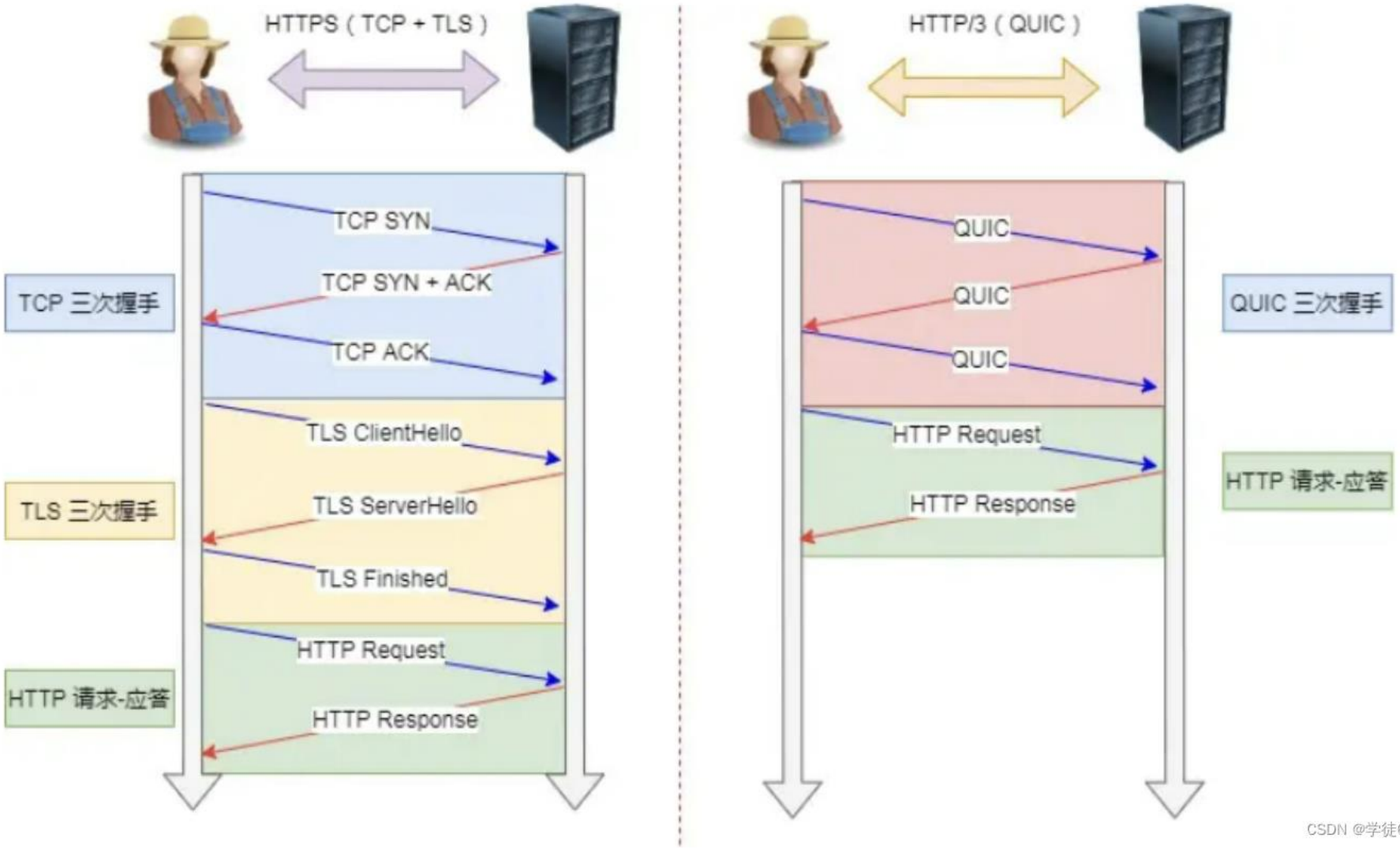


HTTP versions

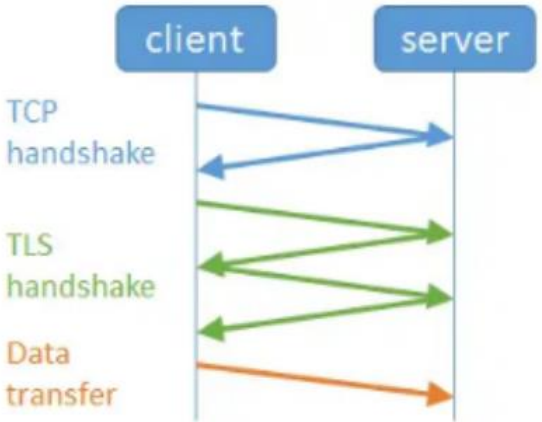


CSDN @学徒630

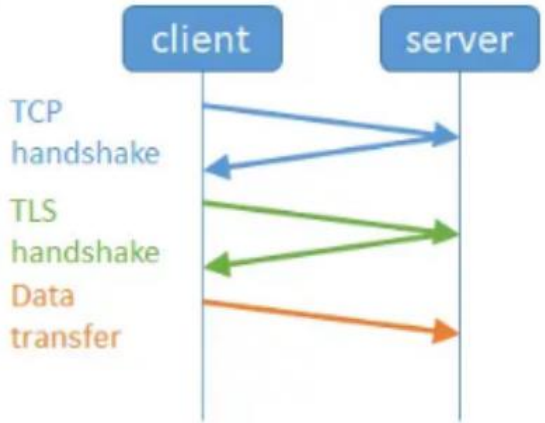
HTTPS vs. HTTP/3 (QUIC)



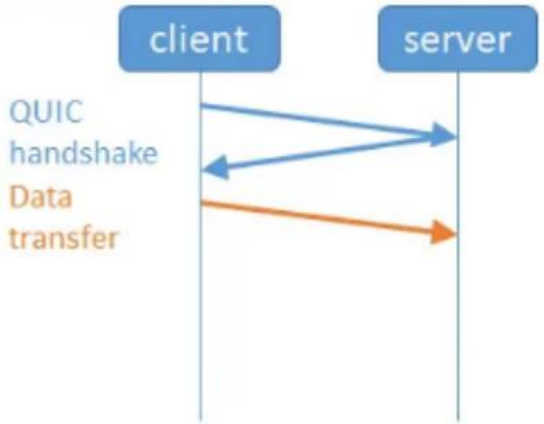
CSDN @学徒630



TCP + TLS 1.2
3RTT

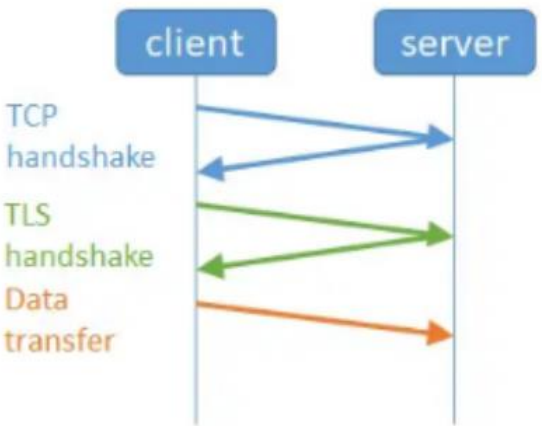


TCP + TLS 1.3
2RTT

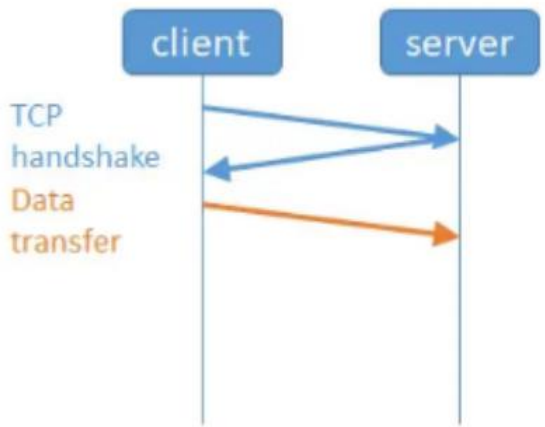


QUIC
1RTT

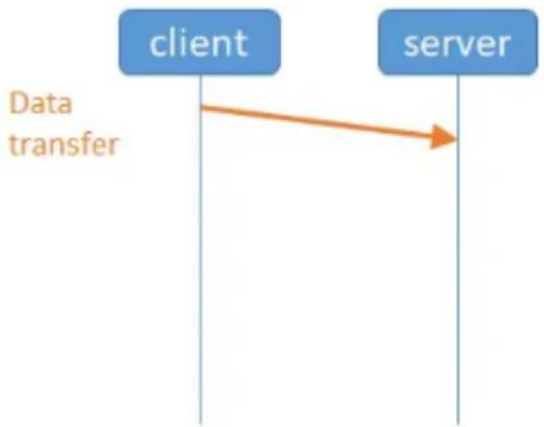
Initial connections



TCP + TLS 1.2
2RTT



TCP + TLS 1.3
1RTT



QUIC
0RTT

Resumed connections

CSDN @学徒630

Application layer: overview

- Principles of network applications
- Web and HTTP
- **E-mail, SMTP, IMAP**
- The Domain Name System DNS
- video streaming and content distribution networks
- socket programming with UDP and TCP



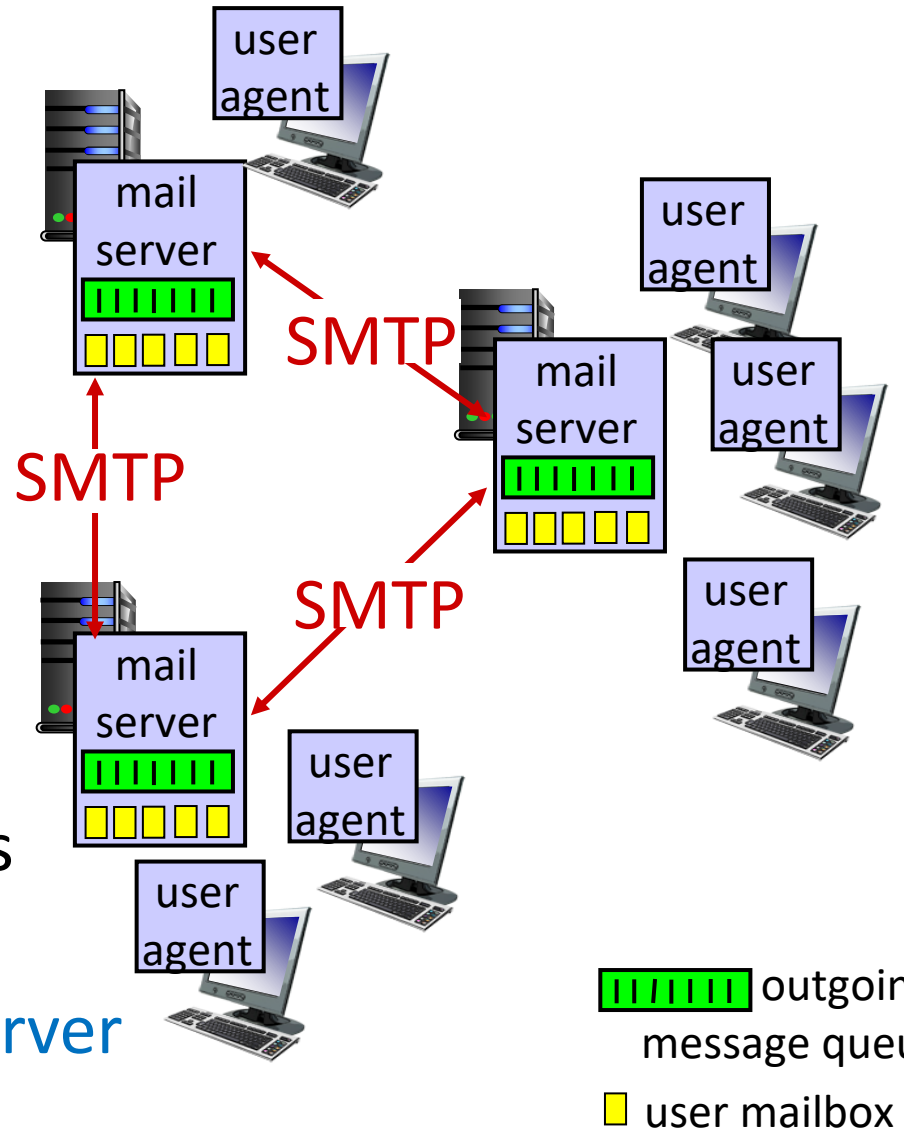
E-mail

Three major components:

- user agents
- mail servers
- Simple Mail Transfer Protocol: SMTP

User Agent

- a.k.a. "mail reader"
- composing, editing, reading mail messages
- e.g., Outlook, iPhone mail client
- outgoing, incoming messages stored on server



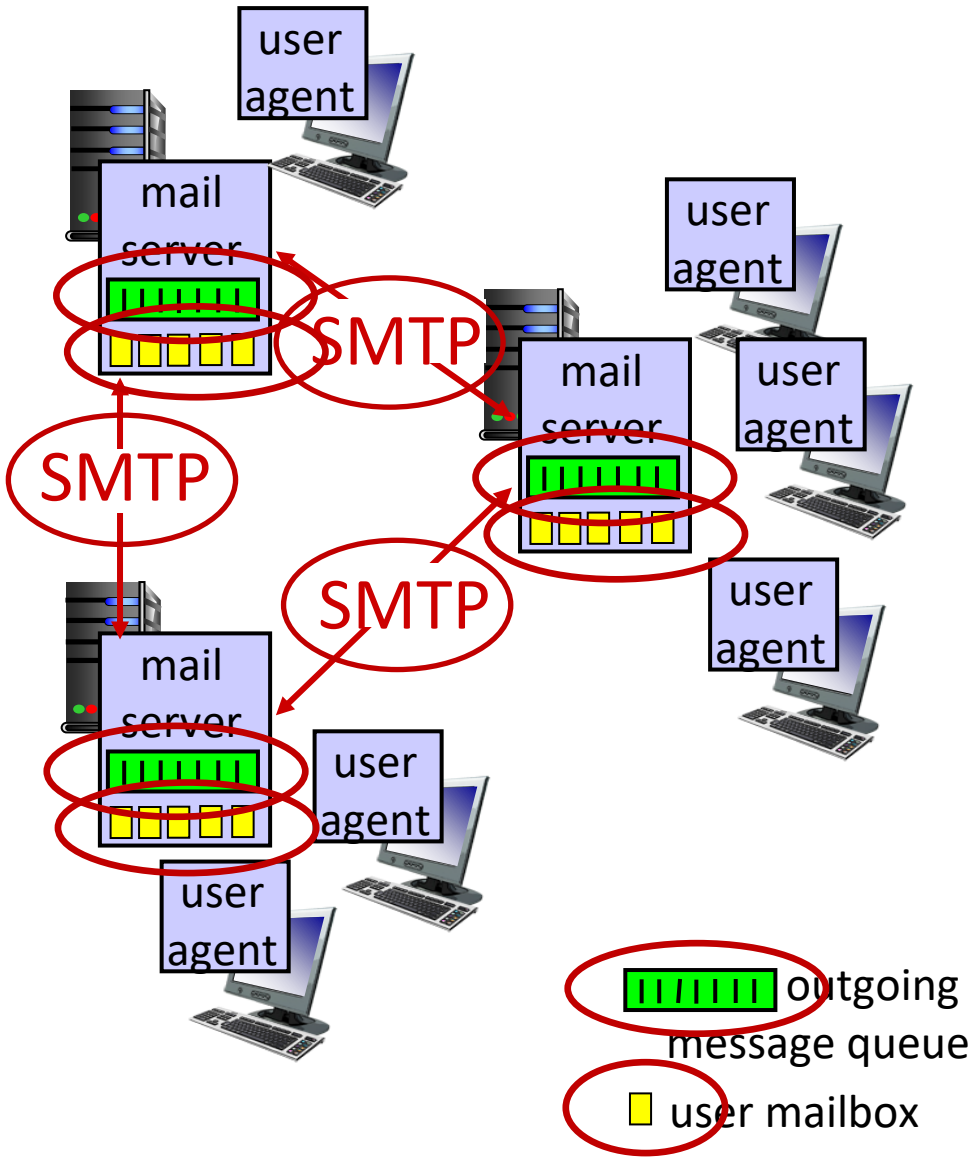
E-mail: mail servers

mail servers:

- *mailbox* contains incoming messages for user
- *message queue* of outgoing (to be sent) mail messages

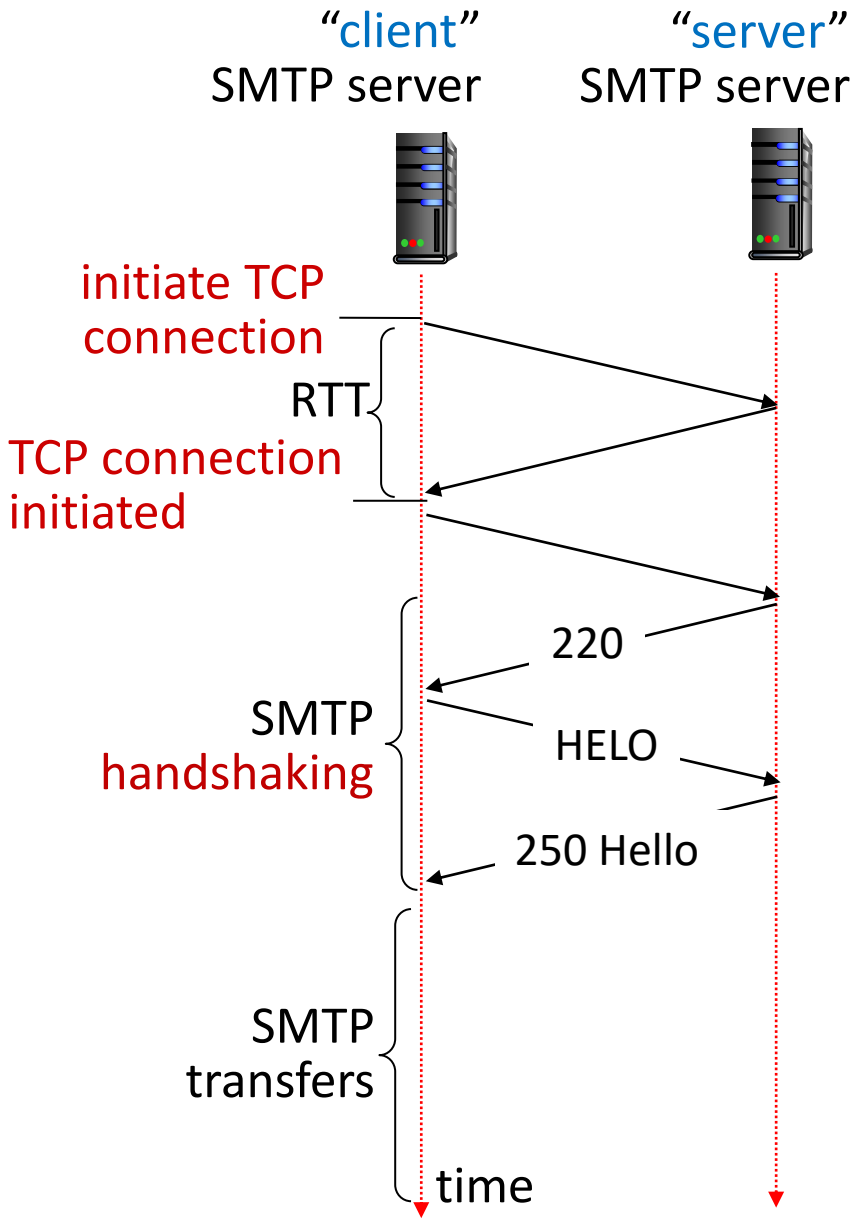
SMTP protocol between mail servers to send email messages

- client: sending mail server
- "server": receiving mail server



SMTP RFC (5321)

- uses TCP to reliably transfer email message from client (mail server initiating connection) to server, port 25
  - direct transfer: sending server (acting like client) to receiving server
- three phases of transfer
  - SMTP handshaking (greeting)
  - SMTP transfer of messages
  - SMTP closure
- command/response interaction (like HTTP)
  - commands: ASCII text
  - response: status code and phrase



Scenario: Alice sends e-mail to Bob

1) Alice uses user agent (UA) to compose e-mail message "to" bob@someschool.edu

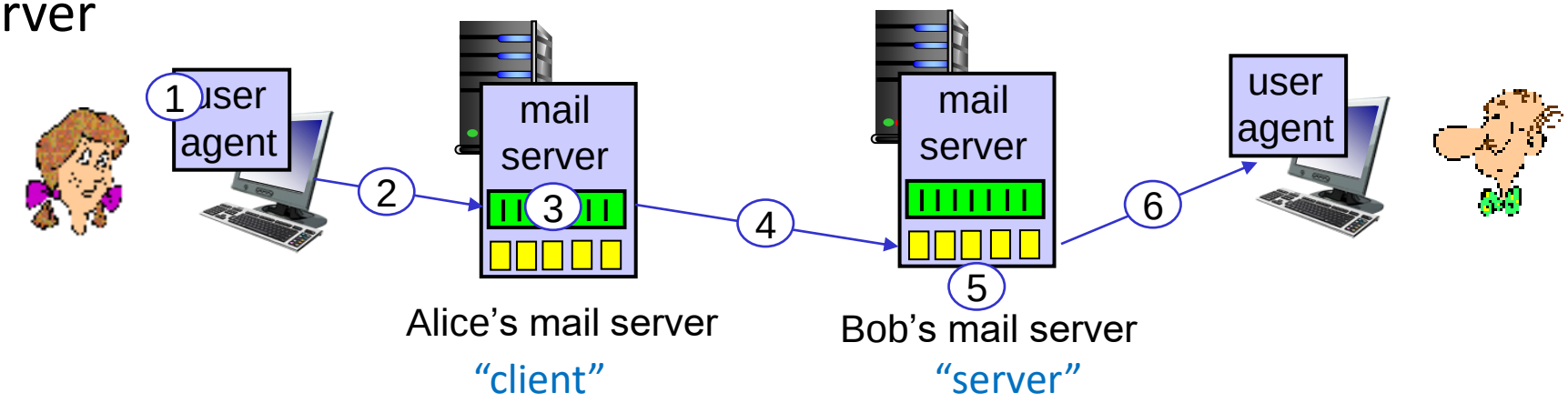
2) Alice's UA sends message to her mail server using SMTP; message placed in message queue

3) client side of SMTP at mail server opens TCP connection with Bob's mail server

4) SMTP client sends Alice's message over the TCP connection

5) Bob's mail server places the message in Bob's mailbox

6) Bob invokes his user agent (UA) to read message



Sample SMTP interaction

Server
Client

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250  Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

SMTP handshaking

Client command

C: "client" SMTP server

S: "server" SMTP server

In ASCII jargon, each message ends with `CRLF.CRLF`

SMTP: observations

comparison with HTTP:

- HTTP (basic): client pull
- SMTP: client push
- both have ASCII command/response interaction, status codes
- HTTP: each object encapsulated in its own response message
- SMTP: multiple objects sent in multipart message
- SMTP uses persistent connections
- SMTP requires message (header & body) to be in 7-bit ASCII
- SMTP server uses CRLF.CRLF to determine end of message

Mail message format

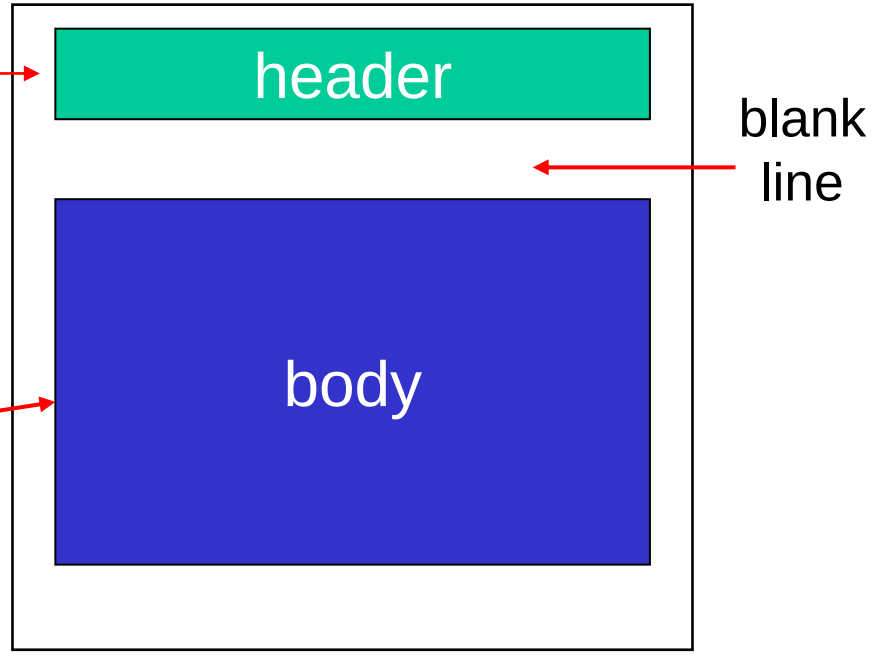
```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Searching for the meaning of life
```

SMTP: protocol for exchanging e-mail messages, defined in RFC 5321 (like RFC 7231 defines HTTP)

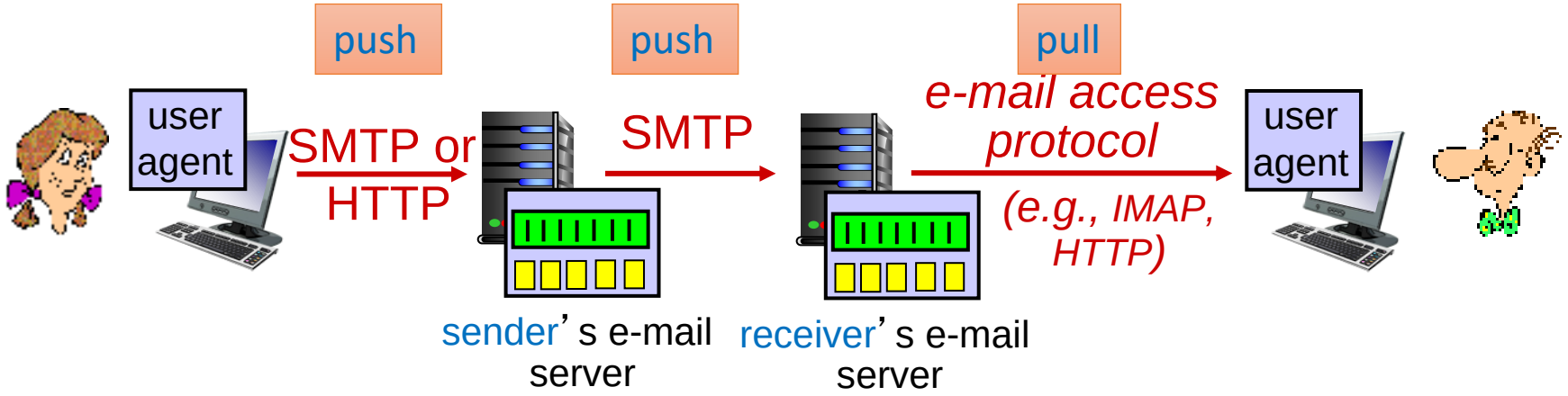
RFC 2822 defines *syntax* for e-mail message itself (like HTML defines syntax for web documents)

- header lines, e.g.,
  - From:
  - To:
  - Subject:

  these lines, within the body of the email message are different from SMTP MAIL FROM:, RCPT TO: commands!
- Body: the "message" , ASCII characters only



Retrieving email: mail access protocols



- SMTP: delivery/storage of e-mail messages to receiver's server (push)
- **mail access protocol**: retrieval from server（pull）
  - **IMAP:** Internet Mail Access Protocol [RFC 3501]: messages stored on server, IMAP provides retrieval, deletion, folders of stored messages on server
- HTTP: gmail, Hotmail, Yahoo!Mail, etc. provides web-based interface on top of SMTP (to send), IMAP (or POP) to retrieve e-mail messages

Application Layer: Overview

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- **The Domain Name System DNS**
- video streaming and content distribution networks
- socket programming with UDP and TCP



DNS: Domain Name System

people: many identifiers:

- SSN, name, passport #



Internet hosts, routers:

- IP address (32 bit) - used for addressing datagrams by routers
- "hostname", e.g., cs.umass.edu - used by humans

Q: how to map between IP address and name, and vice versa?

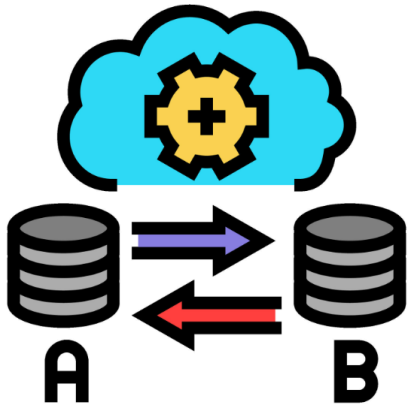
Domain Name System (DNS):

- *distributed database* implemented in hierarchy of many *name servers*
- *application-layer protocol:* hosts, DNS servers communicate to *resolve* names (address/name translation)
  - ***note:*** core Internet function, implemented as application-layer protocol
  - complexity at network's "edge"

DNS: services, structure

DNS services:

1. hostname-to-IP-address translation
   - Employed by other, e.g., HTTP, SMTP
2. host aliasing
   - canonical name: `relay1.west-coast.enterprise.com`
   - alias names: `enterprise.com`
3. mail server aliasing
4. load distribution
   - replicated Web servers:
   - many IP addresses
   - correspond to one name



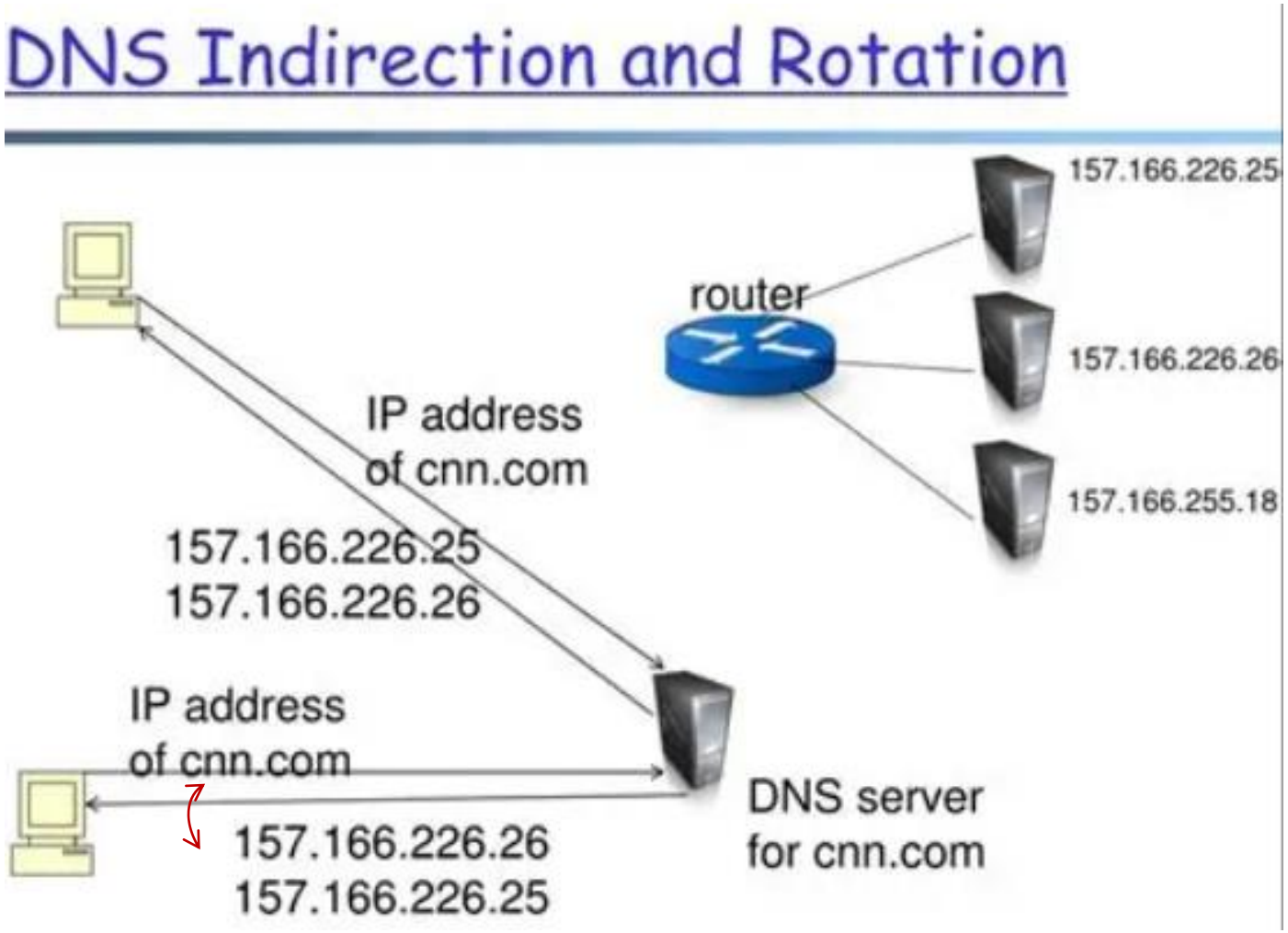
Q: Why not centralize DNS?

- single point of failure
- traffic volume
- distant centralized database
- maintenance

A: doesn't scale!

- Comcast DNS servers alone: 600B DNS queries/day
- Akamai DNS servers alone: 2.2T DNS queries/day

Load distribution among replicated servers



- replicated Web servers with a *set* of IP addresses

- rotate the ordering of the addresses within each reply
- **DNS rotation** is also used for e-mail so that multiple mail servers can have the same alias name.

Thinking about the DNS

humongous distributed database:

- ~ billion records, each simple

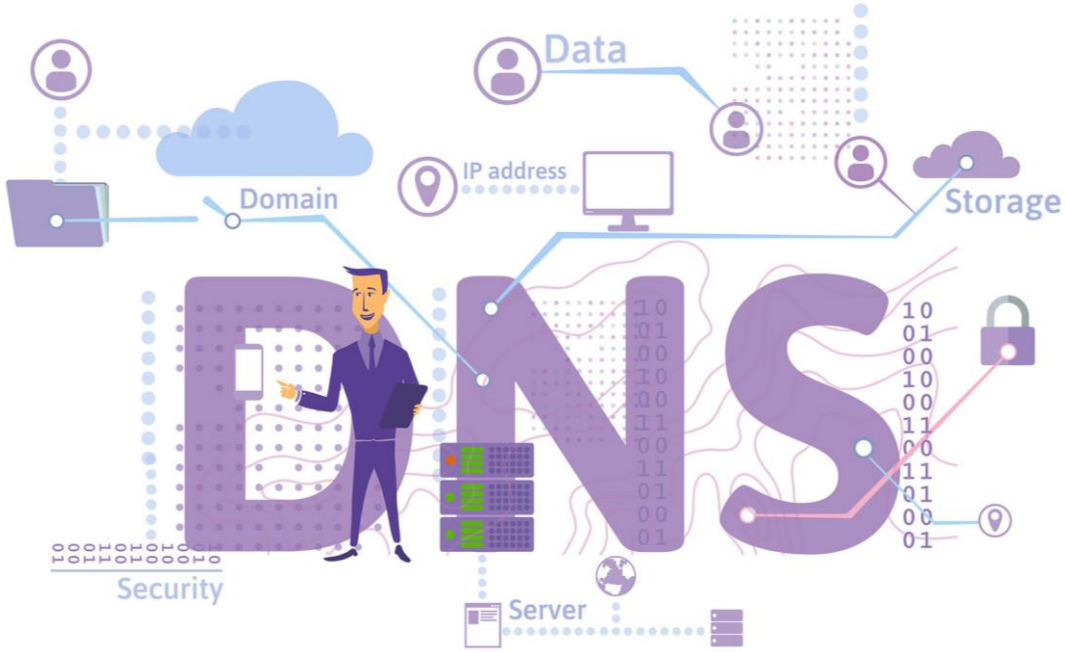
handles many *trillions* of queries/day:

- *many* more reads than writes
- *performance matters:* almost every Internet transaction interacts with DNS - msecs count!

organizationally, physically decentralized:

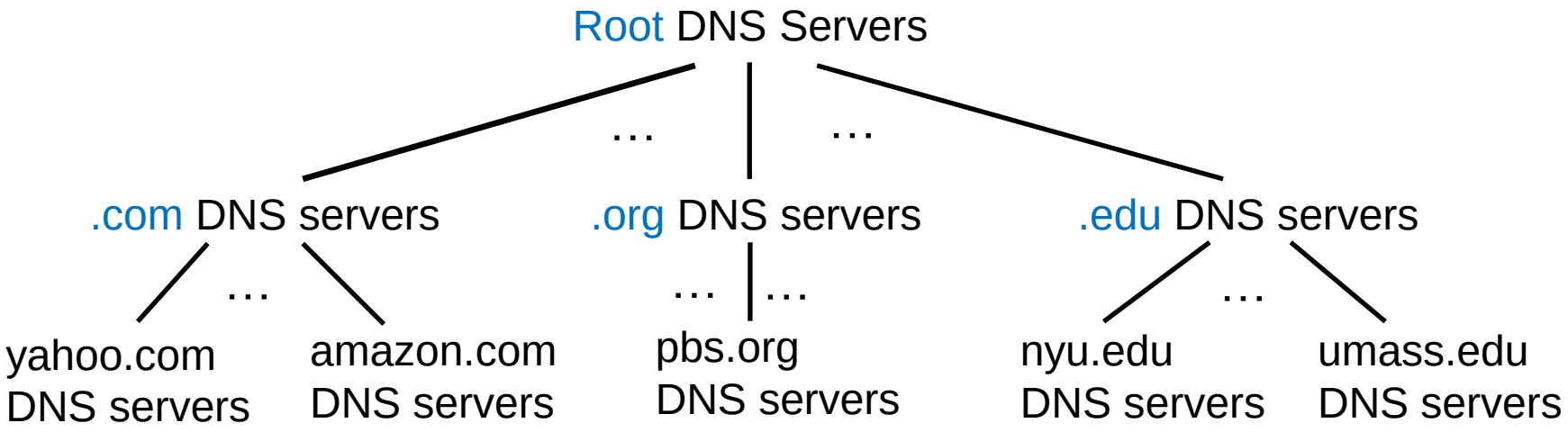
- millions of different organizations responsible for their records

"bulletproof": reliability, security





DNS: a distributed, hierarchical database

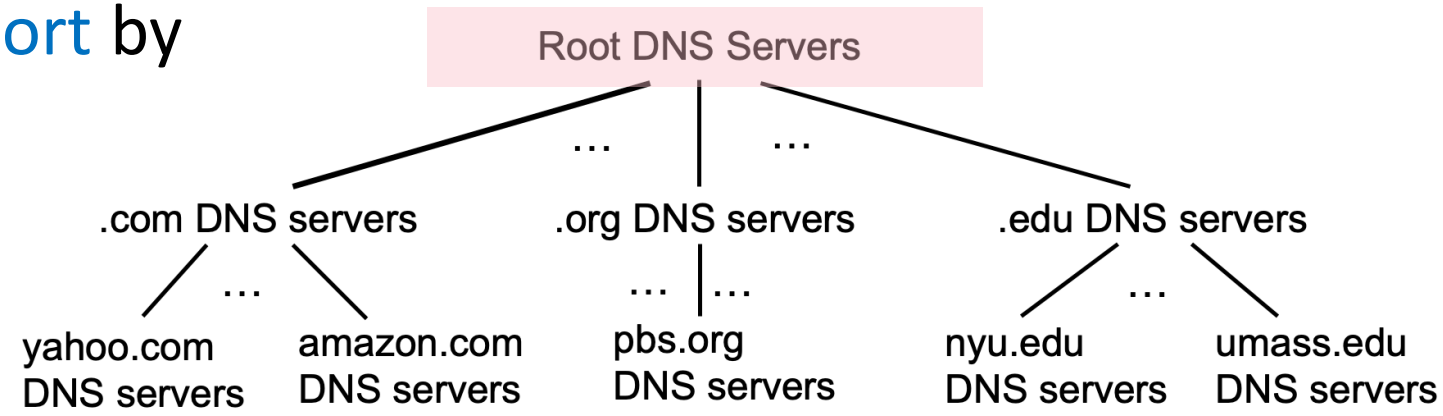


Client wants IP address for www.amazon.com; 1st approximation:

- client queries root server to find .com DNS server
- client queries .com DNS server to get amazon.com DNS server
- client queries amazon.com DNS server to get IP address for www.amazon.com

DNS: root name servers

- official, contact-of-last-resort by name servers that can not resolve name



DNS: root name servers

- official, contact-of-last-resort by name servers that can not resolve name
- *incredibly important* Internet function
  - Internet couldn't function without it!
  - DNSSEC – provides security (authentication, message integrity)
- ICANN manages root DNS domain

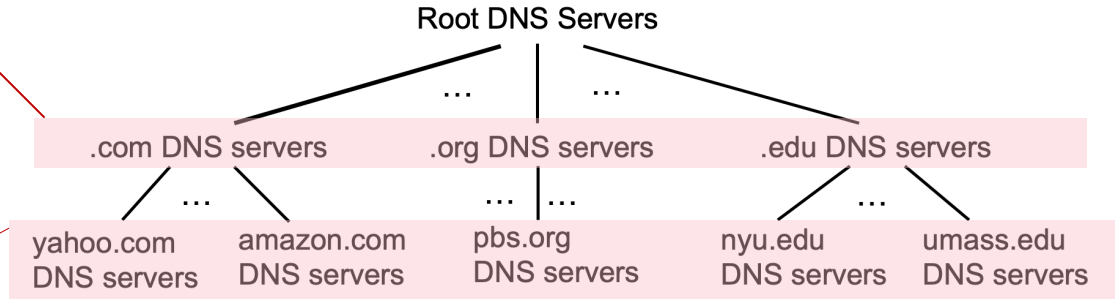


13 logical root name "servers" worldwide each "server" replicated many times (~200 servers in US)

Top-Level Domain, and authoritative servers

Top-Level Domain (TLD) servers:

- responsible for .com, .org, .net, .edu, .aero, .jobs, .museums, and all top-level country domains, e.g.: .cn, .uk, .fr, .ca, .jp
- Network Solutions: authoritative registry for .com, .net TLD
- Educause: .edu TLD



authoritative DNS servers:

- organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider

Local DNS name servers

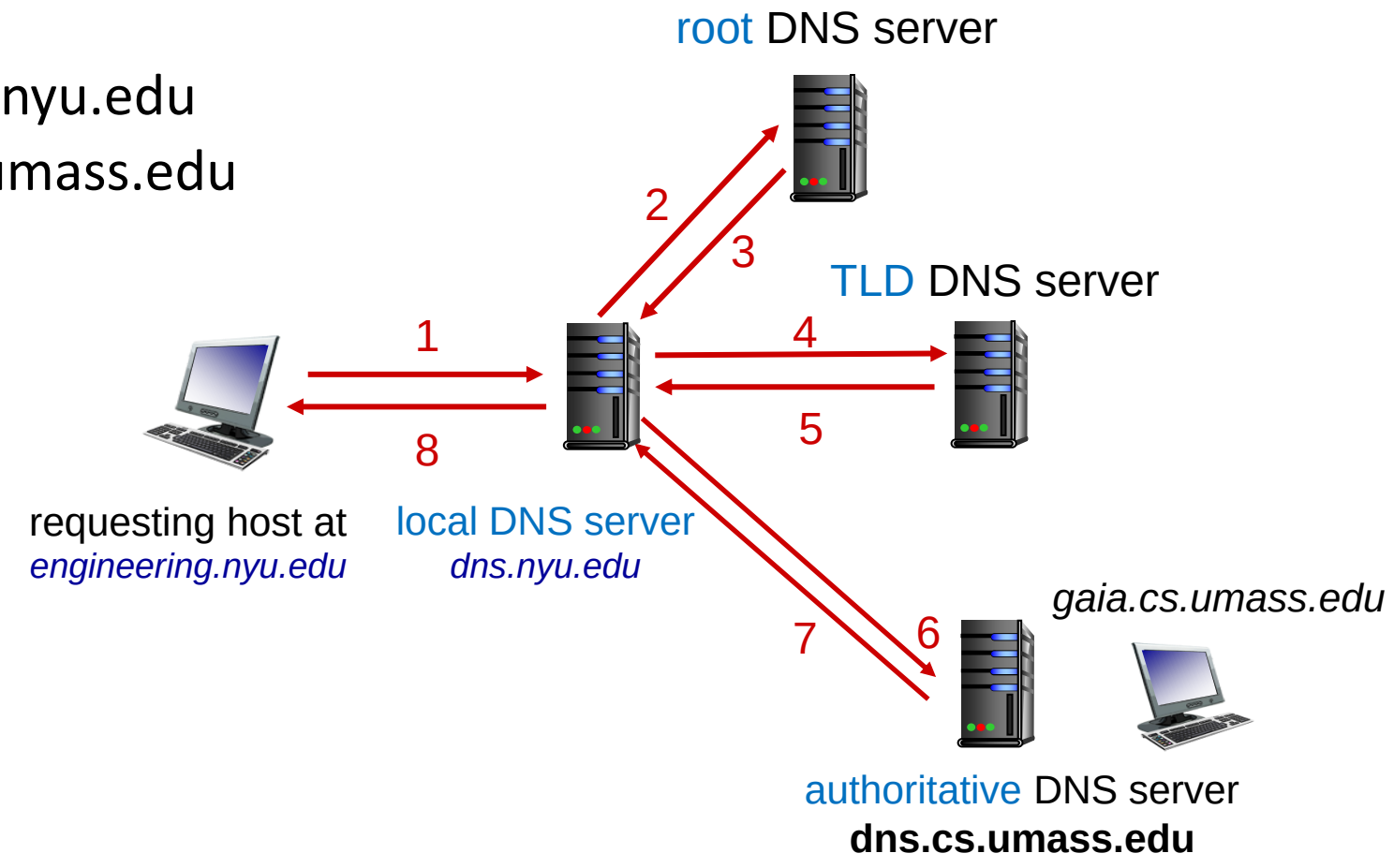
- when host makes DNS query, it is sent to its *local* DNS server
  - Local DNS server returns reply, answering:
    - from its local cache of recent name-to-address translation pairs (possibly out of date!)
    - forwarding request into DNS hierarchy for resolution
  - each ISP has local DNS name server; to find yours:
    - MacOS: `% scutil --dns`
    - Windows: `>ipconfig /all`
- **local DNS server** doesn't strictly belong to hierarchy

DNS name resolution: iterated query

Example: host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

Iterated query:

- contacted server replies with name of server to contact
- "I don't know this name, but ask this server"

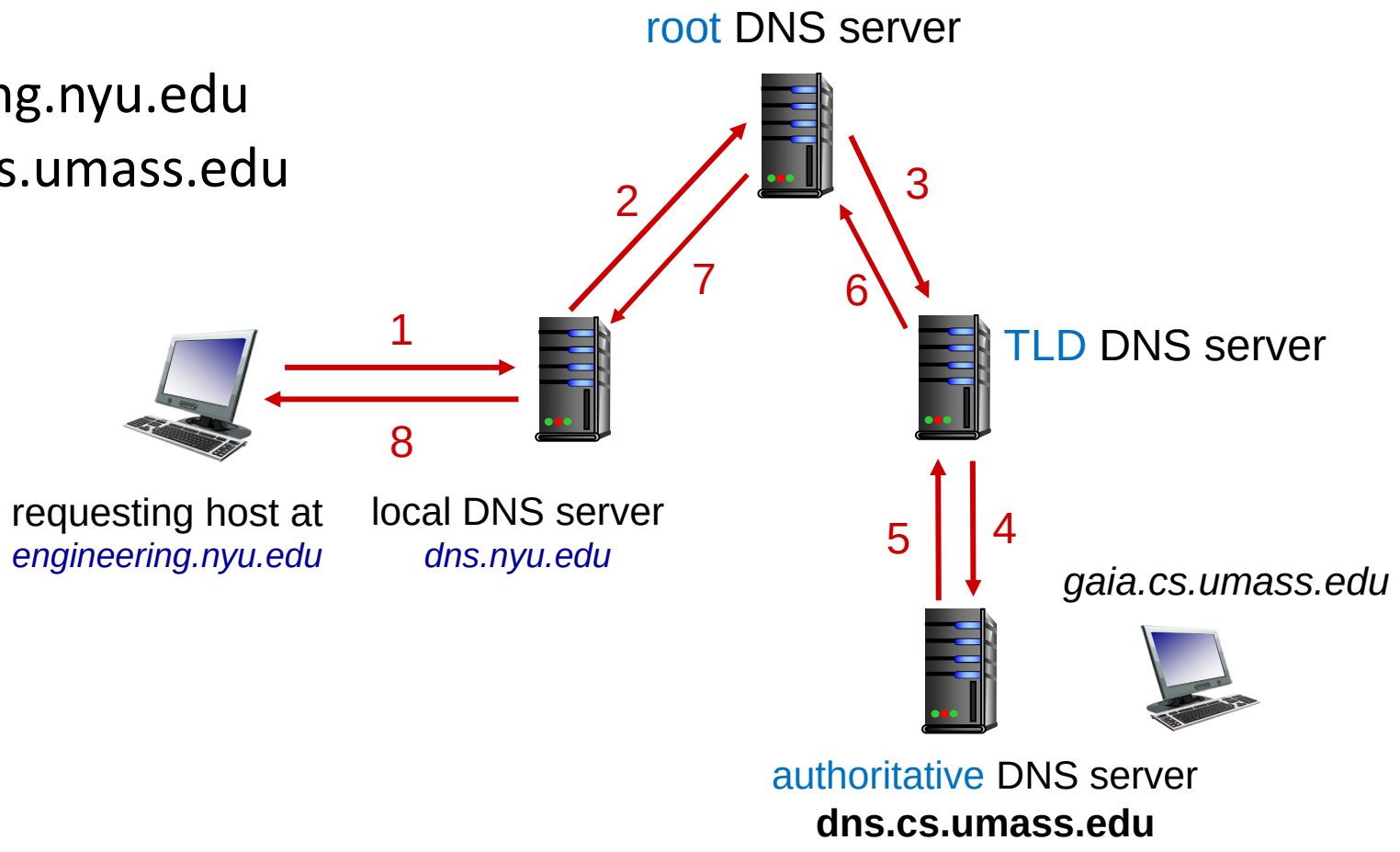


DNS name resolution: recursive query

Example: host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

Recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



Caching DNS Information



- once (any) name server learns mapping, it *caches* mapping, and *immediately* returns a cached mapping in response to a query
  - caching improves response time
  - cache entries timeout (disappear) after some time (TTL)
  - TLD servers typically cached in **local name servers**
- cached entries may be *out-of-date*
  - if named host changes IP address, may not be known Internet-wide until all TTLs expire!
  - *best-effort name-to-address translation!*



DNS records

DNS: distributed database storing Resource Records (RR)

four-tuple **RR format:** (`name, value, type, TTL`)

type=A

- `name` is hostname
- `value` is IP address

`(relay1.bar.foo.com, 145.37.93.126, A)`

type=NS

- `name` is domain (e.g., foo.com)
- `value` is hostname of authoritative Name Server for this domain

`(foo.com, dns.foo.com, NS)`

type=CNAME

- `name` is alias name for some "canonical" (the real) name
- www.ibm.com is really servereast.backup2.ibm.com
- `value` is canonical name

`(foo.com, relay1.bar.foo.com, CNAME)`

type=MX

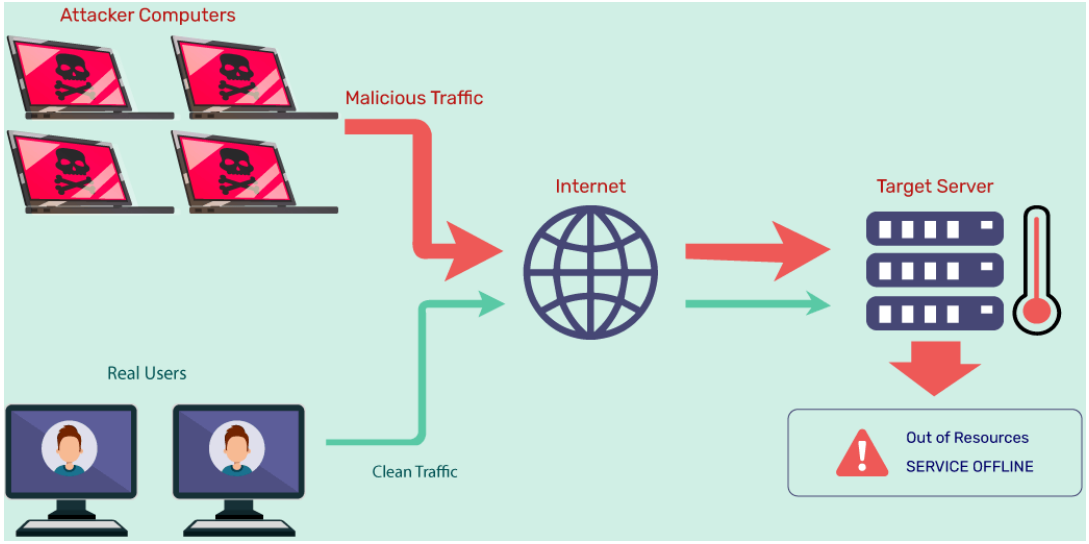
- `value` is name of SMTP mail server associated with `name`

`(foo.com, mail.bar.foo.com, MX)`

DNS security

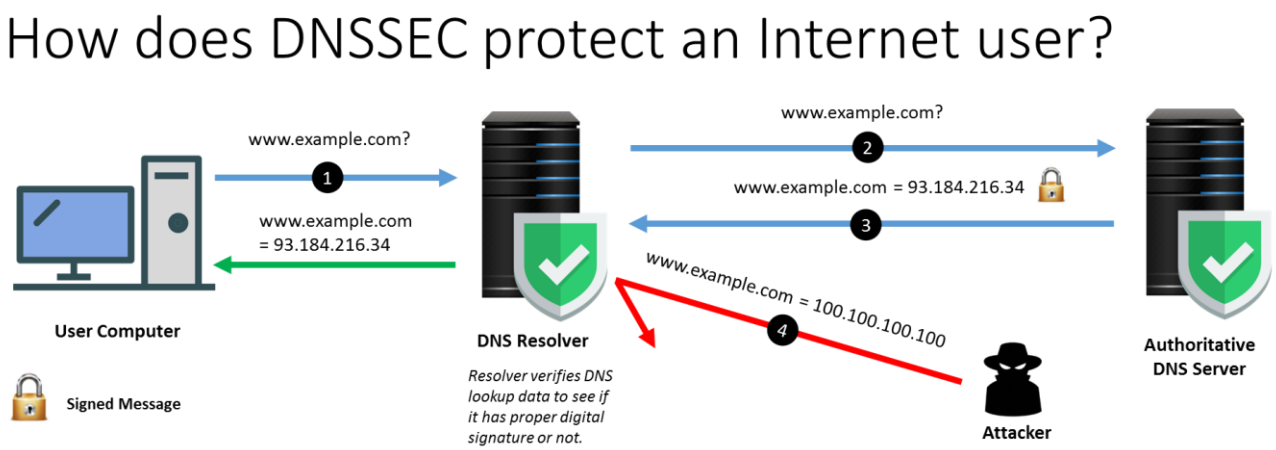
DDoS attacks

- bombard root servers with traffic
  - not successful to date
  - traffic filtering
  - **local DNS servers** cache IPs of TLD servers, allowing root server bypass
- bombard TLD servers
  - potentially more dangerous



Spoofing attacks

- intercept DNS queries, returning bogus replies
  - DNS cache poisoning
  - RFC 4033: DNSSEC authentication services



Application layer: overview

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS

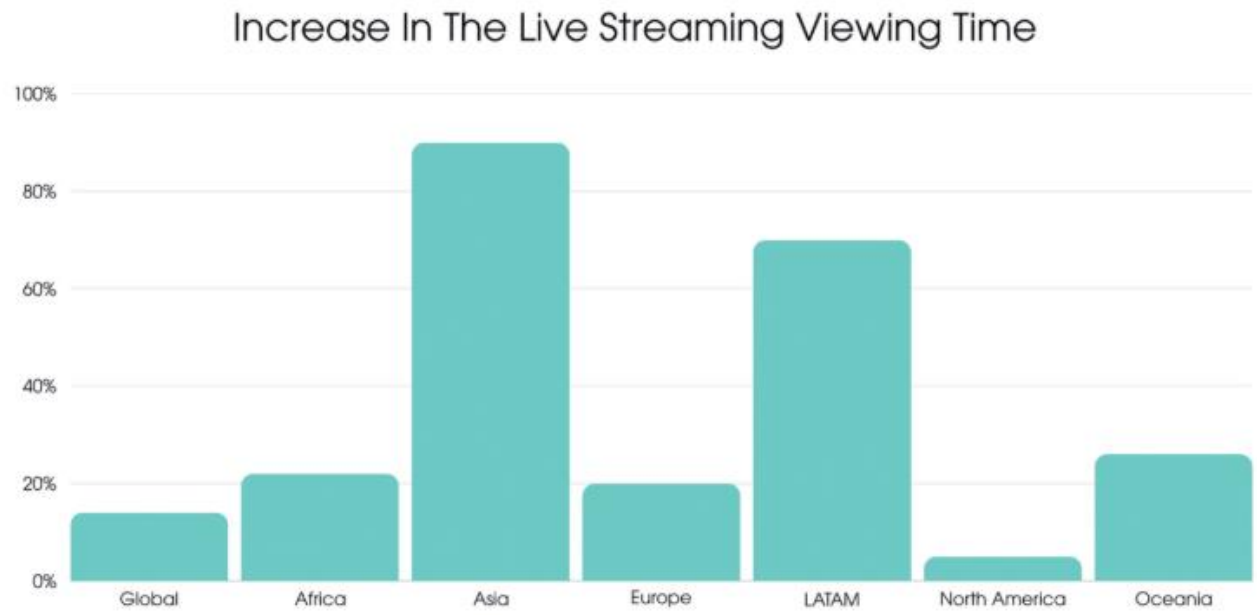
- video streaming and content distribution networks
- socket programming with UDP and TCP



Video Streaming and CDNs: context

- stream video traffic: major consumer of Internet bandwidth
- *challenge:* heterogeneity
  - different users have different capabilities (e.g., wired versus mobile; bandwidth rich versus bandwidth poor)
- *solution:* distributed, application-level infrastructure

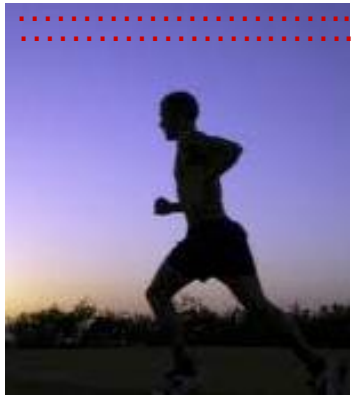




Multimedia: video

- video: sequence of images displayed at constant rate
  - e.g., 24 images/sec
- digital image: array of pixels
  - each pixel represented by bits
- coding: use redundancy *within* and *between* images to decrease # bits used to encode image
  - spatial (within image)
  - temporal (from one image to next)

spatial coding example: instead of sending *N* values of same color (all purple), send only two values: color value (*purple*) *and number of repeated values* (N)



frame *i*



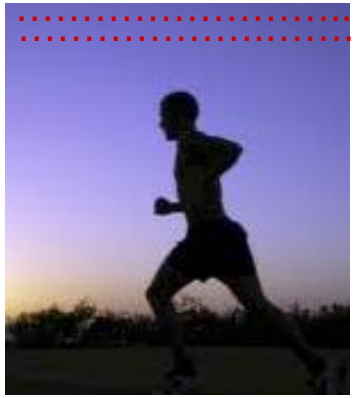
frame *i+1*

temporal coding example: instead of sending complete frame at i+1, send only differences from frame i

Multimedia: video

- CBR: (constant bit rate): video encoding rate fixed
- VBR: (variable bit rate): video encoding rate changes as amount of spatial, temporal coding changes
- examples:
  - MPEG1 (CD-ROM) 1.5 Mbps
  - MPEG2 (DVD) 3-6 Mbps
  - MPEG4 (often used in Internet, 64Kbps – 12 Mbps)

spatial coding example: instead of sending *N* values of same color (all purple), send only two values: color value (*purple*) *and number of repeated values* (N)



frame *i*

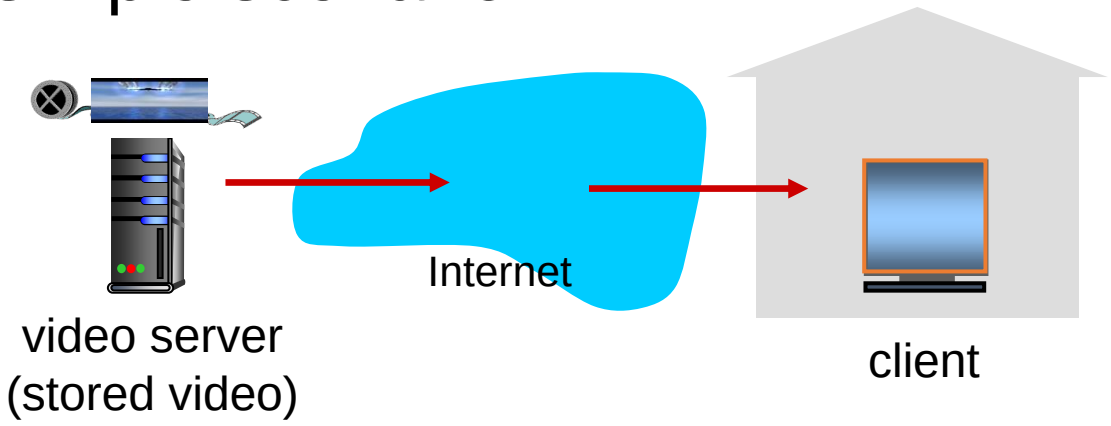


frame *i+1*

temporal coding example: instead of sending complete frame at i+1, send only differences from frame i

Streaming stored video

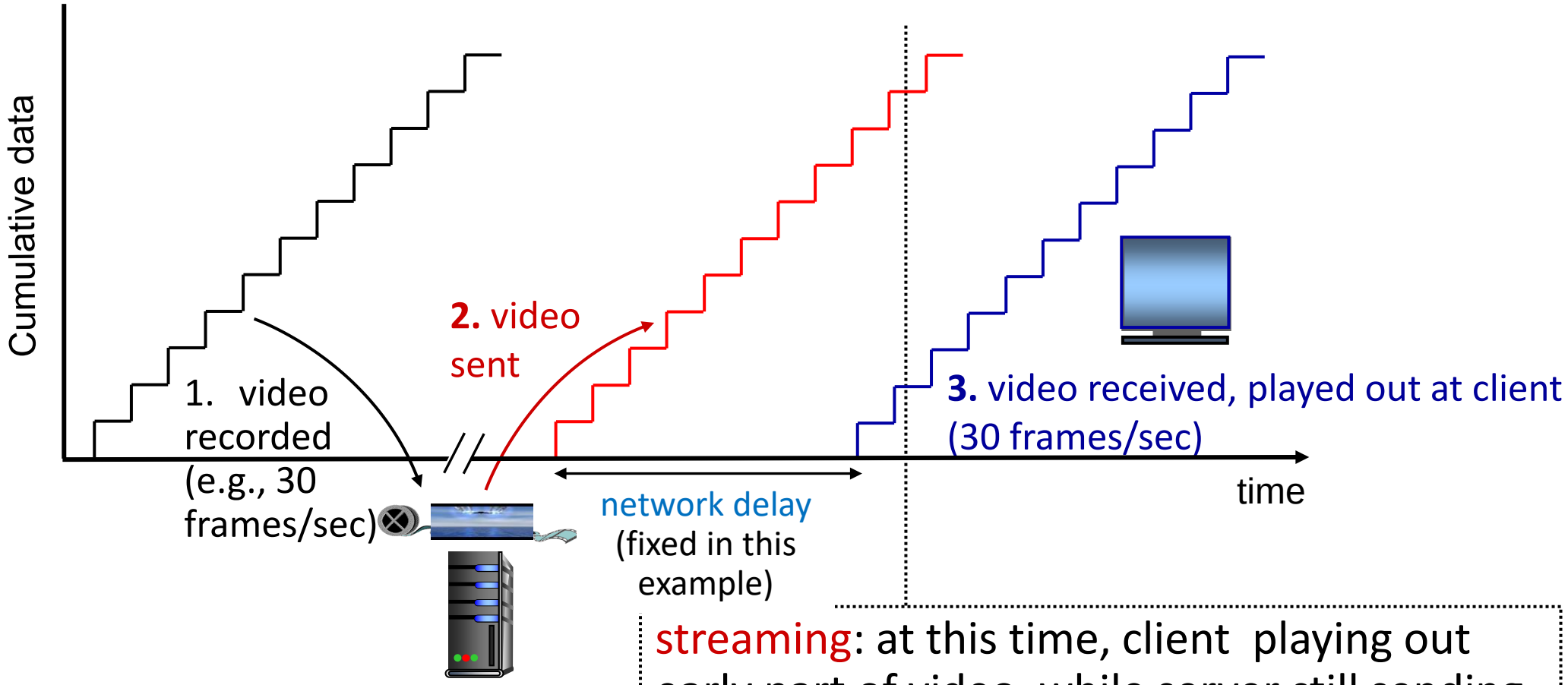
simple scenario:



Main challenges:

- server-to-client bandwidth will *vary* over time, with changing network congestion levels (in house, access network, network core, video server)
- packet loss, delay due to congestion will delay playout, or result in poor video quality

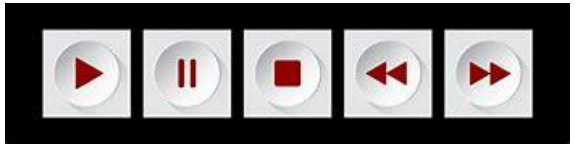
Streaming stored video



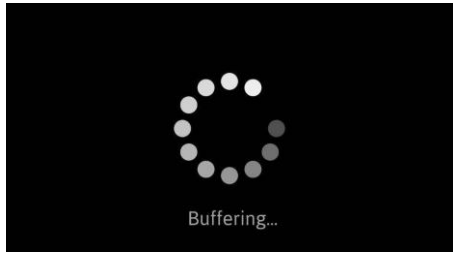
streaming: at this time, client playing out early part of video, while server still sending later part of video

Streaming stored video: challenges

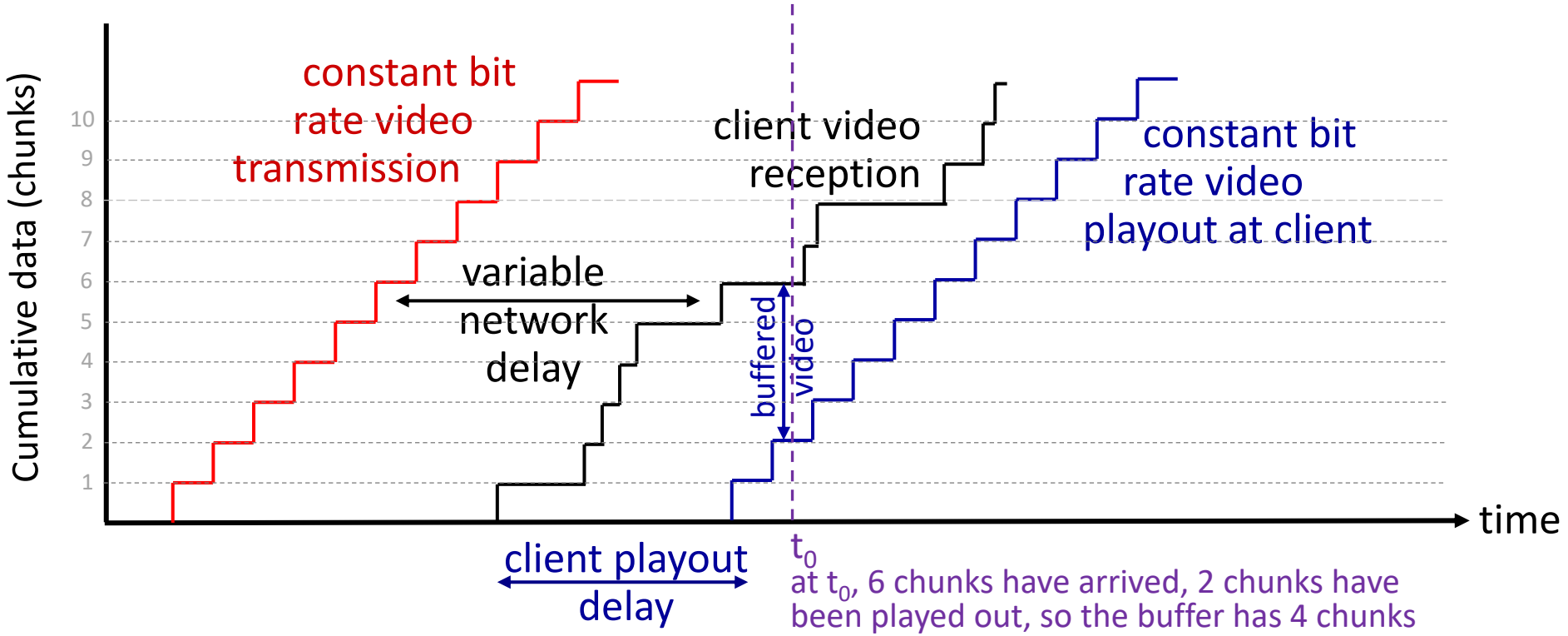
- continuous playout constraint: during client video playout, playout timing must match original timing
  - … but network delays are variable (jitter), so will need client-side buffer to match continuous playout constraint
- other challenges:



  - client interactivity: pause, fast-forward, rewind, jump through video
  - video packets may be lost, retransmitted



Streaming stored video: playout buffering



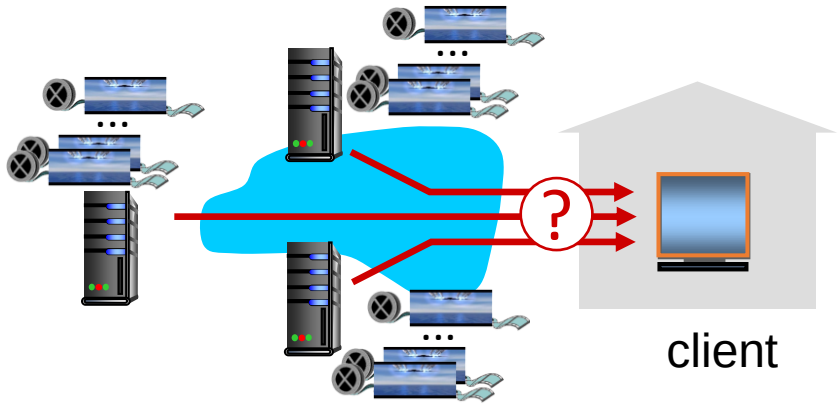
- *client-side buffering and playout delay:* compensate for network-added delay, delay jitter

Streaming multimedia: DASH

*D*ynamic, *A*daptive
*S*treaming over *H*TTP

server:

- divides video file into multiple chunks
- each chunk encoded at multiple different rates
- different rate encodings stored in different files
- files replicated in various CDN nodes
- *manifest file:* provides URLs for different chunks



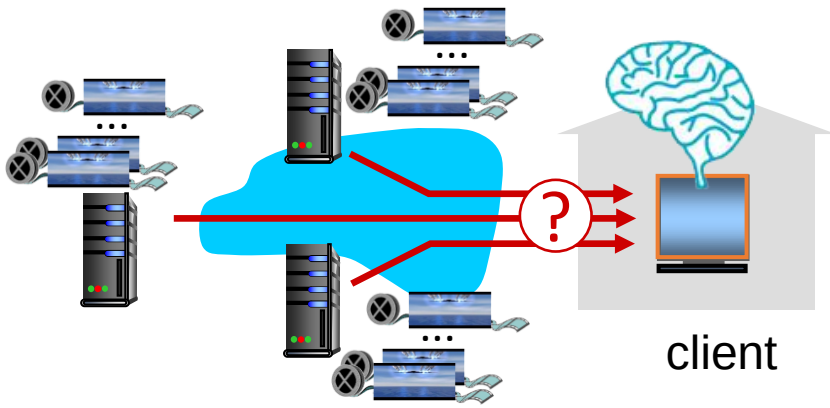
client:

- periodically estimates server-to-client bandwidth
- consulting manifest, requests one chunk at a time
  - chooses maximum coding rate sustainable given current bandwidth
  - can choose different coding rates at different points in time (depending on available bandwidth at time), and from different servers



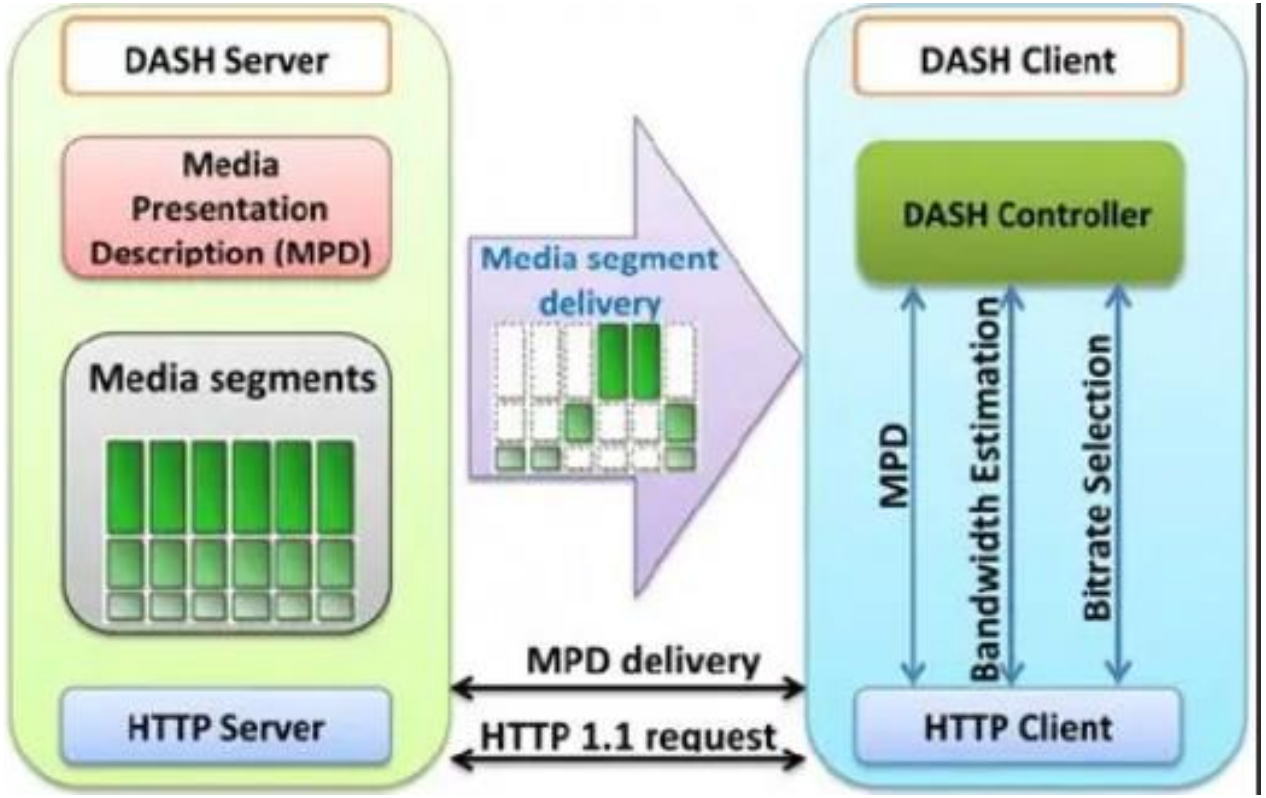
Streaming multimedia: DASH

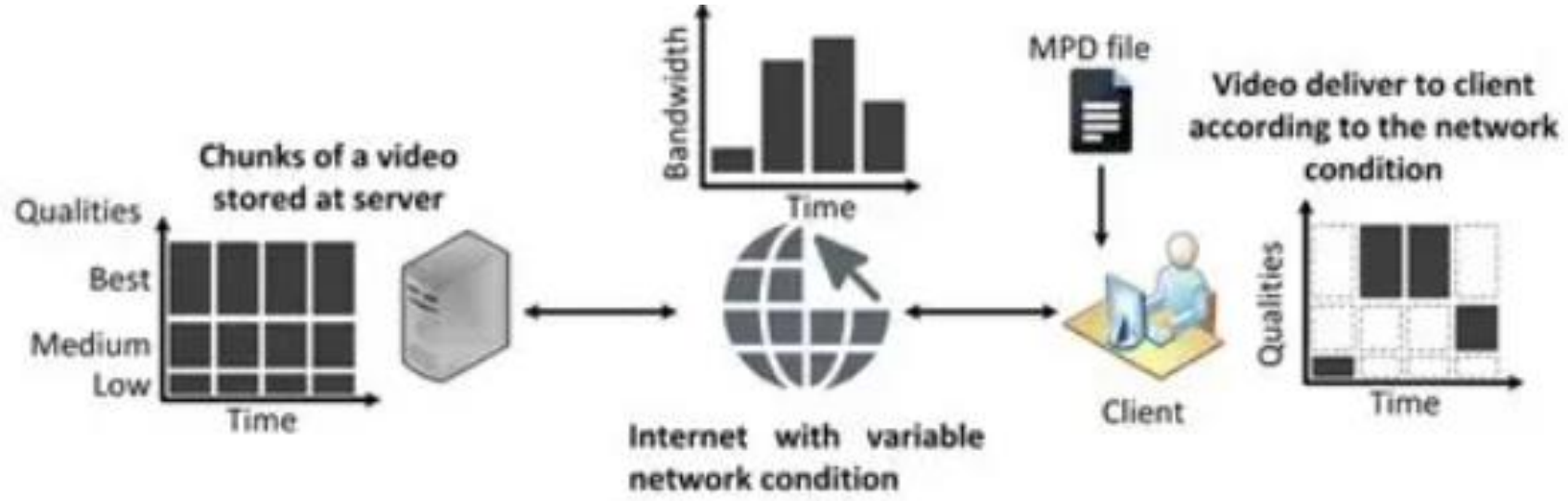
- *"intelligence"* at client: client determines
  - *when* to request chunk (so that buffer starvation, or overflow does not occur)
  - *what encoding rate* to request (higher quality when more bandwidth available)
  - *where* to request chunk (can request from URL server that is "close" to client or has high available bandwidth)



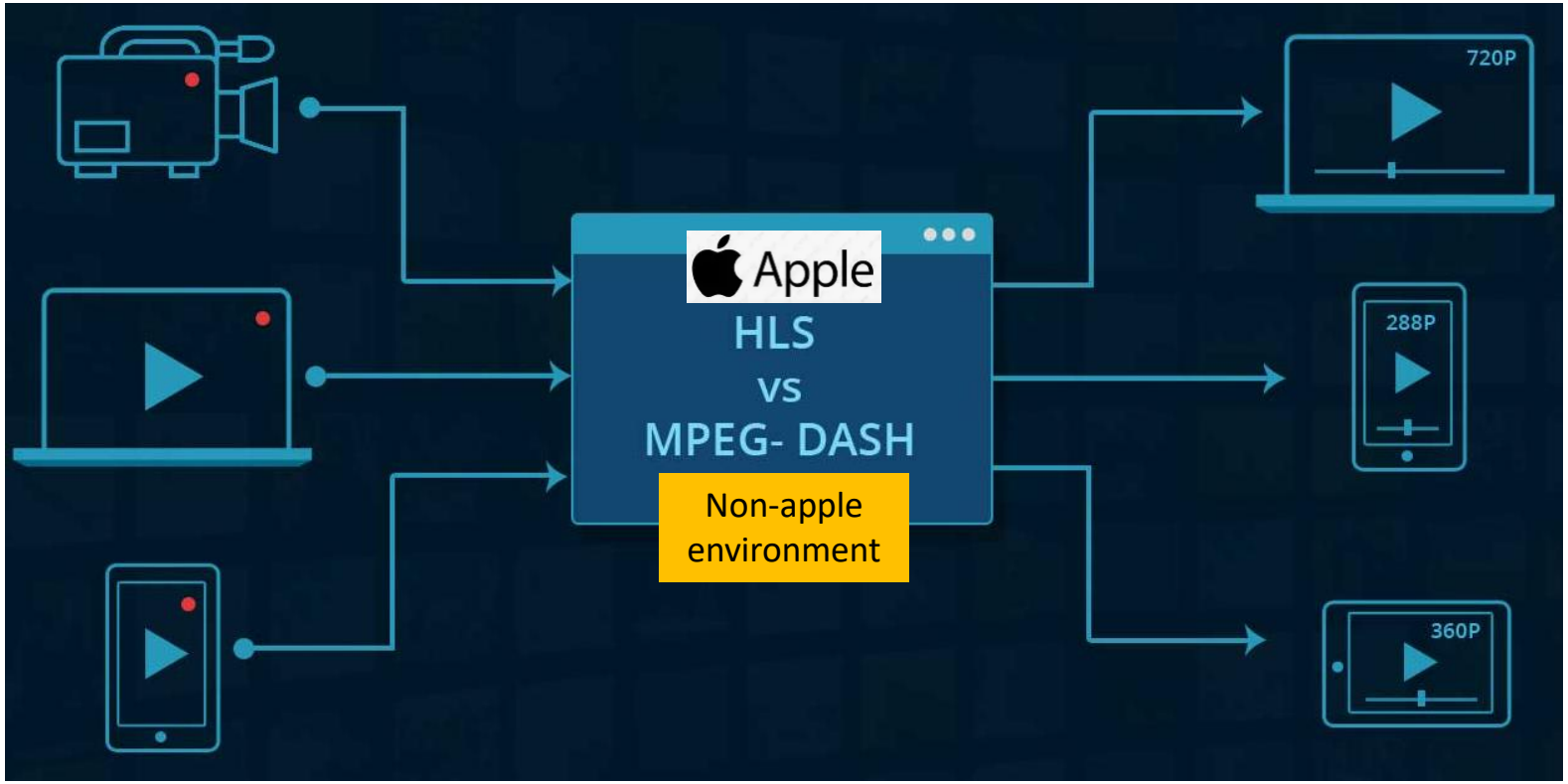
Streaming video = encoding + DASH + playout buffering

*D*ynamic, *A*daptive *S*treaming over *H*TTP (DASH)





Live Streaming



Parameter	HLS	HESP	MPEG-DASH
Ease of Implementation	Easy	Moderate	Difficult
Device Support	Excellent	Moderate	Good
Video Quality	Good	Excellent	Excellent
Bitrate Flexibility	Limited	Good	Excellent
Latency	Comparatively Higher	Low	Low
Technical Complexity	Simple	Moderate	Complex

- **HESP**: High-Efficiency Streaming Protocol.
- **HLS**: HTTP Live Streaming
  - LL-HLS: Low-Latency HTTP Live Streaming Protocol

Content distribution networks (CDNs)

challenge: how to stream content (selected from millions of videos) to hundreds of thousands of *simultaneous* users?

- *option 1:* single, large "mega-server"
  - single point of failure
  - point of network congestion
  - long (and possibly congested) path to distant clients



….quite simply: this solution *doesn't scale*

Content distribution networks (CDNs)

challenge: how to stream content (selected from millions of videos) to hundreds of thousands of *simultaneous* users?

- *option 2:* store/serve multiple copies of videos at multiple geographically distributed sites *(CDN)*

- ***enter deep:*** push CDN servers deep into many access networks
  - close to users
  - Earlier Akamai: 240,000 servers deployed in > 120 countries (2015)



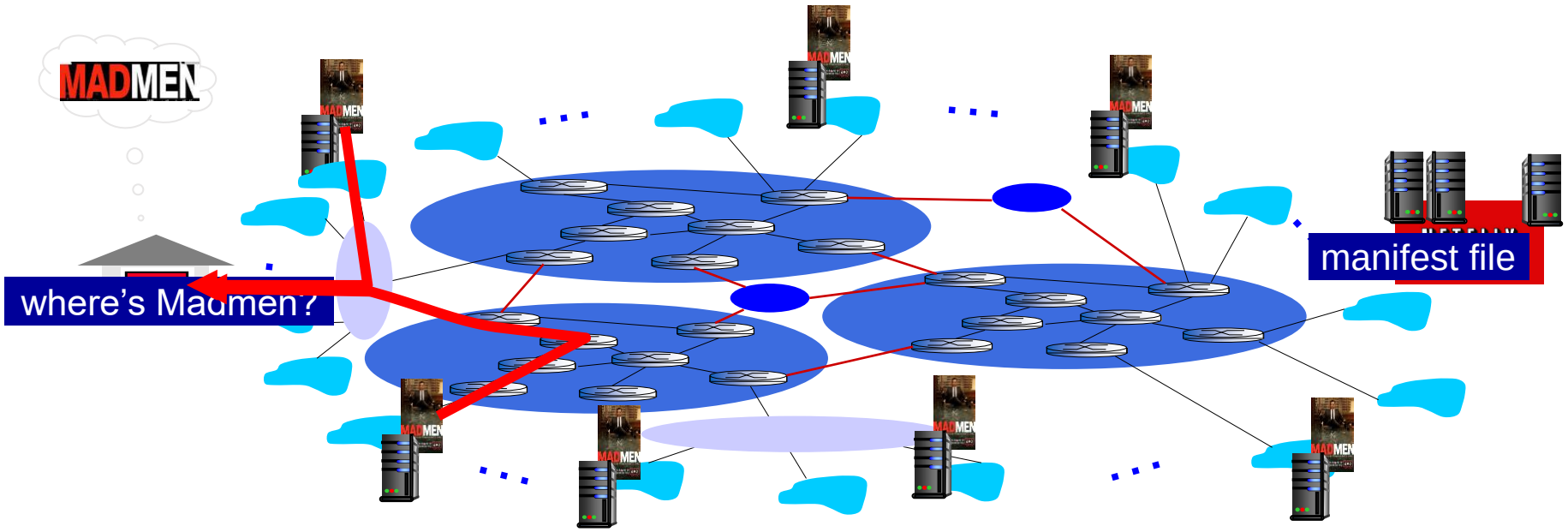
- ***bring home:*** smaller number (10's) of larger clusters in POPs near access nets
  - typically place their clusters in **IXPs**
  - Cloudflare, Amazon CloudFront, Google Cloud CDN





Content distribution networks (CDNs)

- CDN: stores copies of content (e.g. MADMEN) at CDN nodes
- subscriber requests content, service provider returns manifest
  - using manifest, client retrieves content at highest supportable rate
  - may choose different rate or copy if network path congested



Content distribution networks (CDNs)



Internet host-host communication as a service

OTT challenges: coping with a congested Internet from the "edge"

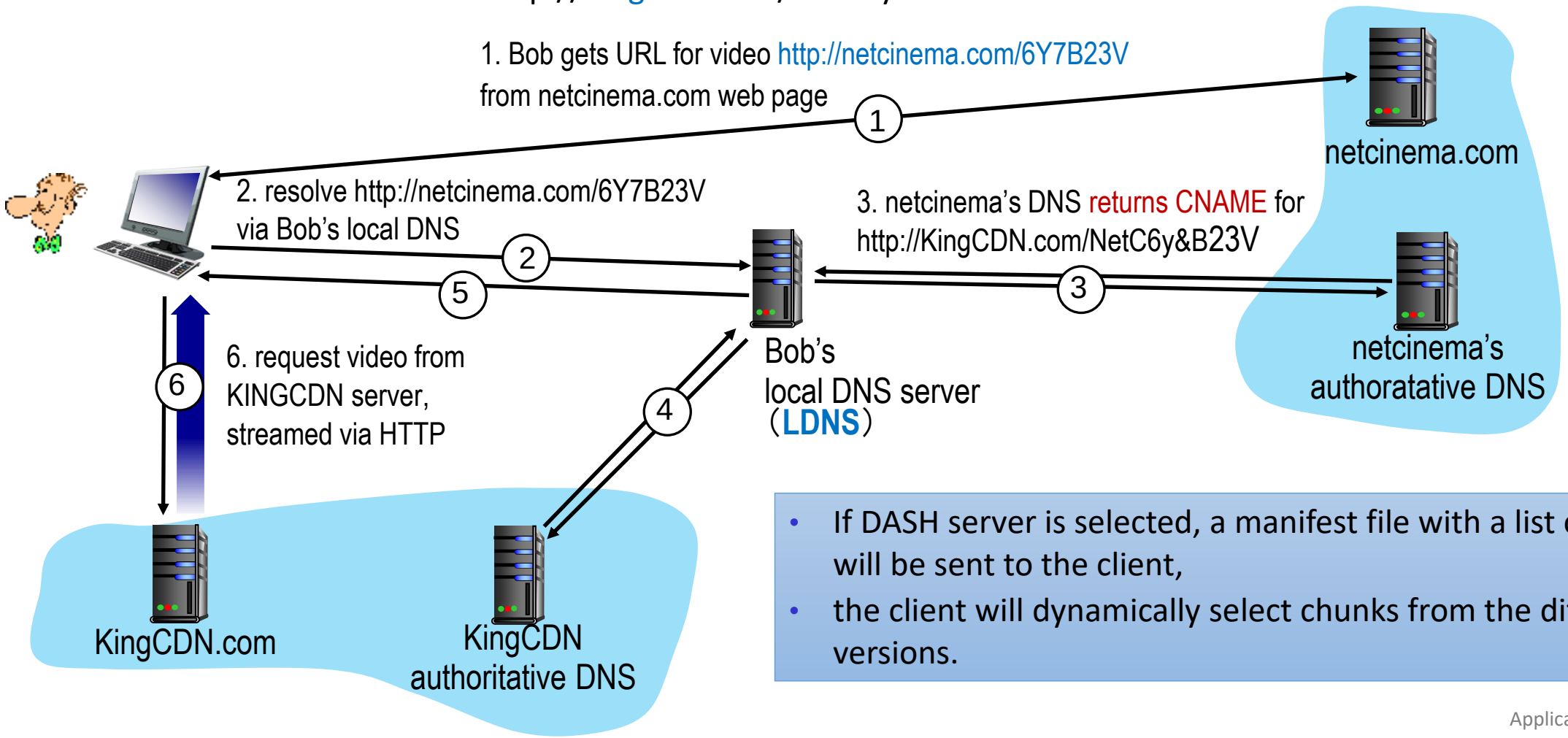
- what content to place in which CDN node?
- from which CDN node to retrieve content? At which rate?

CDN content access: a closer look

DNS redirects a user's request to a CDN server

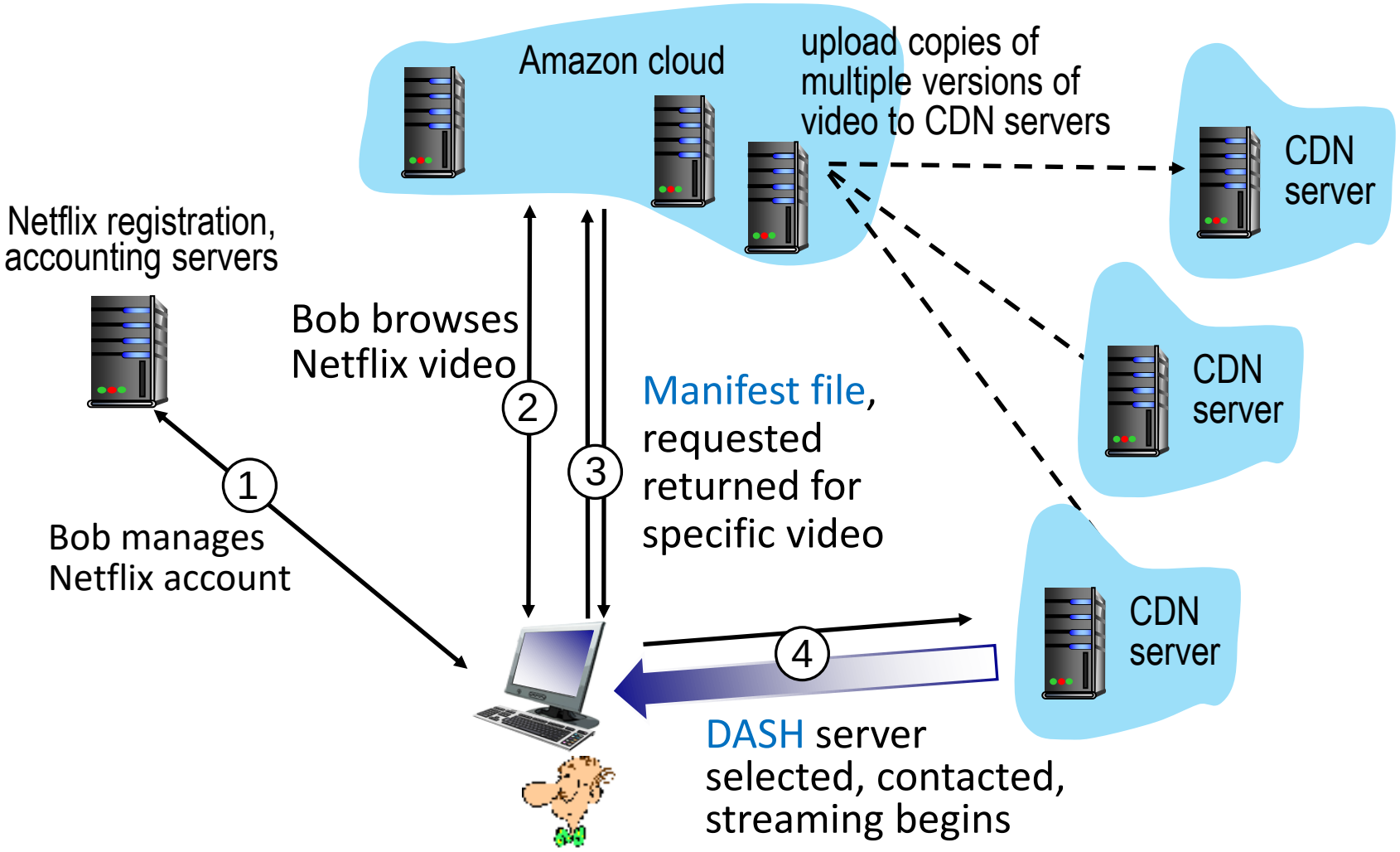
Bob (client) requests video http://netcinema.com/6Y7B23V

- video stored in CDN at http://KingCDN.com/NetC6y&B23V



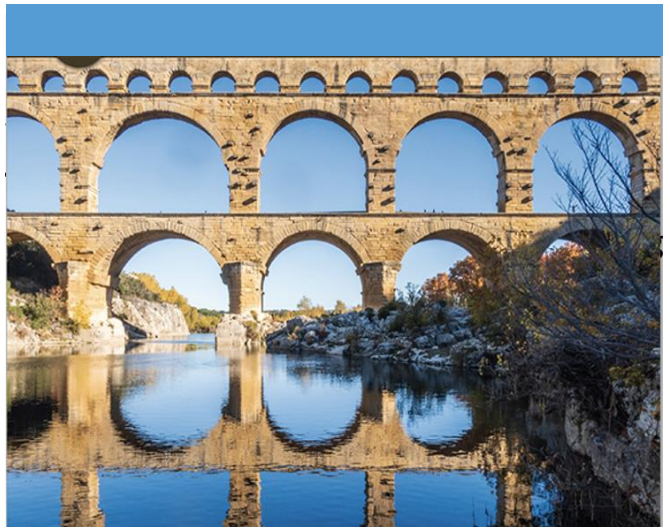
- If DASH server is selected, a manifest file with a list of URLs will be sent to the client,
- the client will dynamically select chunks from the different versions.

Case study: Netflix



Application Layer: Overview

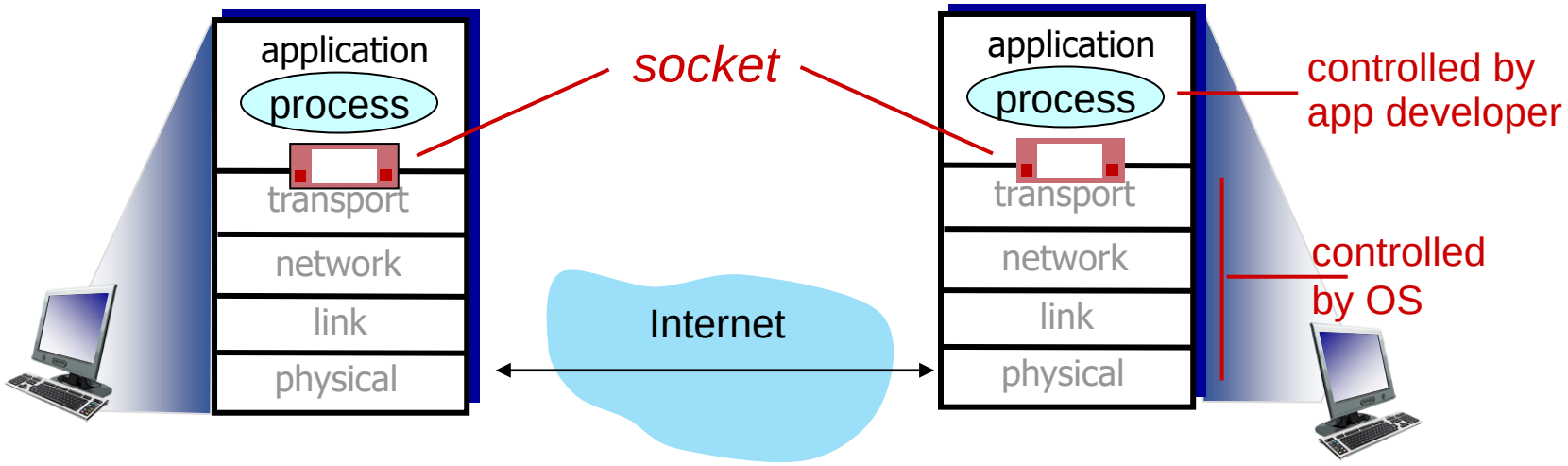
- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS
- video streaming and content distribution networks
- socket programming with UDP and TCP



Socket programming

goal: learn how to build client/server applications that communicate using sockets

socket: door between application process and end-end-transport protocol



Socket programming

Two socket types for two transport services:

- *UDP:* unreliable datagram
- *TCP:* reliable, byte stream-oriented

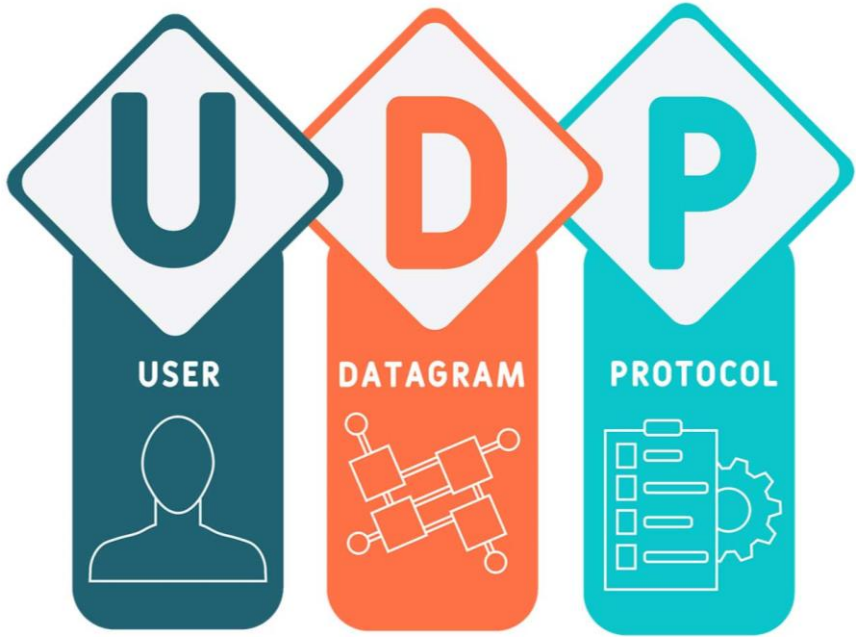
Application over UDP Example:

1. client reads a line of characters (data) from its keyboard and sends data to server
2. server receives the data and converts characters to uppercase
3. server sends modified data to client
4. client receives modified data and displays line on its screen

Socket programming with UDP

UDP: no "connection" between client and server:

- no handshaking before sending data
- sender explicitly attaches IP destination address and port # to each packet
- receiver extracts sender IP address and port# from received packet

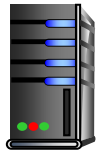


UDP: transmitted data may be lost or received out-of-order

Application viewpoint:

- UDP provides *unreliable* transfer of groups of bytes ("datagrams") between client and server processes

Client/server socket interaction: UDP



server (running on serverIP)

create socket, port= x:
serverSocket =
socket(AF_INET,SOCK_DGRAM)

read datagram from
serverSocket

write reply to
serverSocket
specifying
client address,
port number

client 

create socket:
clientSocket =
socket(AF_INET,SOCK_DGRAM)

Create datagram with server IP address
and port=x; send datagram via
clientSocket

read datagram from
clientSocket

close
clientSocket

AF_INET indicates that the underlying network is using IPv4.
SOCK_DGRAM means a UDP socket

Socket programming with TCP

Client must contact server

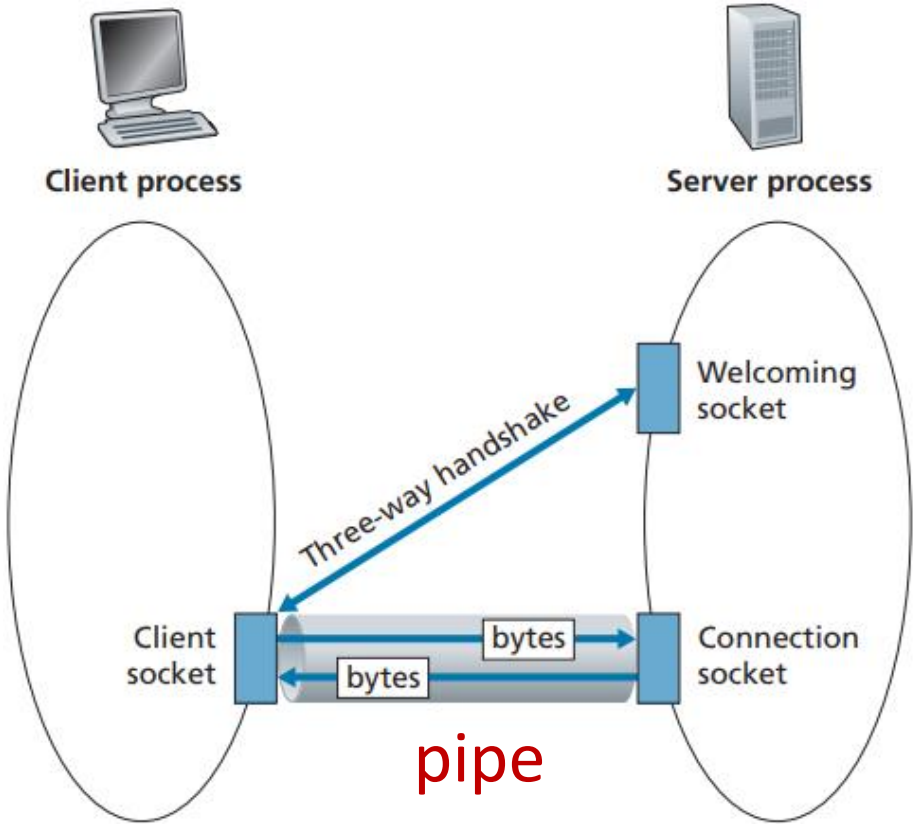
- server process must first be running
- server must have created socket (door) that welcomes client's contact

Client contacts server by:

- Creating TCP socket, specifying IP address, port number of server process
- *when client creates socket:* client TCP establishes connection to server TCP

when contacted by client, *server TCP creates **NEW** socket* for server process to communicate with that particular client

- allows server to talk with multiple clients
- *source* port numbers used to distinguish clients (more in Chap 3)



The TCPServer process has two sockets

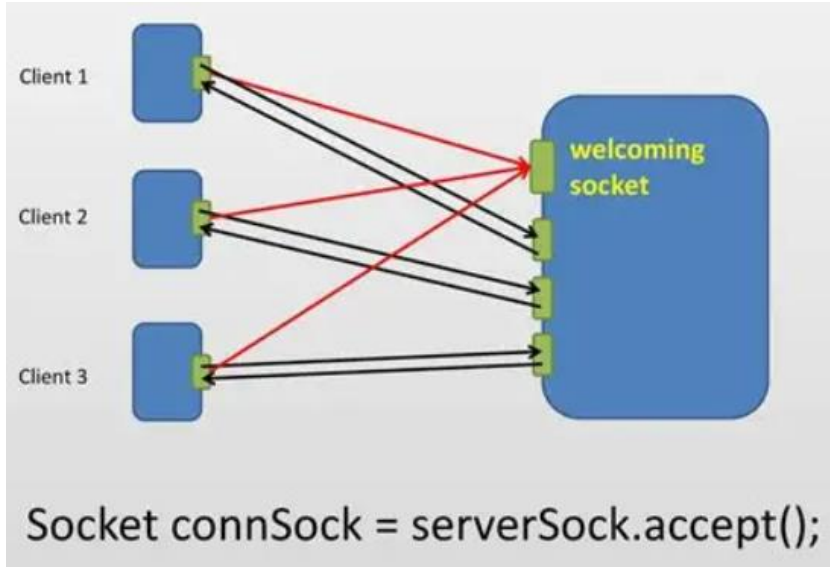
Application viewpoint

TCP provides reliable, in-order byte-stream transfer ("pipe") between client and server processes

Client/server socket interaction: TCP

server (running on hostid)

client



create socket,
port=**x**, for incoming request:
serverSocket = socket()

wait for incoming
connection request
connectionSocket =
serverSocket.accept()

read request from
connectionSocket

write reply to
connectionSocket

close
connectionSocket

TCP
connection setup

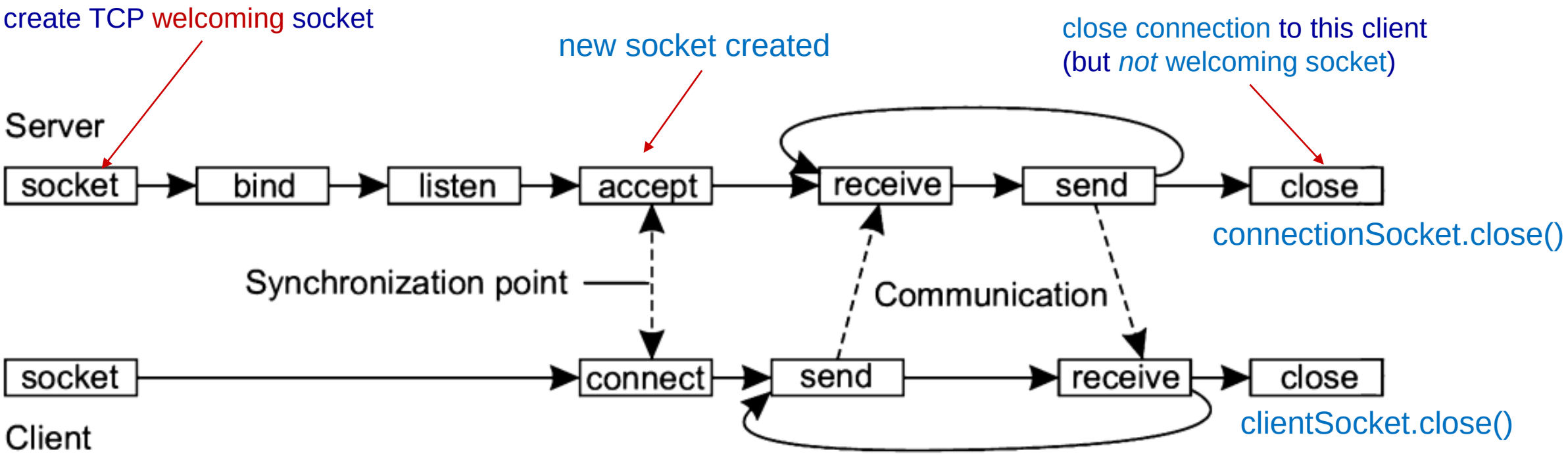
create socket,
connect to **hostid**, port=**x**
clientSocket = socket()

send request using
clientSocket

read reply from
clientSocket

close
clientSocket

TCP client/server



Chapter 2: Summary

our study of network application layer is now complete!

- application architectures
  - client-server
  - P2P
- application service requirements:
  - reliability, bandwidth, delay
- Internet transport service model
  - connection-oriented, reliable: TCP
  - unreliable, datagrams: UDP
- specific protocols:
  - HTTP
  - SMTP, IMAP
  - DNS
  - P2P: BitTorrent
- video streaming, CDNs
- socket programming: TCP, UDP sockets

Chapter 2: Summary

Most importantly: learned about *protocols*!

- typical request/reply message exchange:
  - client requests info or service
  - server responds with data, status code
- message formats:
  - *headers*: fields giving info about data
  - *data:* info(payload) being communicated

important themes:

- centralized vs. decentralized
- stateless vs. stateful
- scalability
- reliable vs. unreliable message transfer
- "complexity at network edge"

HOMEWORK



1. Compare the pros and cons of proprietary software and open source software.
2. Communication models adopted in WeChat and QQ
3. Cookies' regulation
4. Differences among http/1.0, 1.1, 2.0, 3.0
5. Summarize the relationships and connections among the different types of DNS Resource Records: Type A, NS, MX, and CNAME
6. CDNs typically adopt one of two different server placement philosophies. Name and briefly compare them. Which one has bigger market share?
7. The principle of DASH

Wireshark Lab

